30-Day Django Mastery — Nexora CRM

Oct 15, 2026 · @tanvik

30 days, 40 code exercises a day (1,200 total). Every exercise feeds one project: a working Nexora CRM with leads, clients, brokers, call logs and reports by Day 30.

How each day works

Each Day tab = 40 exercises in four blocks, solutions at the bottom. Tick each box once your code runs.

Block	Exercises	Focus
A. Drills	1–10	Syntax and commands, one concept each
B. Core	11–20	The day's main Django feature
C. Nexora CRM	21–30	Same feature applied to the CRM project
D. Challenge	31–40	Edge cases, refactors, tests

Rules:

- One project for all 30 days: folder `nexora-crm`, project package `django_project`. Each day builds on the last.
- Type the code, don't paste. Solve first, then check the solutions.
- End every day with `python manage.py test` and `git commit -m "day N"`.
- Target: Django 5.2 LTS, Python 3.10+.
- Days arrive 3 at a time; say "next" for the following batch.

Roadmap

Five phases: foundations, data, views and users, advanced Django, then APIs and a two-day capstone.

Day	Phase	Topic	CRM piece built
1	Foundations	Project, apps, settings, function views	`nexora_crm` project, `core` + `leads` apps, first pages and JSON
2	Foundations	URL routing and the request object	Converters, namespaces, query params, JSON create/update/delete

Day	Phase	Topic	CRM piece built
3	Foundations	Templates, inheritance, static files	CRM layout with sidebar, lead list/detail pages, dashboard cards
4	Foundations	Class-based views	Days 1–3 pages rebuilt as CBVs, RedirectView, view mixins
5	Foundations	Models and migrations	Lead, Client, Broker models with choices, Meta, constraints
6	Data	ORM CRUD and the shell	Seed 200 leads; create, get, filter, update, delete
7	Data	Lookups, Q, F, ordering, values	Lead search, stale-lead finders, status counts
8	Data	Relationships	Custom user (agents) → Lead FK, call logs, Client↔TradingAccount↔Deposit, M2M tags, self-FK broker tree
9	Data	Aggregation and annotation	Conversion per agent, deposits per broker, Case/When, Subquery
10	Data	Query performance and transactions	select_related, prefetch_related, bulk ops, indexes, atomic
11	Data	Admin customization	Full back office: list_display, filters, search, inlines, actions
12	Views and users	ListView, DetailView, pagination	Paginated lead list, lead detail with call history
13	Views and users	Forms and validation	LeadForm, ModelForms, clean methods, phone/email validation
14	Views and users	Editing views and messages	Lead create/edit/delete, status changes, flash messages
15	Views and users	Authentication	Login/logout, password change/reset, admin-created accounts only (user model built Day 8)
16	Views and users	Permissions and groups	Agents see own leads, managers see their team
17	Views and users	Search, filters, sorting	Lead filter bar: status, source, date range, sort

Day	Phase	Topic	CRM piece built
18	Views and users	Testing	Model, view, form and permission test suite
19	Advanced	Files: uploads, CSV import/export	Bulk lead import, CSV export, KYC document uploads
20	Advanced	Managers, QuerySets, signals	Custom lead QuerySets, audit log via signals
21	Advanced	Middleware, context processors, template tags	Activity logger, global counters, CRM template filters
22	Advanced	Settings, env vars, PostgreSQL, logging	environs, Postgres connection, split settings
23	Advanced	Email and management commands	Lead-assignment emails, `daily_report` command
24	Advanced	Caching and profiling	Debug toolbar, query counts, cached dashboards
25	APIs and capstone	Django REST Framework 1	Serializers, APIView, lead API
26	APIs and capstone	Django REST Framework 2	ViewSets, routers, token auth, filtering, pagination
27	APIs and capstone	JSON + JavaScript front end	fetch() with CSRF, Chart.js dashboard from annotate()
28	APIs and capstone	Security and deployment	check --deploy, WhiteNoise, gunicorn, Docker basics
29	APIs and capstone	Capstone 1	Dialer call-log flow, SOP compliance checks, broker reports
30	APIs and capstone	Capstone 2	Full test pass, polish, deploy, review

Day 1 — Project, apps, settings, function views

Goal: a running `nexora-crm` project with `core` and `leads` apps, 15+ function views, JSON and CSV responses, and first tests.

A. Drills (1–10)

- [ ] **1.** Create folder `nexora-crm`, a virtual env `.venv` inside it, and activate it.
- [ ] **2.** Install Django 5.2 and print the installed version from the command line.
- [ ] **3.** Create project `django_project` in the current folder (no nested duplicate folder).
- [ ] **4.** Apply the default migrations, then run the dev server on port 8080.
- [ ] **5.** Create two apps: `core` and `leads`.
- [ ] **6.** Register both apps in `INSTALLED_APPS` using their AppConfig paths.
- [ ] **7.** Set the project time zone to Dubai and confirm `USE_TZ` is on.
- [ ] **8.** Write a `.gitignore` (venv, database, pycache, `.env`), `git init`, first commit.
- [ ] **9.** Save installed packages to `requirements.txt`.
- [ ] **10.** Run Django's system check and list migration status for every app.

B. Core (11–20) — function views in `core`

- [ ] **11.** View `home` returning the text `Nexora CRM`.
- [ ] **12.** Create `core/urls.py`, map `""` to `home` (name `home`), include it from the project urls.
- [ ] **13.** View `health` at `/health/` returning `OK` with an explicit status 200.
- [ ] **14.** View `dashboard` at `/dashboard/` returning the HTML `<h1>Dashboard</h1>`.
- [ ] **15.** View `about` at `/about/` returning plain text (not HTML) via `content_type`.
- [ ] **16.** View `api_info` at `/api/info/` returning JSON `{"app": "Nexora CRM", "version": "0.1"}`.
- [ ] **17.** View `secret` at `/secret/` that raises a 404 with the message `Not here`.
- [ ] **18.** View at `/home/` that redirects to the `home` URL by name.
- [ ] **19.** View at `/request-info/` returning the request's method and path, e.g. `GET /request-info/`.
- [ ] **20.** View at `/get-only/` that allows GET only; confirm other methods get 405.

C. Nexora CRM (21–30) — `leads` app, in-memory data

- [ ] **21.** In `leads/views.py` define a module-level `LEADS` list of 5 dicts: `id, name, phone, status, source` (UAE numbers like `971501234567`; statuses `new/contacted/qualified/lost`).
- [ ] **22.** View `lead_list` returning all lead names separated by `<br>`.
- [ ] **23.** Create `leads/urls.py` with `app_name = "leads"`; mount it at `/leads/`.
- [ ] **24.** Helper `get_lead(lead_id)` that raises 404 if missing; view `lead_detail` at `/leads/<id>/` returning `name — status`.
- [ ] **25.** View `lead_count` at `/leads/count/` returning `{"total": 5}`.
- [ ] **26.** View `leads_by_status` at `/leads/status/<status>/` returning matching names.
- [ ] **27.** View `lead_list_json` at `/leads/json/` returning the whole list as JSON.
- [ ] **28.** Make `lead_list` filter by an optional `?source=` query parameter.
- [ ] **29.** View `lead_phone` at `/leads/<id>/phone/` returning a masked number: `9715****4567`.
- [ ] **30.** View `lead_menu` at `/leads/menu/` returning an HTML list of links built with `reverse()` (list, JSON, count).

D. Challenge (31–40)

- [ ] **31.** Custom 404 page: a `custom_404` view in `core` wired with `handler404`. See it with `DEBUG = False`.
- [ ] **32.** View `leads_csv` at `/leads/export/` that downloads `leads.csv` using the `csv` module.
- [ ] **33.** Make `leads_by_status` return 400 for any status outside the four valid ones.
- [ ] **34.** Write a `log_request` decorator printing method, path, status code and duration in ms; apply it to `lead_list`.
- [ ] **35.** Add `CRM_COMPANY_NAME = "Nexora"` to settings and read it in a `welcome` view.
- [ ] **36.** View `quick_add` at `/leads/quick-add/`: GET shows instructions, POST adds a lead from form field `name` and returns it as JSON with status 201. Test with curl.
- [ ] **37.** `core/tests.py`: test that `home` returns 200 and contains `Nexora CRM`.
- [ ] **38.** `leads/tests.py`: test that lead 999 returns 404.
- [ ] **39.** Test that `/api/info/` JSON has `app == "Nexora CRM"`.
- [ ] **40.** In `python manage.py shell`, use Django's test `Client` to GET `/leads/` and print the status code and first 50 bytes.

Solutions

A. Drills

1–5.

```
mkdir nexora-crm && cd nexora-crm
python -m venv .venv
.venv\Scripts\Activate.ps1           # macOS/Linux: source .venv/bin/activate

python -m pip install "django~=5.2"   # 2
python -m django --version

django-admin startproject django_project .   # 3

python manage.py migrate              # 4
python manage.py runserver 8080

python manage.py startapp core        # 5
python manage.py startapp leads
```

6–7.

```
# django_project/settings.py
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "core.apps.CoreConfig",
    "leads.apps.LeadsConfig",
]

TIME_ZONE = "Asia/Dubai"
USE_TZ = True
```

8–10.

```
# .gitignore
.venv/
db.sqlite3
__pycache__/
.env
```

```
git init
git add -A
git commit -m "day 1: project setup"

python -m pip freeze > requirements.txt   # 9

python manage.py check                    # 10
python manage.py showmigrations
```

B. Core

11–20. All views in one file, then the urls.

```
# core/views.py
from django.http import HttpResponse, JsonResponse, Http404
from django.shortcuts import redirect
from django.views.decorators.http import require_GET


def home(request):                                  # 11
    return HttpResponse("Nexora CRM")


def health(request):                                # 13
    return HttpResponse("OK", status=200)


def dashboard(request):                             # 14
    return HttpResponse("<h1>Dashboard</h1>")


def about(request):                                 # 15
    return HttpResponse("Nexora CRM — internal tool",
content_type="text/plain")


def api_info(request):                              # 16
    return JsonResponse({"app": "Nexora CRM", "version": "0.1"})


def secret(request):                                # 17
    raise Http404("Not here")


def old_home(request):                              # 18
```

```
    return redirect("home")


def request_info(request):                              # 19
    return HttpResponse(f"{request.method} {request.path}")


@require_GET                                            # 20
def get_only(request):
    return HttpResponse("GET only")
```

```
# core/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name="home"),
    path("health/", views.health, name="health"),
    path("dashboard/", views.dashboard, name="dashboard"),
    path("about/", views.about, name="about"),
    path("api/info/", views.api_info, name="api_info"),
    path("secret/", views.secret, name="secret"),
    path("home/", views.old_home, name="old_home"),
    path("request-info/", views.request_info, name="request_info"),
    path("get-only/", views.get_only, name="get_only"),
]
```

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("leads/", include("leads.urls")),   # added in 23
    path("", include("core.urls")),
]
```

```
# 20: OPTIONS skips CSRF, so you see the 405 (a POST would hit CSRF first → 403)
curl -i -X OPTIONS http://127.0.0.1:8000/get-only/
```

C. Nexora CRM

21–30.

```
# leads/views.py
from django.http import HttpResponse, JsonResponse, Http404
from django.urls import reverse

LEADS = [                                                          # 21
    {"id": 1, "name": "Aisha Khan", "phone": "971501234567", "status": "new",
"source": "facebook"},
    {"id": 2, "name": "Omar Haddad", "phone": "971502345678", "status":
"contacted", "source": "google"},
    {"id": 3, "name": "Priya Nair", "phone": "971503456789", "status":
"qualified", "source": "referral"},
    {"id": 4, "name": "Daniel Ortiz", "phone": "971504567890", "status":
"new", "source": "google"},
    {"id": 5, "name": "Mei Lin", "phone": "971505678901", "status": "lost",
"source": "facebook"},
]


def get_lead(lead_id):                                             # 24
    for lead in LEADS:
        if lead["id"] == lead_id:
            return lead
    raise Http404("Lead not found")


def lead_list(request):                                            # 22 + 28
    source = request.GET.get("source")
    leads = [l for l in LEADS if not source or l["source"] == source]
    return HttpResponse("<br>".join(l["name"] for l in leads))


def lead_detail(request, lead_id):                                 # 24
    lead = get_lead(lead_id)
    return HttpResponse(f"{lead['name']} — {lead['status']}")


def lead_count(request):                                           # 25
    return JsonResponse({"total": len(LEADS)})


def leads_by_status(request, status):                              # 26
    names = [l["name"] for l in LEADS if l["status"] == status]
    return HttpResponse("<br>".join(names) or "No leads")


def lead_list_json(request):                                       # 27
```

```
    return JsonResponse(LEADS, safe=False)   # safe=False: top-level list


def lead_phone(request, lead_id):                                    # 29
    phone = get_lead(lead_id)["phone"]
    return HttpResponse(phone[:4] + "*" * (len(phone) - 8) + phone[-4:])


def lead_menu(request):                                              # 30
    links = [
        ("All leads", reverse("leads:list")),
        ("JSON", reverse("leads:json")),
        ("Count", reverse("leads:count")),
    ]
    items = "".join(f'<li><a href="{url}">{label}</a></li>' for label, url in
links)
    return HttpResponse(f"<ul>{items}</ul>")
```

```
# leads/urls.py                                                      # 23
from django.urls import path
from . import views

app_name = "leads"
urlpatterns = [
    path("", views.lead_list, name="list"),
    path("count/", views.lead_count, name="count"),
    path("json/", views.lead_list_json, name="json"),
    path("menu/", views.lead_menu, name="menu"),
    path("status/<str:status>/", views.leads_by_status, name="by_status"),
    path("<int:lead_id>/", views.lead_detail, name="detail"),
    path("<int:lead_id>/phone/", views.lead_phone, name="phone"),
]
```

D. Challenge

31.

```
# core/views.py
def custom_404(request, exception):
    return HttpResponse("<h1>Nexora CRM — page not found</h1>", status=404)

# django_project/urls.py (module level, below urlpatterns)
handler404 = "core.views.custom_404"

# settings.py — to see it
DEBUG = False
ALLOWED_HOSTS = ["127.0.0.1", "localhost"]
```

32.

```
# leads/views.py
import csv

def leads_csv(request):
    response = HttpResponse(content_type="text/csv")
    response["Content-Disposition"] = 'attachment; filename="leads.csv"'
    writer = csv.DictWriter(response, fieldnames=["id", "name", "phone",
"status", "source"])
    writer.writeheader()
    writer.writerows(LEADS)
    return response

# leads/urls.py
path("export/", views.leads_csv, name="export"),
```

33.

```
from django.http import HttpResponseBadRequest

VALID_STATUSES = {"new", "contacted", "qualified", "lost"}

def leads_by_status(request, status):
    if status not in VALID_STATUSES:
        return HttpResponseBadRequest(f"Unknown status: {status}")
    names = [l["name"] for l in LEADS if l["status"] == status]
    return HttpResponse("<br>".join(names) or "No leads")
```

34.

```
# core/decorators.py
import functools
import time
```

```
def log_request(view):
    @functools.wraps(view)
    def wrapper(request, *args, **kwargs):
        start = time.perf_counter()
        response = view(request, *args, **kwargs)
        ms = (time.perf_counter() - start) * 1000
        print(f"{request.method} {request.path} -> {response.status_code}
({ms:.1f} ms)")
        return response
    return wrapper

# leads/views.py
from core.decorators import log_request

@log_request
def lead_list(request):
    ...
```

35.

```
# settings.py
CRM_COMPANY_NAME = "Nexora"

# core/views.py
from django.conf import settings

def welcome(request):
    return HttpResponse(f"Welcome to {settings.CRM_COMPANY_NAME} CRM")

# core/urls.py
path("welcome/", views.welcome, name="welcome"),
```

36.

```
# leads/views.py
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt            # practice only — real forms use {% csrf_token %}
(Day 13)
def quick_add(request):
    if request.method == "POST":
        name = request.POST.get("name", "").strip()
        new_id = max(l["id"] for l in LEADS) + 1
        lead = {"id": new_id, "name": name, "phone": "", "status": "new",
"source": "manual"}
        LEADS.append(lead)
        return JsonResponse(lead, status=201)
    return HttpResponse("POST a 'name' field to add a lead")

# leads/urls.py
path("quick-add/", views.quick_add, name="quick_add"),
```

```
curl -X POST -d "name=Sara Ali" http://127.0.0.1:8000/leads/quick-add/
```

37–39.

```
# core/tests.py
from django.test import SimpleTestCase
from django.urls import reverse


class CoreTests(SimpleTestCase):
    def test_home(self):                                    # 37
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Nexora CRM")

    def test_api_info(self):                                # 39
        response = self.client.get(reverse("api_info"))
        self.assertEqual(response.json()["app"], "Nexora CRM")
```

```
# leads/tests.py
from django.test import SimpleTestCase
from django.urls import reverse


class LeadViewTests(SimpleTestCase):
    def test_missing_lead_404(self):                        # 38
        response = self.client.get(reverse("leads:detail", args=[999]))
        self.assertEqual(response.status_code, 404)
```

40.

```
# python manage.py shell
from django.test.utils import setup_test_environment
setup_test_environment()        # allows the 'testserver' host outside the
test runner

from django.test import Client
r = Client().get("/leads/")
print(r.status_code, r.content[:50])
```

Day 2 — URL routing and the request object

Goal: every URL feature you'll need (converters, namespaces, reverse/resolve) plus full control of the request: query params, headers, body, cookies, session. Builds on Day 1's `LEADS` list.

A. Drills (1–10) — URL patterns

- [ ] **1.** Route `leads/type-check/<int:pk>/` to a view that returns the Python type name of `pk`.
- [ ] **2.** Route `leads/source/<slug:source>/` returning names of leads from that source (name `by_source`).
- [ ] **3.** Route `leads/ref/<uuid:ref>/` echoing the UUID and its type.
- [ ] **4.** Route `files/<path:doc_path>` in `core` that echoes the full path, slashes included.
- [ ] **5.** Use `re_path` to match 7-digit account numbers: `accounts/1234567/`.
- [ ] **6.** In the shell: `reverse()` `leads:detail` once with `args`, once with `kwargs`, and `leads:by_source`.
- [ ] **7.** Mount `leads.urls` a second time at `/archive/` with instance namespace `archive`; reverse `archive:list` and `leads:list`.
- [ ] **8.** View `latest_lead` that redirects to the detail page of the last lead using `redirect()` with kwargs.
- [ ] **9.** Module-level constant `LEADS_HOME = reverse_lazy("leads:list")`, used by a `go_leads` redirect view.
- [ ] **10.** In the shell, `resolve("/leads/3/")` and print `view_name`, `kwargs`, `func.__name__`.

B. Core (11–20) — the request object (put these under `/debug/...` in `core`)

- [ ] **11.** `echo_get`: return all query params as JSON.
- [ ] **12.** `page_param`: read `?page=`, default 1, fall back to 1 on non-numbers.
- [ ] **13.** In `leads`: `multi_status` reading repeated `?status=new&status=lost` and returning matching leads as JSON.
- [ ] **14.** `user_agent`: return the User-Agent header, default `unknown`.
- [ ] **15.** `client_ip`: return the caller's IP from `META`.
- [ ] **16.** `echo_post` (CSRF-exempt): return all POST form fields as JSON; test with curl.

- [ ] **17.** `json_echo` (CSRF-exempt): parse a JSON body, return `{"received": ...}`, 400 on invalid JSON.
- [ ] **18.** `absolute_url`: return the full absolute URL of lead 1's detail page.
- [ ] **19.** In `leads`: `remember_lead` at `<id>/remember/` that shows the previously viewed lead id from a cookie, then sets cookie `last_lead` (1 hour).
- [ ] **20.** `visit_counter`: count visits in the session.

C. Nexora CRM (21–30)

- [ ] **21.** Custom path converter `uae_phone` (pattern `9715` + 8 digits); route `leads/phone/<uae_phone:phone>/` returning the lead as JSON.
- [ ] **22.** `search` at `leads/search/?q=` — case-insensitive name match, return `query, count, results`.
- [ ] **23.** `paged` at `leads/paged/?page=&size=` — return `total, page, size, results` (size capped at 50).
- [ ] **24.** `sorted_leads` at `leads/sorted/?sort=name` or `-name`; 400 for unknown fields.
- [ ] **25.** `create_lead` at `leads/create/`: POST-only, JSON body, append lead with status `new`, return 201.
- [ ] **26.** `update_status` at `leads/<id>/status/`: POST-only form field `status`, validate, return updated lead.
- [ ] **27.** `delete_lead` at `leads/<id>/delete/`: DELETE-only, return 204 with empty body.
- [ ] **28.** New `clients` app: `CLIENTS` list with `ref` (UUID), `name`, `account`; routes `clients:list` and `clients:detail` by UUID.
- [ ] **29.** Versioned API: `leads/api_urls.py` (namespace `leads_api`) mounted at `/api/v1/` with `leads/` and `leads/<id>/`.
- [ ] **30.** "Recently viewed": `lead_detail` stores the last 5 viewed ids in the session; `leads/recent/` returns them as JSON.

D. Challenge (31–40)

- [ ] **31.** Decorator `require_api_key`: header `X-API-Key` must equal `settings.CRM_API_KEY`, else 401 JSON. Apply to the v1 API.
- [ ] **32.** Helper `json_error(message, status=400)` in `core/utils.py`; refactor 24 and 26 to use it.
- [ ] **33.** Decorator `rate_limited(limit=10, window=60)` using the session; return 429 when exceeded.

- [ ] **34.** One view, two routes: `leads/browse/` (page 1 default) and `leads/browse/<int:page>/`.
- [ ] **35.** Move the `/debug/` views into an inline `include((patterns, "debug"))` in the project urls; reverse `debug:get`.
- [ ] **36.** Test `search` with query params passed as a dict to `client.get`.
- [ ] **37.** Test `create_lead` posting JSON (`content_type="application/json"`) → 201, status `new`.
- [ ] **38.** Test that `/leads/phone/12345/` returns 404 (converter rejects it).
- [ ] **39.** One test that reverses 5 named URLs in a loop with `subTest`.
- [ ] **40.** A POST-only `protected_action` view (no csrf_exempt); test with `Client(enforce_csrf_checks=True)` that a token-less POST gets 403.

Solutions

A. Drills

1–3, 8–9.

```
# leads/views.py (add)
from django.shortcuts import redirect
from django.urls import reverse_lazy

LEADS_HOME = reverse_lazy("leads:list")   # 9: plain reverse() here fails —
URLconf not loaded at import time


def type_check(request, pk):                                    # 1
    return HttpResponse(f"{pk!r} is {type(pk).__name__}")       # → 3 is int


def leads_by_source(request, source):                           # 2
    names = [l["name"] for l in LEADS if l["source"] == source]
    return HttpResponse("<br>".join(names) or "No leads")


def lead_by_ref(request, ref):                                  # 3
    return HttpResponse(f"{ref} is {type(ref).__name__}")       # → UUID


def latest_lead(request):                                       # 8
    return redirect("leads:detail", lead_id=LEADS[-1]["id"])
```

```
def go_leads(request):                                          # 9
    return redirect(LEADS_HOME)
```

4–5.

```
# core/views.py (add)
def echo_path(request, doc_path):
    return HttpResponse(doc_path)            # /files/kyc/2026/passport.pdf → kyc/2026/passport.pdf


def account_detail(request, acc):
    return HttpResponse(f"Account {acc} ({type(acc).__name__})")   # re_path captures are str

# core/urls.py
from django.urls import path, re_path

path("files/<path:doc_path>", views.echo_path, name="echo_path"),
re_path(r"^accounts/(?P<acc>\d{7})/$", views.account_detail, name="account_detail"),
```

6, 10.

```
# python manage.py shell
from django.urls import reverse, resolve

reverse("leads:detail", args=[3])                          # '/leads/3/'
reverse("leads:detail", kwargs={"lead_id": 3})             # '/leads/3/'
reverse("leads:by_source", kwargs={"source": "google"})  # '/leads/source/google/'

match = resolve("/leads/3/")                               # 10
print(match.view_name, match.kwargs, match.func.__name__)
# leads:detail {'lead_id': 3} lead_detail
```

7.

```
# django_project/urls.py
path("leads/", include("leads.urls")),
path("archive/", include("leads.urls", namespace="archive")),

# shell
reverse("archive:list")   # '/archive/'
reverse("leads:list")     # '/leads/'
```

B. Core

11–12, 14–18, 20.

```
# core/views.py (add)
import json
from django.urls import reverse
from django.views.decorators.csrf import csrf_exempt


def echo_get(request):                                          # 11
    return JsonResponse(request.GET.dict())


def page_param(request):                                        # 12
    try:
        page = int(request.GET.get("page", 1))
    except ValueError:
        page = 1
    return HttpResponse(f"Page {page}")


def user_agent(request):                                        # 14
    return HttpResponse(request.headers.get("User-Agent", "unknown"))


def client_ip(request):                                         # 15
    return HttpResponse(request.META.get("REMOTE_ADDR"))        # behind a
proxy: X-Forwarded-For


@csrf_exempt
def echo_post(request):                                         # 16
    return JsonResponse(request.POST.dict())


@csrf_exempt
```

```
def json_echo(request):                                          # 17
    try:
        data = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({"error": "invalid JSON"}, status=400)
    return JsonResponse({"received": data})


def absolute_url(request):                                       # 18
    return HttpResponse(request.build_absolute_uri(reverse("leads:detail",
args=[1])))


def visit_counter(request):                                      # 20
    count = request.session.get("visits", 0) + 1
    request.session["visits"] = count
    return HttpResponse(f"Visits this session: {count}")
```

```
# core/urls.py (add)
path("debug/get/", views.echo_get, name="echo_get"),
path("debug/page/", views.page_param, name="page_param"),
path("debug/ua/", views.user_agent, name="user_agent"),
path("debug/ip/", views.client_ip, name="client_ip"),
path("debug/post/", views.echo_post, name="echo_post"),
path("debug/json/", views.json_echo, name="json_echo"),
path("debug/absolute/", views.absolute_url, name="absolute_url"),
path("debug/visits/", views.visit_counter, name="visit_counter"),
```

```
# PowerShell: type curl.exe instead of curl
curl -X POST -d "name=Ali&source=google" http://127.0.0.1:8000/debug/post/
curl -X POST -H "Content-Type: application/json" -d "{\"name\": \"Ali\"}"
http://127.0.0.1:8000/debug/json/
```

13, 19.

```
# leads/views.py (add)
def multi_status(request):                                          # 13
    statuses = request.GET.getlist("status")
    leads = [l for l in LEADS if l["status"] in statuses] if statuses else
LEADS
    return JsonResponse(leads, safe=False)


def remember_lead(request, lead_id):                                # 19
    lead = get_lead(lead_id)
    previous = request.COOKIES.get("last_lead", "none")
    response = HttpResponse(f"Viewing {lead['name']}. Previously viewed:
{previous}")
    response.set_cookie("last_lead", str(lead_id), max_age=3600)
    return response
```

C. Nexora CRM

21.

```
# leads/converters.py
class UAEPhoneConverter:
    regex = r"9715\d{8}"

    def to_python(self, value):
        return value

    def to_url(self, value):
        return value

# leads/views.py
def lead_by_phone(request, phone):
    for lead in LEADS:
        if lead["phone"] == phone:
            return JsonResponse(lead)
    raise Http404("No lead with that phone")
```

22–27, 30.

```
# leads/views.py (add)
import json
from django.views.decorators.csrf import csrf_exempt
from django.views.decorators.http import require_POST, require_http_methods
```

```
def search(request):                                                    # 22
    q = request.GET.get("q", "").strip().lower()
    results = [l for l in LEADS if q in l["name"].lower()] if q else []
    return JsonResponse({"query": q, "count": len(results), "results":
results})


def paged(request):                                                     # 23
    page = max(int(request.GET.get("page", 1)), 1)
    size = min(int(request.GET.get("size", 2)), 50)
    start = (page - 1) * size
    return JsonResponse({
        "total": len(LEADS), "page": page, "size": size,
        "results": LEADS[start:start + size],
    })


def sorted_leads(request):                                              # 24
    sort = request.GET.get("sort", "id")
    key = sort.lstrip("-")
    if key not in {"id", "name", "status", "source"}:
        return JsonResponse({"error": f"cannot sort by {key}"}, status=400)
    leads = sorted(LEADS, key=lambda l: l[key], reverse=sort.startswith("-"))
    return JsonResponse(leads, safe=False)


@csrf_exempt
@require_POST
def create_lead(request):                                               # 25
    data = json.loads(request.body)
    lead = {
        "id": max(l["id"] for l in LEADS) + 1,
        "name": data["name"],
        "phone": data.get("phone", ""),
        "status": "new",
        "source": data.get("source", "manual"),
    }
    LEADS.append(lead)
    return JsonResponse(lead, status=201)


@csrf_exempt
@require_POST
def update_status(request, lead_id):                                    # 26
    lead = get_lead(lead_id)
    status = request.POST.get("status")
```

```
    if status not in VALID_STATUSES:
        return JsonResponse({"error": "invalid status"}, status=400)
    lead["status"] = status
    return JsonResponse(lead)


@csrf_exempt
@require_http_methods(["DELETE"])
def delete_lead(request, lead_id):                                  # 27
    LEADS.remove(get_lead(lead_id))
    return HttpResponse(status=204)


def lead_detail(request, lead_id):                                  # 30
(replaces Day 1 version)
    lead = get_lead(lead_id)
    recent = request.session.get("recent_leads", [])
    if lead_id in recent:
        recent.remove(lead_id)
    recent.insert(0, lead_id)
    request.session["recent_leads"] = recent[:5]
    return HttpResponse(f"{lead['name']} — {lead['status']}")


def recent_leads(request):                                          # 30
    ids = request.session.get("recent_leads", [])
    return JsonResponse([get_lead(i) for i in ids], safe=False)
```

```
curl -X DELETE http://127.0.0.1:8000/leads/5/delete/
curl -X POST -d "status=qualified" http://127.0.0.1:8000/leads/1/status/
```

Full `leads/urls.py` after Day 2:

```
from django.urls import path, register_converter
from . import converters, views

register_converter(converters.UAEPhoneConverter, "uae_phone")

app_name = "leads"
urlpatterns = [
    path("", views.lead_list, name="list"),
    path("count/", views.lead_count, name="count"),
    path("json/", views.lead_list_json, name="json"),
    path("menu/", views.lead_menu, name="menu"),
    path("export/", views.leads_csv, name="export"),
    path("quick-add/", views.quick_add, name="quick_add"),
```

```
    path("latest/", views.latest_lead, name="latest"),
    path("go/", views.go_leads, name="go"),
    path("multi/", views.multi_status, name="multi_status"),
    path("search/", views.search, name="search"),
    path("paged/", views.paged, name="paged"),
    path("sorted/", views.sorted_leads, name="sorted"),
    path("create/", views.create_lead, name="create"),
    path("recent/", views.recent_leads, name="recent"),
    path("browse/", views.lead_page, name="browse"),                      #
34
    path("browse/<int:page>/", views.lead_page, name="browse_page"),      #
34
    path("protected/", views.protected_action, name="protected"),          #
40
    path("type-check/<int:pk>/", views.type_check, name="type_check"),
    path("status/<str:status>/", views.leads_by_status, name="by_status"),
    path("source/<slug:source>/", views.leads_by_source, name="by_source"),
    path("ref/<uuid:ref>/", views.lead_by_ref, name="by_ref"),
    path("phone/<uae_phone:phone>/", views.lead_by_phone, name="by_phone"),
    path("<int:lead_id>/", views.lead_detail, name="detail"),
    path("<int:lead_id>/phone/", views.lead_phone, name="phone"),
    path("<int:lead_id>/remember/", views.remember_lead, name="remember"),
    path("<int:lead_id>/status/", views.update_status, name="update_status"),
    path("<int:lead_id>/delete/", views.delete_lead, name="delete"),
]
```

28.

```
python manage.py startapp clients     # add "clients.apps.ClientsConfig" to
INSTALLED_APPS
```

```
# clients/views.py
import uuid
from django.http import JsonResponse, Http404

CLIENTS = [
    {"ref": uuid.UUID("3f1c2b8e-5d4a-4c1e-9a7b-1e2f3a4b5c6d"), "name": "Aisha
Khan", "account": "1000001"},
    {"ref": uuid.UUID("7a9d0e1f-2b3c-4d5e-8f90-a1b2c3d4e5f6"), "name": "Priya
Nair", "account": "1000002"},
]


def client_list(request):
    return JsonResponse(CLIENTS, safe=False)   # JsonResponse's encoder
handles UUID
```

```
def client_detail(request, ref):
    for client in CLIENTS:
        if client["ref"] == ref:
            return JsonResponse(client)
    raise Http404("Client not found")

# clients/urls.py
from django.urls import path
from . import views

app_name = "clients"
urlpatterns = [
    path("", views.client_list, name="list"),
    path("<uuid:ref>/", views.client_detail, name="detail"),
]

# django_project/urls.py
path("clients/", include("clients.urls")),
```

29 + 31.

```
# core/decorators.py (add)                                          # 31
from django.conf import settings
from django.http import JsonResponse


def require_api_key(view):
    @functools.wraps(view)
    def wrapper(request, *args, **kwargs):
        if request.headers.get("X-API-Key") != settings.CRM_API_KEY:
            return JsonResponse({"error": "invalid API key"}, status=401)
        return view(request, *args, **kwargs)
    return wrapper

# settings.py
CRM_API_KEY = "dev-key-123"     # moves to .env on Day 22
```

```
# leads/api_urls.py                                                    # 29
from django.urls import path
from core.decorators import require_api_key
from . import views

app_name = "leads_api"
urlpatterns = [
    path("leads/", require_api_key(views.lead_list_json), name="list"),
    path("leads/<int:lead_id>/", require_api_key(views.lead_detail_json),
name="detail"),
]

# leads/views.py
def lead_detail_json(request, lead_id):
    return JsonResponse(get_lead(lead_id))

# django_project/urls.py
path("api/v1/", include("leads.api_urls")),
```

```
curl -H "X-API-Key: dev-key-123" http://127.0.0.1:8000/api/v1/leads/
```

D. Challenge

32–33.

```
# core/utils.py
from django.http import JsonResponse


def json_error(message, status=400, **extra):
    return JsonResponse({"error": message, "status": status, **extra},
status=status)

# leads/views.py — e.g. in sorted_leads / update_status
from core.utils import json_error
return json_error(f"cannot sort by {key}")
return json_error("invalid status")
```

```
# core/decorators.py (add)                                             # 33
from core.utils import json_error


def rate_limited(limit=10, window=60):
    def decorator(view):
        @functools.wraps(view)
```

```
        def wrapper(request, *args, **kwargs):
            now = time.time()
            hits = [t for t in request.session.get("hits", []) if now - t <
window]
            if len(hits) >= limit:
                return json_error("rate limit exceeded", status=429)
            hits.append(now)
            request.session["hits"] = hits
            return view(request, *args, **kwargs)
        return wrapper
    return decorator

# usage
@rate_limited(limit=5, window=60)
def search(request): ...
```

34.

```
def lead_page(request, page=1):          # default covers the route with no
<page>
    size = 2
    start = (page - 1) * size
    names = [l["name"] for l in LEADS[start:start + size]]
    return HttpResponse(f"Page {page}: " + ", ".join(names))
```

35.

```
# django_project/urls.py
from core import views as core_views

debug_patterns = [
    path("get/", core_views.echo_get, name="get"),
    path("page/", core_views.page_param, name="page"),
    path("ua/", core_views.user_agent, name="ua"),
    path("ip/", core_views.client_ip, name="ip"),
    path("post/", core_views.echo_post, name="post"),
    path("json/", core_views.json_echo, name="json"),
    path("absolute/", core_views.absolute_url, name="absolute"),
    path("visits/", core_views.visit_counter, name="visits"),
]

urlpatterns = [
    path("admin/", admin.site.urls),
    path("debug/", include((debug_patterns, "debug"))),   # (patterns,
app_name)
```

```
    ...
]
# delete the debug/ lines from core/urls.py; reverse("debug:get") →
'/debug/get/'
```

36–40.

```
# leads/views.py (add)                                          # 40
@require_POST
def protected_action(request):
    return HttpResponse("done")
```

```
# leads/tests.py (add to LeadViewTests)
from django.test import Client

    def test_search(self):                                      # 36
        response = self.client.get(reverse("leads:search"), {"q": "aisha"})
        self.assertEqual(response.json()["count"], 1)

    def test_create_lead(self):                                 # 37
        response = self.client.post(
            reverse("leads:create"), {"name": "Test Lead"},
content_type="application/json"
        )
        self.assertEqual(response.status_code, 201)
        self.assertEqual(response.json()["status"], "new")

    def test_invalid_phone_404(self):                           # 38
        self.assertEqual(self.client.get("/leads/phone/12345/").status_code,
404)

    def test_named_urls(self):                                  # 39
        cases = [
            ("leads:list", {}, "/leads/"),
            ("leads:detail", {"lead_id": 1}, "/leads/1/"),
            ("leads:by_source", {"source": "google"},
"/leads/source/google/"),
            ("clients:list", {}, "/clients/"),
            ("leads_api:list", {}, "/api/v1/leads/"),
        ]
        for name, kwargs, expected in cases:
            with self.subTest(name=name):
                self.assertEqual(reverse(name, kwargs=kwargs), expected)
```

```
    def test_csrf_enforced(self):                                    # 40
        client = Client(enforce_csrf_checks=True)
        response = client.post(reverse("leads:protected"))
        self.assertEqual(response.status_code, 403)
```

Day 3 — Templates, inheritance, static files

Goal: replace raw `HttpResponse` HTML with real templates — a CRM layout with sidebar, lead pages, dashboard cards, and your first custom template filters and tags.

A. Drills (1–10) — template language

- [ ] **1.** Create a project-level `templates/` folder and register it in `TEMPLATES["DIRS"]`.
- [ ] **2.** Create `core/templates/core/home.html`; make `home` use `render()` with it.
- [ ] **3.** Pass context `company="Nexora"` and `user_count=12`; show `Nexora CRM — 12 users`.
- [ ] **4.** Apply `upper`, `lower`, `title`, `length`, and `default:"N/A"` (on a missing variable).
- [ ] **5.** Pass `timezone.now()` and display it as `15 Oct 2026, 14:30`.
- [ ] **6.** Loop over `leads` with `{% for %}` and an `{% empty %}` fallback.
- [ ] **7.** Number the loop with `forloop.counter`, tag the first item, comma-separate all but the last.
- [ ] **8.** `{% if %}` / `{% elif %}` with `and`, `or`, `not`, `in`: label leads Hot / Warm / Cold.
- [ ] **9.** Link each lead with `{% url %}`: once positional (`leads:detail`), once keyword (`leads:by_source`).
- [ ] **10.** Use `{# #}`, `{% comment %}`, and `{% with %}` + `pluralize` to print `5 leads`.

B. Core (11–20) — inheritance, includes, static

- [ ] **11.** `templates/base.html` with blocks `title`, `extra_css`, `content`, `extra_js`.
- [ ] **12.** Make `core/home.html` extend base and override `title`.
- [ ] **13.** Use `{{ block.super }}` so the leads page title reads `Leads · Nexora CRM`.
- [ ] **14.** Partial `templates/partials/_navbar.html`, included from base.
- [ ] **15.** Partial `_badge.html` included with `with status=lead.status only`.
- [ ] **16.** Static setup: `STATICFILES_DIRS`, `static/css/crm.css`, linked from base via `{% static %}`.
- [ ] **17.** Show `static/img/logo.png` in the navbar.
- [ ] **18.** `static/js/crm.js` logging `Nexora CRM loaded`, loaded with `defer`.
- [ ] **19.** Enable `django.contrib.humanize`; show `intcomma`, `floatformat:2`, `truncatechars:30`.
- [ ] **20.** Pass `note = "<b>VIP</b> client"`: show it escaped, with `|safe`, and inside `{% autoescape off %}`.

C. Nexora CRM (21–30)

- [ ] **21.** CRM layout in base: dark sidebar (Dashboard, Leads, Clients, Reports), topbar, content area (CSS grid).
- [ ] **22.** Highlight the active sidebar link using `request.resolver_match`.
- [ ] **23.** Convert `lead_list` to `render()` with `leads/lead_list.html`: table of name, phone, status, source.
- [ ] **24.** Status badges with a colour per status (new, contacted, qualified, lost).
- [ ] **25.** Convert `lead_detail` to `leads/lead_detail.html` using a `<dl>`; keep the session "recent" logic.
- [ ] **26.** Convert `dashboard` to `core/dashboard.html`: total leads + one card per status.
- [ ] **27.** On the lead list, add a "By status" section using `{% regroup %}`.
- [ ] **28.** Zebra-stripe table rows with `{% cycle %}`.
- [ ] **29.** Search form (GET `q`) above the table; keep the typed value; empty-state row when nothing matches.
- [ ] **30.** Footer with the current year via `{% now %}`.

D. Challenge (31–40)

- [ ] **31.** Custom filter `mask_phone` in `leads/templatetags/crm_filters.py`; use it in the lead table.
- [ ] **32.** `simple_tag` `status_count leads "new"`, also used with `as new_count`.
- [ ] **33.** `templates/404.html` extending base; make `custom_404` render it.
- [ ] **34.** `inclusion_tag` `lead_card` rendering `leads/partials/_lead_card.html`.
- [ ] **35.** Dashboard "Recent activity" list with `naturaltime` (`5 minutes ago`).
- [ ] **36.** In the shell, `render_to_string` the lead card for `LEADS[0]` and print it.
- [ ] **37.** Test the lead list uses both `leads/lead_list.html` and `base.html`.
- [ ] **38.** Test the list contains the `badge-new` class.
- [ ] **39.** Test that a lead named `<script>alert(1)</script>` is escaped in the HTML.
- [ ] **40.** Prove lookup order: a project-level `templates/leads/lead_list.html` overrides the app one. Test it, then delete it.

Solutions

A. Drills

1–2.

```
# django_project/settings.py
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],   # project-level, searched first
        "APP_DIRS": True,                   # then <app>/templates/
        ...
    },
]
```

```
# core/views.py
from django.shortcuts import render
from django.utils import timezone
from leads.views import LEADS


def home(request):
    context = {
        "company": "Nexora",
        "user_count": 12,
        "now": timezone.now(),
        "leads": LEADS,
        "paid_sources": ["google", "facebook"],
        "note": "<b>VIP</b> client",   # 20
    }
    return render(request, "core/home.html", context)
```

3–10, 12, 19–20. Final `home.html` (extends base from 12 onward).

```
<!-- core/templates/core/home.html -->
{% extends "base.html" %}
{% load humanize %}
{% block title %}Home · {{ block.super }}{% endblock title %}

{% block content %}
<h1>{{ company }} CRM — {{ user_count }} users</h1>
{# 3 #}

<p>{{ company|upper }} · {{ company|lower }} · {{ "omar haddad"|title }}</p>
{# 4 #}
<p>{{ leads|length }} leads · Region: {{ region|default:"N/A" }}</p>

<p>{{ now|date:"d M Y, H:i" }}</p>
{# 5 #}
```

```
<ul>
{# 6 #}
  {% for lead in leads %}
    <li>{{ lead.name }}</li>
  {% empty %}
    <li>No leads yet.</li>
  {% endfor %}
</ul>

<p>
{# 7 #}
  {% for lead in leads %}
    {{ forloop.counter }}. {{ lead.name }}{% if forloop.first %} (first){%
endif %}{% if not forloop.last %}, {% endif %}
  {% endfor %}
</p>

{% for lead in leads %}
{# 8 #}
  <p>{{ lead.name }}:
    {% if lead.status == "new" and lead.source in paid_sources %}Hot
    {% elif lead.status == "lost" or not lead.phone %}Cold
    {% else %}Warm{% endif %}
  </p>
{% endfor %}

{% for lead in leads %}
{# 9 #}
  <a href="{% url 'leads:detail' lead.id %}">{{ lead.name }}</a>
  (<a href="{% url 'leads:by_source' source=lead.source %}">{{ lead.source }}
</a>)
{% endfor %}

{# 10: one-line comment #}
{% comment "hidden on purpose" %}
  <p>Multi-line comment — never rendered.</p>
{% endcomment %}
{% with total=leads|length %}
  <p>{{ total }} lead{{ total|pluralize }}</p>
{% endwith %}

<p>{{ 1250000|intcomma }} AED · {{ 1234.5678|floatformat:2 }} ·
{# 19 #}
   {{ "Client asked for a callback about the gold account"|truncatechars:30
}}</p>
```

```
<p>{{ note }}</p>                                   {# 20: shows the tags as
text #}
<p>{{ note|safe }}</p>                              {# renders bold #}
{% autoescape off %}<p>{{ note }}</p>{% endautoescape %}
{% endblock content %}
```

```
# settings.py — 19
INSTALLED_APPS = [..., "django.contrib.humanize", ...]
```

B. Core

11, 14, 16–18, 21–22, 30. Final `base.html`.

```
<!-- templates/base.html -->
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Nexora CRM{% endblock title %}</title>
  <link rel="stylesheet" href="{% static 'css/crm.css' %}">
  {% block extra_css %}{% endblock extra_css %}
</head>
<body>
<div class="layout">
  <aside class="sidebar">
{# 21 #}
    <a href="{% url 'home' %}" class="brand">Nexora CRM</a>
    {% with ns=request.resolver_match.namespace
name=request.resolver_match.url_name %}   {# 22 #}
      <a href="{% url 'dashboard' %}" class="{% if name == 'dashboard'
%}active{% endif %}">Dashboard</a>
      <a href="{% url 'leads:list' %}" class="{% if ns == 'leads' %}active{%
endif %}">Leads</a>
      <a href="{% url 'clients:list' %}" class="{% if ns == 'clients'
%}active{% endif %}">Clients</a>
      <a href="#">Reports</a>
    {% endwith %}
  </aside>
  <div class="main">
    {% include "partials/_navbar.html" %}
{# 14 #}
    <main class="content">
      {% block content %}{% endblock content %}
    </main>
```

```
    <footer class="footer">© {% now "Y" %} Nexora · Nexora CRM</footer>
{# 30 #}
  </div>
</div>
<script src="{% static 'js/crm.js' %}" defer></script>
{# 18 #}
{% block extra_js %}{% endblock extra_js %}
</body>
</html>
```

```
<!-- templates/partials/_navbar.html  (14, 17) -->
{% load static %}   {# loads don't carry into included templates #}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>Nexora CRM</span>
</header>
```

```
<!-- templates/partials/_badge.html  (15, 24) -->
<span class="badge badge-{{ status }}">{{ status|title }}</span>
```

```
# settings.py — 16
STATIC_URL = "static/"
STATICFILES_DIRS = [BASE_DIR / "static"]
```

```
mkdir -p static/css static/js static/img     # drop any PNG in
static/img/logo.png (17)
```

```
// static/js/crm.js — 18
console.log("Nexora CRM loaded");
```

```
/* static/css/crm.css — 16, 21, 24, 26, 28 */
* { box-sizing: border-box; }
body { margin: 0; font-family: system-ui, sans-serif; color: #0f172a;
background: #f8fafc; }
.layout { display: grid; grid-template-columns: 220px 1fr; min-height: 100vh;
}
.sidebar { background: #0f172a; padding: 1rem; }
.sidebar .brand { color: #fff; font-weight: 700; display: block; margin-
bottom: 1.5rem; text-decoration: none; }
.sidebar a { display: block; color: #cbd5e1; padding: .5rem .75rem; border-
radius: 6px; text-decoration: none; }
.sidebar a.active, .sidebar a:hover { background: #1e293b; color: #fff; }
.topbar { display: flex; align-items: center; gap: .75rem; padding: .75rem
1.5rem; background: #fff; border-bottom: 1px solid #e2e8f0; }
```

```
.content { padding: 1.5rem; }
.footer { padding: 1rem 1.5rem; color: #64748b; font-size: .875rem; }
.table { width: 100%; border-collapse: collapse; background: #fff; }
.table th, .table td { padding: .6rem .75rem; text-align: left; }
.row-dark { background: #f1f5f9; }
.badge { padding: .15rem .5rem; border-radius: 999px; font-size: .75rem;
font-weight: 600; }
.badge-new { background: #dbeafe; color: #1e40af; }
.badge-contacted { background: #fef9c3; color: #854d0e; }
.badge-qualified { background: #dcfce7; color: #166534; }
.badge-lost { background: #fee2e2; color: #991b1b; }
.cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px,
1fr)); gap: 1rem; }
.card { background: #fff; border: 1px solid #e2e8f0; border-radius: 10px;
padding: 1rem; }
.card h3 { margin: 0; font-size: 1.75rem; }
.lead-card { background: #fff; border: 1px solid #e2e8f0; border-radius: 8px;
padding: .75rem; margin-bottom: .5rem; }
```

C. Nexora CRM

23, 25, 29. Views.

```
# leads/views.py (replace these two)
from django.shortcuts import render


@log_request
def lead_list(request):
    source = request.GET.get("source")
    q = request.GET.get("q", "").strip().lower()                # 29
    leads = [l for l in LEADS if not source or l["source"] == source]
    if q:
        leads = [l for l in leads if q in l["name"].lower()]
    return render(request, "leads/lead_list.html", {"leads": leads})


def lead_detail(request, lead_id):
    lead = get_lead(lead_id)
    recent = request.session.get("recent_leads", [])
    if lead_id in recent:
        recent.remove(lead_id)
    recent.insert(0, lead_id)
    request.session["recent_leads"] = recent[:5]
    return render(request, "leads/lead_detail.html", {"lead": lead})
```

13, 23–24, 27–29, 31. `lead_list.html` (add the `crm_filters` load and `mask_phone` after exercise 31).

```
<!-- leads/templates/leads/lead_list.html -->
{% extends "base.html" %}
{% load crm_filters %}
{% block title %}Leads · {{ block.super }}{% endblock title %}
{# 13 #}

{% block content %}
<h1>Leads</h1>

<form method="get" action="{% url 'leads:list' %}">
{# 29 #}
  <input type="text" name="q" value="{{ request.GET.q }}" placeholder="Search
by name">
  <button type="submit">Search</button>
</form>

<table class="table">
  <thead><tr><th>#</th><th>Name</th><th>Phone</th><th>Status</th>
<th>Source</th></tr></thead>
  <tbody>
  {% for lead in leads %}
    <tr class="{% cycle 'row-light' 'row-dark' %}">
{# 28 #}
      <td>{{ forloop.counter }}</td>
      <td><a href="{% url 'leads:detail' lead.id %}">{{ lead.name }}</a></td>
      <td>{{ lead.phone|mask_phone }}</td>
{# 31 #}
      <td>{% include "partials/_badge.html" with status=lead.status only %}
</td>
      <td>{{ lead.source }}</td>
    </tr>
  {% empty %}
    <tr><td colspan="5">No leads match "{{ request.GET.q }}".</td></tr>
  {% endfor %}
  </tbody>
</table>

<h2>By status</h2>
{# 27 #}
{% regroup leads|dictsort:"status" by status as status_groups %}
{# regroup needs sorted input #}
{% for group in status_groups %}
```

```
  <h3>{{ group.grouper|title }} ({{ group.list|length }})</h3>
  <ul>{% for lead in group.list %}<li>{{ lead.name }}</li>{% endfor %}</ul>
{% endfor %}
{% endblock content %}
```

```
<!-- leads/templates/leads/lead_detail.html  (25) -->
{% extends "base.html" %}
{% load crm_filters %}
{% block title %}{{ lead.name }} · {{ block.super }}{% endblock title %}

{% block content %}
<h1>{{ lead.name }}</h1>
<dl>
  <dt>Phone</dt><dd>{{ lead.phone|mask_phone|default:"—" }}</dd>
  <dt>Status</dt><dd>{% include "partials/_badge.html" with
status=lead.status only %}</dd>
  <dt>Source</dt><dd>{{ lead.source|title }}</dd>
</dl>
<p><a href="{% url 'leads:list' %}">← All leads</a></p>
{% endblock content %}
```

26, 32, 34–35. Dashboard.

```
# core/views.py
from datetime import timedelta


def dashboard(request):
    statuses = ["new", "contacted", "qualified", "lost"]
    counts = {s: sum(1 for l in LEADS if l["status"] == s) for s in statuses}
    now = timezone.now()
    activity = [                                                    # 35
        {"text": "Aisha Khan called back", "at": now - timedelta(minutes=5)},
        {"text": "Priya Nair qualified", "at": now - timedelta(hours=2)},
        {"text": "Mei Lin marked lost", "at": now - timedelta(days=1)},
    ]
    context = {"total": len(LEADS), "counts": counts, "leads": LEADS,
"activity": activity}
    return render(request, "core/dashboard.html", context)
```

```
<!-- core/templates/core/dashboard.html -->
{% extends "base.html" %}
{% load humanize crm_filters %}
{% block title %}Dashboard · {{ block.super }}{% endblock title %}

{% block content %}
```

```
<h1>Dashboard</h1>
<div class="cards">
  <div class="card"><h3>{{ total }}</h3><p>Total leads</p></div>
  {% for status, n in counts.items %}
    <div class="card"><h3>{{ n }}</h3><p>{{ status|title }}</p></div>
  {% endfor %}
</div>

{% status_count leads "new" as new_count %}
{# 32 #}
<p>{{ new_count }} new lead{{ new_count|pluralize }} waiting for a first
call.</p>

<h2>Recent activity</h2>
{# 35 #}
<ul>
  {% for item in activity %}<li>{{ item.text }} — {{ item.at|naturaltime }}
</li>{% endfor %}
</ul>

<h2>Lead cards</h2>
{# 34 #}
{% for lead in leads %}{% lead_card lead %}{% endfor %}
{% endblock content %}
```

D. Challenge

31–32, 34. Custom filters and tags (restart `runserver` after creating the package).

```
# leads/templatetags/__init__.py   ← empty file, required

# leads/templatetags/crm_filters.py
from django import template

register = template.Library()


@register.filter
def mask_phone(value):                                          # 31
    value = str(value)
    if len(value) < 8:
        return value
    return value[:4] + "*" * (len(value) - 8) + value[-4:]


@register.simple_tag
```

```
def status_count(leads, status):                                    # 32
    return sum(1 for lead in leads if lead["status"] == status)


@register.inclusion_tag("leads/partials/_lead_card.html")
def lead_card(lead):                                                # 34
    return {"lead": lead}
```

```
<!-- leads/templates/leads/partials/_lead_card.html -->
{% load crm_filters %}
<div class="lead-card">
  <strong>{{ lead.name }}</strong>
  {% include "partials/_badge.html" with status=lead.status only %}
  <div>{{ lead.phone|mask_phone }}</div>
</div>
```

33.

```
<!-- templates/404.html -->
{% extends "base.html" %}
{% block title %}Not found · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Page not found</h1>
<p><a href="{% url 'dashboard' %}">Back to the dashboard</a></p>
{% endblock content %}
```

```
# core/views.py
def custom_404(request, exception):
    return render(request, "404.html", status=404)
```

36.

```
# python manage.py shell
from django.template.loader import render_to_string
from leads.views import LEADS

print(render_to_string("leads/partials/_lead_card.html", {"lead": LEADS[0]}))
```

37–40.

```
# leads/tests.py (add to LeadViewTests)
from .views import LEADS

    def test_lead_list_templates(self):                             # 37
        response = self.client.get(reverse("leads:list"))
        self.assertEqual(response.status_code, 200)
```

```
        self.assertTemplateUsed(response, "leads/lead_list.html")
        self.assertTemplateUsed(response, "base.html")

    def test_status_badge(self):                                    # 38
        response = self.client.get(reverse("leads:list"))
        self.assertContains(response, "badge-new")

    def test_lead_name_is_escaped(self):                            # 39
        evil = {"id": 9999, "name": "<script>alert(1)</script>",
                "phone": "", "status": "new", "source": "test"}
        LEADS.append(evil)
        try:
            response = self.client.get(reverse("leads:list"))
            self.assertNotContains(response, "<script>alert(1)</script>")
            self.assertContains(response,
"&lt;script&gt;alert(1)&lt;/script&gt;")
        finally:
            LEADS.remove(evil)
```

```
<!-- 40: templates/leads/lead_list.html  (project-level copy) -->
{% extends "base.html" %}
{% block content %}PROJECT OVERRIDE{% endblock content %}
```

```
    def test_project_template_overrides_app(self):              # 40 — DIRS
beats APP_DIRS
        response = self.client.get(reverse("leads:list"))
        self.assertContains(response, "PROJECT OVERRIDE")
# passes → delete the override file and this test
```

Day 4 — Class-based views

Goal: rebuild the Day 1–3 function views as classes and write your own reusable mixins. URL names stay the same, so every template and test keeps working. Data is still the in-memory `LEADS` list (the database arrives on Day 5–6).

A. Drills (1–10) — the base classes

- [ ] **1.** Serve `/about/` with `TemplateView.as_view(template_name=...)` straight in `urls.py` (new `core/about.html`).
- [ ] **2.** `HomeView(TemplateView)` replacing `home`; move the context into `get_context_data`.
- [ ] **3.** Move the static part of the home context (`company`, `user_count`) into `extra_context`.
- [ ] **4.** `/home/` → `home` with `RedirectView` using `pattern_name`, non-permanent.
- [ ] **5.** `/crm/` → `/leads/` with `RedirectView(url=...)`, keeping the query string (`/crm/?source=google`).
- [ ] **6.** `HealthView(View)` with a `get()` returning `OK`.
- [ ] **7.** `GetOnlyView` restricted with `http_method_names`.
- [ ] **8.** `GreetingView` with class attribute `greeting = "Hello"`; override it in `as_view(greeting="Marhaba")`.
- [ ] **9.** `LeadPhoneView(View)` reading the id from `self.kwargs`; returns the masked phone.
- [ ] **10.** `RequestInfoView` overriding `dispatch()` to print the method and class name before handling.

B. Core (11–20) — converting and extending

- [ ] **11.** `QuickAddView` with `get()` and `post()`; CSRF-exempt via `method_decorator` on `dispatch`.
- [ ] **12.** Apply Day 1's `log_request` decorator to a class with `method_decorator(..., name="dispatch")`.
- [ ] **13.** `core/mixins.py`: `JsonResponseMixin.render_to_json(data, status=200)` handling dicts and lists; use it in `LeadCountView`.
- [ ] **14.** `LeadListView(TemplateView)` rendering `leads/lead_list.html` with filtered leads.
- [ ] **15.** `LeadDetailView(TemplateView)` with the 404 and session "recent" logic.

- [ ] **16.** `DashboardView(TemplateView)` replacing `dashboard`.
- [ ] **17.** Override `get_template_names()` on `LeadListView`: `?compact=1` uses `leads/lead_list_compact.html`.
- [ ] **18.** Send an OPTIONS request to any CBV and read the `Allow` header Django builds for you.
- [ ] **19.** `JsonMethodNotAllowedMixin` overriding `http_method_not_allowed` to return JSON 405 with the allowed methods.
- [ ] **20.** Override `setup()` on `LeadCountView` to attach `self.leads` before `dispatch` runs.

C. Nexora CRM (21–30) — your own mixins

- [ ] **21.** `LeadQueryMixin.get_leads()` applying `source`, `status` and `q` from `request.GET`; use it in `LeadListView`.
- [ ] **22.** `LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View)` — same filters, JSON out.
- [ ] **23.** `LeadCSVView(LeadQueryMixin, View)` — filtered CSV download.
- [ ] **24.** `LeadStatusUpdateView` — POST only, CSRF-exempt, JSON 405 for anything else.
- [ ] **25.** `LeadDeleteView` — implement `delete()`, return 204.
- [ ] **26.** `RecentLeadsView` — session "recent" ids as JSON.
- [ ] **27.** `ClientListView` and `ClientDetailView` as CBVs in `clients`.
- [ ] **28.** `ApiKeyRequiredMixin` checking `X-API-Key` in `dispatch`; replace the Day 2 decorator in `api_urls.py`.
- [ ] **29.** `PageTitleMixin` adding `page_title` to the context; show it in `_navbar.html`.
- [ ] **30.** Rewrite `leads/urls.py` to use the classes. Keep every URL name unchanged; run the tests.

D. Challenge (31–40)

- [ ] **31.** Write `simple_as_view(view_class, **initkwargs)` — your own mini `as_view()` — and route a URL through it.
- [ ] **32.** Print the MRO of `LeadListJSONView` as a list of class names.
- [ ] **33.** Turn Day 2's `rate_limited` decorator into `RateLimitMixin` with `rate_limit` and `rate_window` attributes.
- [ ] **34.** `CompanyContextMixin` + `PageTitleMixin` on one `AboutView`; confirm both context keys arrive (cooperative `super()`).
- [ ] **35.** `LatestLeadRedirectView(RedirectView)` overriding `get_redirect_url()`.

- [ ] **36.** `/robots.txt` from a `TemplateView` with `content_type="text/plain"`.
- [ ] **37.** Test that `resolve("/").func.view_class` is `HomeView`.
- [ ] **38.** Test `LeadListJSONView` directly with `RequestFactory` and `?status=lost`.
- [ ] **39.** Test that OPTIONS on `leads:json` lists `GET` in `Allow`.
- [ ] **40.** Test that GET on `leads:update_status` returns JSON 405.

Solutions

Shared mixins (13, 19, 21, 28, 29, 33, 34)

Built across the day; final file shown once.

```
# core/mixins.py
import time

from django.conf import settings
from django.http import JsonResponse

from core.utils import json_error


class JsonResponseMixin:                                            # 13
    def render_to_json(self, data, status=200):
        return JsonResponse(data, status=status, safe=isinstance(data, dict))


class JsonMethodNotAllowedMixin:                                    # 19
    def http_method_not_allowed(self, request, *args, **kwargs):
        return JsonResponse(
            {"error": f"{request.method} not allowed", "allowed":
self._allowed_methods()},
            status=405,
        )


class ApiKeyRequiredMixin:                                          # 28
    def dispatch(self, request, *args, **kwargs):
        if request.headers.get("X-API-Key") != settings.CRM_API_KEY:
            return JsonResponse({"error": "invalid API key"}, status=401)
        return super().dispatch(request, *args, **kwargs)


class PageTitleMixin:                                               # 29
    page_title = ""
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["page_title"] = self.page_title
        return context


class CompanyContextMixin:                                          # 34
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["company"] = settings.CRM_COMPANY_NAME
        return context


class RateLimitMixin:                                               # 33
    rate_limit = 10
    rate_window = 60

    def dispatch(self, request, *args, **kwargs):
        now = time.time()
        hits = [t for t in request.session.get("hits", []) if now - t <
self.rate_window]
        if len(hits) >= self.rate_limit:
            return json_error("rate limit exceeded", status=429)
        hits.append(now)
        request.session["hits"] = hits
        return super().dispatch(request, *args, **kwargs)
```

Rule: mixins go left, the Django base class goes last. Every overridden method calls `super()` so the chain continues.

A. Drills

1–8, 10, 16, 34–36. `core` views.

```
# core/views.py (class versions)
from datetime import timedelta

from django.http import HttpResponse
from django.utils import timezone
from django.views import View
from django.views.generic import TemplateView

from core.mixins import PageTitleMixin, CompanyContextMixin
from leads.views import LEADS
```

```
class HomeView(TemplateView):                                        # 2–3
    template_name = "core/home.html"
    extra_context = {"company": "Nexora", "user_count": 12}

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)   # includes
extra_context
        context.update(
            now=timezone.now(),
            leads=LEADS,
            paid_sources=["google", "facebook"],
            note="<b>VIP</b> client",
        )
        return context


class HealthView(View):                                              # 6
    def get(self, request):
        return HttpResponse("OK")


class GetOnlyView(View):                                             # 7
    http_method_names = ["get"]

    def get(self, request):
        return HttpResponse("GET only")


class GreetingView(View):                                            # 8
    greeting = "Hello"

    def get(self, request):
        return HttpResponse(f"{self.greeting} from Nexora CRM")


class RequestInfoView(View):                                         # 10
    def dispatch(self, request, *args, **kwargs):
        print(f"dispatching {request.method} to {self.__class__.__name__}")
        return super().dispatch(request, *args, **kwargs)

    def get(self, request):
        return HttpResponse(f"{request.method} {request.path}")


class DashboardView(PageTitleMixin, TemplateView):                   # 16
```

```
    template_name = "core/dashboard.html"
    page_title = "Dashboard"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        statuses = ["new", "contacted", "qualified", "lost"]
        now = timezone.now()
        context.update(
            total=len(LEADS),
            counts={s: sum(1 for l in LEADS if l["status"] == s) for s in
statuses},
            leads=LEADS,
            activity=[
                {"text": "Aisha Khan called back", "at": now -
timedelta(minutes=5)},
                {"text": "Priya Nair qualified", "at": now -
timedelta(hours=2)},
            ],
        )
        return context


class AboutView(CompanyContextMixin, PageTitleMixin, TemplateView):   # 34
    template_name = "core/about.html"
    page_title = "About"
```

```
<!-- core/templates/core/about.html -->
{% extends "base.html" %}
{% block content %}<h1>About {{ company|default:"Nexora" }} CRM</h1>{%
endblock content %}
```

```
# templates/robots.txt  (36)
User-agent: *
Disallow: /
```

```
# core/urls.py
from django.urls import path
from django.views.generic import TemplateView, RedirectView
from . import views

urlpatterns = [
    path("", views.HomeView.as_view(), name="home"),
    path("about/", views.AboutView.as_view(), name="about"),         # 1 → 34
    # 1 first version: path("about/",
TemplateView.as_view(template_name="core/about.html"), name="about"),
    path("home/", RedirectView.as_view(pattern_name="home", permanent=False),
name="old_home"),   # 4
    path("crm/", RedirectView.as_view(url="/leads/", query_string=True),
name="crm"),            # 5
    path("health/", views.HealthView.as_view(), name="health"),
    path("get-only/", views.GetOnlyView.as_view(), name="get_only"),
    path("hi/", views.GreetingView.as_view(greeting="Marhaba"),
name="greeting"),                # 8
    path("request-info/", views.RequestInfoView.as_view(),
name="request_info"),
    path("dashboard/", views.DashboardView.as_view(), name="dashboard"),
    path("robots.txt", TemplateView.as_view(template_name="robots.txt",
content_type="text/plain")),  # 36
    # keep the remaining Day 1–2 function views (api/info, secret, files,
accounts...) as they are
]
```

`as_view()` only accepts keyword arguments that already exist as class attributes (8).

9.

```
# leads/views.py
class LeadPhoneView(View):
    def get(self, request, *args, **kwargs):
        phone = get_lead(self.kwargs["lead_id"])["phone"]
        return HttpResponse(phone[:4] + "*" * (len(phone) - 8) + phone[-4:])
```

B + C. Leads and clients as classes

11–15, 17, 20–26, 35.

```
# leads/views.py (class versions — keep LEADS, VALID_STATUSES, get_lead)
import csv

from django.http import HttpResponse, JsonResponse
from django.urls import reverse
```

```
from django.utils.decorators import method_decorator
from django.views import View
from django.views.decorators.csrf import csrf_exempt
from django.views.generic import TemplateView, RedirectView

from core.decorators import log_request
from core.mixins import (
    ApiKeyRequiredMixin, JsonResponseMixin, JsonMethodNotAllowedMixin,
PageTitleMixin,
)
from core.utils import json_error


class LeadQueryMixin:                                           # 21
    def get_leads(self):
        params = self.request.GET
        leads = LEADS
        if source := params.get("source"):
            leads = [l for l in leads if l["source"] == source]
        if status := params.get("status"):
            leads = [l for l in leads if l["status"] == status]
        if q := params.get("q", "").strip().lower():
            leads = [l for l in leads if q in l["name"].lower()]
        return leads


@method_decorator(log_request, name="dispatch")                  # 12
class LeadListView(PageTitleMixin, LeadQueryMixin, TemplateView):   # 14
    template_name = "leads/lead_list.html"
    page_title = "Leads"

    def get_template_names(self):                               # 17
        if self.request.GET.get("compact") == "1":
            return ["leads/lead_list_compact.html"]
        return [self.template_name]

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["leads"] = self.get_leads()
        return context


class LeadDetailView(PageTitleMixin, TemplateView):             # 15
    template_name = "leads/lead_detail.html"
    page_title = "Lead"
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead_id = self.kwargs["lead_id"]
        lead = get_lead(lead_id)                                  # raises
Http404
        recent = self.request.session.get("recent_leads", [])
        if lead_id in recent:
            recent.remove(lead_id)
        recent.insert(0, lead_id)
        self.request.session["recent_leads"] = recent[:5]
        context["lead"] = lead
        return context


class LeadCountView(JsonResponseMixin, View):                     # 13 + 20
    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.leads = LEADS

    def get(self, request):
        return self.render_to_json({"total": len(self.leads)})


class LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View):   # 22
    def get(self, request, *args, **kwargs):
        return self.render_to_json(self.get_leads())


class LeadCSVView(LeadQueryMixin, View):                          # 23
    def get(self, request):
        response = HttpResponse(content_type="text/csv")
        response["Content-Disposition"] = 'attachment; filename="leads.csv"'
        writer = csv.DictWriter(response, fieldnames=["id", "name", "phone",
"status", "source"])
        writer.writeheader()
        writer.writerows(self.get_leads())
        return response


@method_decorator(csrf_exempt, name="dispatch")
class QuickAddView(View):                                         # 11
    def get(self, request):
        return HttpResponse("POST a 'name' field to add a lead")

    def post(self, request):
        name = request.POST.get("name", "").strip()
```

```
        lead = {"id": max(l["id"] for l in LEADS) + 1, "name": name,
                "phone": "", "status": "new", "source": "manual"}
        LEADS.append(lead)
        return JsonResponse(lead, status=201)


@method_decorator(csrf_exempt, name="dispatch")
class LeadStatusUpdateView(JsonMethodNotAllowedMixin, View):     # 24 — only
post() exists
    def post(self, request, lead_id):
        lead = get_lead(lead_id)
        status = request.POST.get("status")
        if status not in VALID_STATUSES:
            return json_error("invalid status")
        lead["status"] = status
        return JsonResponse(lead)


@method_decorator(csrf_exempt, name="dispatch")
class LeadDeleteView(View):                                      # 25
    def delete(self, request, lead_id):
        LEADS.remove(get_lead(lead_id))
        return HttpResponse(status=204)


class RecentLeadsView(JsonResponseMixin, View):                  # 26
    def get(self, request):
        ids = request.session.get("recent_leads", [])
        return self.render_to_json([get_lead(i) for i in ids])


class LatestLeadRedirectView(RedirectView):                      # 35
    def get_redirect_url(self, *args, **kwargs):
        return reverse("leads:detail", args=[LEADS[-1]["id"]])
```

```
<!-- leads/templates/leads/lead_list_compact.html  (17) -->
{% extends "base.html" %}
{% block content %}
<ul>{% for lead in leads %}<li>{{ lead.name }} · {{ lead.status }}</li>{%
endfor %}</ul>
{% endblock content %}
```

```
<!-- templates/partials/_navbar.html  (29) -->
{% load static %}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>{{ page_title|default:"Nexora CRM" }}</span>   {# include sees the
parent context #}
</header>
```

27.

```
# clients/views.py
from django.http import Http404
from django.views import View
from core.mixins import JsonResponseMixin


class ClientListView(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json(CLIENTS)


class ClientDetailView(JsonResponseMixin, View):
    def get(self, request, ref):
        for client in CLIENTS:
            if client["ref"] == ref:
                return self.render_to_json(client)
        raise Http404("Client not found")

# clients/urls.py
path("", views.ClientListView.as_view(), name="list"),
path("<uuid:ref>/", views.ClientDetailView.as_view(), name="detail"),
```

28.

```
# leads/views.py
class ApiLeadListView(ApiKeyRequiredMixin, LeadListJSONView):
    pass


class ApiLeadDetailView(ApiKeyRequiredMixin, JsonResponseMixin, View):
    def get(self, request, lead_id):
        return self.render_to_json(get_lead(lead_id))

# leads/api_urls.py
urlpatterns = [
    path("leads/", views.ApiLeadListView.as_view(), name="list"),
    path("leads/<int:lead_id>/", views.ApiLeadDetailView.as_view(),
name="detail"),
]
```

30. Swap the converted routes in `leads/urls.py`; names unchanged. Function views not converted stay as they are — both styles mix fine.

```
# leads/urls.py (changed lines)
path("", views.LeadListView.as_view(), name="list"),
path("count/", views.LeadCountView.as_view(), name="count"),
path("json/", views.LeadListJSONView.as_view(), name="json"),
path("export/", views.LeadCSVView.as_view(), name="export"),
path("quick-add/", views.QuickAddView.as_view(), name="quick_add"),
path("latest/", views.LatestLeadRedirectView.as_view(), name="latest"),
path("recent/", views.RecentLeadsView.as_view(), name="recent"),
path("<int:lead_id>/", views.LeadDetailView.as_view(), name="detail"),
path("<int:lead_id>/phone/", views.LeadPhoneView.as_view(), name="phone"),
path("<int:lead_id>/status/", views.LeadStatusUpdateView.as_view(),
name="update_status"),
path("<int:lead_id>/delete/", views.LeadDeleteView.as_view(), name="delete"),
```

```
python manage.py test     # all Day 1–3 tests should still pass
```

18.

```
curl -i -X OPTIONS http://127.0.0.1:8000/leads/json/
# Allow: GET, HEAD, OPTIONS   (HEAD is added automatically when get() exists)
```

D. Challenge

31.

```
# core/views.py
def simple_as_view(view_class, **initkwargs):
    def view(request, *args, **kwargs):
        self = view_class(**initkwargs)      # View.__init__ sets the
attributes
        self.setup(request, *args, **kwargs)
        return self.dispatch(request, *args, **kwargs)
    return view

# core/urls.py
path("simple-health/", views.simple_as_view(views.HealthView),
name="simple_health"),
```

32.

```
# python manage.py shell
from leads.views import LeadListJSONView
print([c.__name__ for c in LeadListJSONView.__mro__])
# ['LeadListJSONView', 'LeadQueryMixin', 'JsonResponseMixin', 'View',
'object']
```

33. Usage of `RateLimitMixin`:

```
class SearchView(RateLimitMixin, JsonResponseMixin, View):
    rate_limit = 5

    def get(self, request):
        q = request.GET.get("q", "").strip().lower()
        results = [l for l in LEADS if q in l["name"].lower()] if q else []
        return self.render_to_json({"query": q, "count": len(results),
"results": results})
```

37–40.

```
# leads/tests.py (new class)
import json

from django.test import RequestFactory, SimpleTestCase
from django.urls import resolve, reverse

from core.views import HomeView
from .views import LeadListJSONView


class ClassBasedViewTests(SimpleTestCase):
    def test_home_uses_homeview(self):                          # 37
```

```
        self.assertEqual(resolve("/").func.view_class, HomeView)

    def test_json_view_filters_status(self):                    # 38
        request = RequestFactory().get("/leads/json/", {"status": "lost"})
        response = LeadListJSONView.as_view()(request)
        data = json.loads(response.content)
        self.assertTrue(data)
        self.assertTrue(all(l["status"] == "lost" for l in data))

    def test_options_allow_header(self):                        # 39
        response = self.client.options(reverse("leads:json"))
        self.assertIn("GET", response["Allow"])

    def test_status_update_get_is_json_405(self):               # 40
        response = self.client.get(reverse("leads:update_status", args=[1]))
        self.assertEqual(response.status_code, 405)
        self.assertIn("error", response.json())
```

`RequestFactory` skips middleware and URL routing: you call the view directly, so sessions and messages aren't available.

Day 5 — Models and migrations

Goal: real database tables for leads, clients and brokers — built field by field, one migration at a time. Views still use `LEADS` today; Day 6 switches them to the database.

Run `makemigrations` + `migrate` after every exercise that changes a model.

A. Drills (1–10) — fields and migrations

- [ ] **1.** `Lead` model in `leads/models.py` with only `name` (max 120). Make and apply the migration.
- [ ] **2.** Print the SQL of that migration with `sqlmigrate`.
- [ ] **3.** Add `phone` (max 15) and `email` (optional).
- [ ] **4.** Add `status` using a nested `TextChoices` class `Status` (new, contacted, qualified, lost), default new.
- [ ] **5.** Add `source` using `TextChoices` `Source` (facebook, google, referral, walk_in, manual), default manual.
- [ ] **6.** Add `created_at` (set once on insert) and `updated_at` (set on every save).
- [ ] **7.** Add `budget`: decimal, 12 digits, 2 decimal places, nullable.
- [ ] **8.** Add `is_active` (default True) and `notes` (optional long text).
- [ ] **9.** `__str__` returning `Aisha Khan (New)` via `get_status_display()`; `get_absolute_url` to `leads:detail`.
- [ ] **10.** `showmigrations leads`; migrate `leads` back to zero, then forward again.

B. Core (11–20) — Meta, constraints, behaviour

- [ ] **11.** `Meta`: newest first, `verbose_name` and `verbose_name_plural`.
- [ ] **12.** Rename the table to `crm_leads` with `db_table`.
- [ ] **13.** Composite index on `status` + `created_at`, named.
- [ ] **14.** `UniqueConstraint` on `phone`.
- [ ] **15.** `CheckConstraint`: budget must be ⩾ 0 (Django 5.1+ uses `condition=`).
- [ ] **16.** `phone`: verbose name `Phone number` and a help text; index `email`.
- [ ] **17.** Register `Lead` in the admin, create a superuser, add two leads by hand.
- [ ] **18.** In the shell: create a lead, read `status` vs `get_status_display()`, list `Lead.Status.choices`.
- [ ] **19.** Property `is_hot` (new + paid source) and method `mark_contacted()` saving only the changed fields.

- [ ] **20.** Data migration (`--empty` , `RunPython` with a reverse function) that loads the 5 `LEADS` dicts into the table.

C. Nexora CRM (21–30)

- [ ] **21.** `Client` model in `clients` : `ref` (UUID, auto, not editable, unique), name, email, account_number, `account_type` (standard/gold/vip), balance, `kyc_status` (pending/approved/rejected), date_joined, is_active.
- [ ] **22.** `Client.Meta` : order by name; unique account number; balance ⩾ 0.
- [ ] **23.** `RegexValidator` on `account_number` : exactly 7 digits.
- [ ] **24.** New `brokers` app + `Broker` : code (unique), name, email, commission_rate (0–100 with min/max validators), `tier` as `IntegerChoices` 1–3, is_active, created_at.
- [ ] **25.** `RegexValidator` on broker code: `BRF` + 4 digits.
- [ ] **26.** UAE mobile validator on `Lead.phone` : `9715` + 8 digits.
- [ ] **27.** Register `Client` and `Broker` in the admin.
- [ ] **28.** In the shell, prove validators run on `full_clean()` but not on `save()` .
- [ ] **29.** Dialer fields on `Lead` : `contact_attempts` (small positive int, default 0) and `last_contacted_at` (nullable); update `mark_contacted()` to use them.
- [ ] **30.** `__str__` + `get_absolute_url` for `Client` (by `ref`); `__str__` for `Broker` (`BRF1001 — Name`); property `Client.is_kyc_approved` .

D. Challenge (31–40)

- [ ] **31.** Override `Lead.save()` to normalise the phone (`+971 50 123 4567` , `00971...` , `050...` → `971501234567`) and title-case the name.
- [ ] **32.** `Lead.clean()` : a lost lead must have notes. Check it with `full_clean()` in the shell.
- [ ] **33.** Rename `budget` → `expected_deposit` ; answer yes to the rename prompt; fix the constraint.
- [ ] **34.** Add required `country` (max 2) with no default; give `AE` as the one-off default in the prompt; then add `default="AE"` to the model.
- [ ] **35.** Roll `leads` back one migration, then forward again.
- [ ] **36.** CI check: fail if model changes have no migration (`makemigrations --check --dry-run`).
- [ ] **37.** `LeadModelTests(TestCase)` : default status, `__str__` (after title-casing), Meta ordering.
- [ ] **38.** Test duplicate phone raises `IntegrityError` .

- [ ] **39.** Test negative `expected_deposit` raises `IntegrityError`.
- [ ] **40.** Test `full_clean()` rejects a bad lead phone and a bad broker code.

Solutions

Final `Lead` model (1–9, 11–16, 19, 26, 29, 31–34)

The exercise number sits beside each line. Build it step by step, migrating as you go.

```
# leads/models.py
from django.core.exceptions import ValidationError
from django.core.validators import RegexValidator
from django.db import models
from django.urls import reverse
from django.utils import timezone

uae_phone_validator = RegexValidator(r"^9715\d{8}$", "Use UAE mobile format:
9715XXXXXXXX")   # 26


def normalize_uae_phone(raw):
# 31
    digits = "".join(ch for ch in raw if ch.isdigit())
    if digits.startswith("00"):
        digits = digits[2:]
    if digits.startswith("05"):
        digits = "971" + digits[1:]
    return digits


class Lead(models.Model):
    class Status(models.TextChoices):
# 4
        NEW = "new", "New"
        CONTACTED = "contacted", "Contacted"
        QUALIFIED = "qualified", "Qualified"
        LOST = "lost", "Lost"

    class Source(models.TextChoices):
# 5
        FACEBOOK = "facebook", "Facebook"
        GOOGLE = "google", "Google"
        REFERRAL = "referral", "Referral"
        WALK_IN = "walk_in", "Walk-in"
        MANUAL = "manual", "Manual"
```

```
    PAID_SOURCES = {Source.FACEBOOK, Source.GOOGLE}

    name = models.CharField(max_length=120)
# 1
    phone = models.CharField(
# 3, 16, 26
        "Phone number", max_length=15,
        validators=[uae_phone_validator], help_text="9715XXXXXXXX",
    )
    email = models.EmailField(blank=True, db_index=True)
# 3, 16
    status = models.CharField(max_length=20, choices=Status.choices,
default=Status.NEW)
    source = models.CharField(max_length=20, choices=Source.choices,
default=Source.MANUAL)
    expected_deposit = models.DecimalField(
# 7 (was budget), 33
        max_digits=12, decimal_places=2, null=True, blank=True,
    )
    country = models.CharField(max_length=2, default="AE")
# 34
    is_active = models.BooleanField(default=True)
# 8
    notes = models.TextField(blank=True)
    contact_attempts = models.PositiveSmallIntegerField(default=0)
# 29
    last_contacted_at = models.DateTimeField(null=True, blank=True)
# 29
    created_at = models.DateTimeField(auto_now_add=True)
# 6
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = "crm_leads"
# 12
        ordering = ["-created_at"]
# 11
        verbose_name = "lead"
        verbose_name_plural = "leads"
        indexes = [
# 13
            models.Index(fields=["status", "created_at"],
name="lead_status_created_idx"),
        ]
        constraints = [
```

```
            models.UniqueConstraint(fields=["phone"],
name="lead_unique_phone"),        # 14
            models.CheckConstraint(
# 15, 33
                condition=models.Q(expected_deposit__gte=0),
                name="lead_deposit_non_negative",
            ),
        ]

    def __str__(self):
# 9
        return f"{self.name} ({self.get_status_display()})"

    def get_absolute_url(self):
        return reverse("leads:detail", kwargs={"lead_id": self.pk})

    @property
    def is_hot(self):
# 19
        return self.status == self.Status.NEW and self.source in
self.PAID_SOURCES

    def mark_contacted(self):
# 19 → 29
        self.status = self.Status.CONTACTED
        self.contact_attempts += 1
        self.last_contacted_at = timezone.now()
        self.save(update_fields=[
            "status", "contact_attempts", "last_contacted_at",
            "updated_at",   # auto_now only updates when listed in
update_fields
        ])

    def clean(self):
# 32
        if self.status == self.Status.LOST and not self.notes.strip():
            raise ValidationError({"notes": "Give a reason when marking a
lead lost."})

    def save(self, *args, **kwargs):
# 31
        self.phone = normalize_uae_phone(self.phone)
        self.name = self.name.strip().title()
        super().save(*args, **kwargs)
```

`UniqueConstraint` on one field = `unique=True` on the field; the constraint form is preferred because it's named and extensible (14).

A. Drills — commands

1–2, 10.

```
python manage.py makemigrations leads
python manage.py migrate
python manage.py sqlmigrate leads 0001      # 2

python manage.py showmigrations leads       # 10
python manage.py migrate leads zero         # drops the leads tables
python manage.py migrate leads
```

B. Core

17.

```
# leads/admin.py
from django.contrib import admin
from .models import Lead

admin.site.register(Lead)
```

```
python manage.py createsuperuser
```

18.

```
# python manage.py shell
from leads.models import Lead

lead = Lead.objects.create(name="Zara Ahmed", phone="971507770001",
source="google")
lead.status                    # 'new'
lead.get_status_display()      # 'New'
Lead.Status.choices            # [('new', 'New'), ('contacted', 'Contacted'),
...]
Lead.Status.values             # ['new', 'contacted', 'qualified', 'lost']
lead.is_hot                    # True
```

20.

```
python manage.py makemigrations leads --empty --name seed_initial_leads
```

```
# leads/migrations/000X_seed_initial_leads.py
from django.db import migrations

INITIAL = [
    {"name": "Aisha Khan", "phone": "971501234567", "status": "new",
"source": "facebook"},
    {"name": "Omar Haddad", "phone": "971502345678", "status": "contacted",
"source": "google"},
    {"name": "Priya Nair", "phone": "971503456789", "status": "qualified",
"source": "referral"},
    {"name": "Daniel Ortiz", "phone": "971504567890", "status": "new",
"source": "google"},
    {"name": "Mei Lin", "phone": "971505678901", "status": "lost", "source":
"facebook"},
]


def seed(apps, schema_editor):
    Lead = apps.get_model("leads", "Lead")     # historical model: no custom
save()/methods
    for row in INITIAL:
        Lead.objects.get_or_create(phone=row["phone"], defaults=row)


def unseed(apps, schema_editor):
    Lead = apps.get_model("leads", "Lead")
    Lead.objects.filter(phone__in=[r["phone"] for r in INITIAL]).delete()


class Migration(migrations.Migration):
    dependencies = [("leads", "000N_previous")]   # filled in for you by --
empty
    operations = [migrations.RunPython(seed, unseed)]
```

Data migrations also run when the test database is built, so these 5 rows exist in every test.

C. Nexora CRM

21–23, 30.

```
# clients/models.py
import uuid

from django.core.validators import RegexValidator
from django.db import models
```

```
from django.urls import reverse


class Client(models.Model):
    class AccountType(models.TextChoices):
        STANDARD = "standard", "Standard"
        GOLD = "gold", "Gold"
        VIP = "vip", "VIP"

    class KYCStatus(models.TextChoices):
        PENDING = "pending", "Pending"
        APPROVED = "approved", "Approved"
        REJECTED = "rejected", "Rejected"

    ref = models.UUIDField(default=uuid.uuid4, editable=False, unique=True)
    name = models.CharField(max_length=120)
    email = models.EmailField()
    account_number = models.CharField(
# 23
        max_length=7, validators=[RegexValidator(r"^\d{7}$", "Account number
must be 7 digits")],
    )
    account_type = models.CharField(max_length=10,
choices=AccountType.choices, default=AccountType.STANDARD)
    balance = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    kyc_status = models.CharField(max_length=10, choices=KYCStatus.choices,
default=KYCStatus.PENDING)
    date_joined = models.DateField(auto_now_add=True)
    is_active = models.BooleanField(default=True)

    class Meta:
# 22
        ordering = ["name"]
        constraints = [
            models.UniqueConstraint(fields=["account_number"],
name="client_unique_account"),
            models.CheckConstraint(condition=models.Q(balance__gte=0),
name="client_balance_non_negative"),
        ]

    def __str__(self):
# 30
        return f"{self.name} ({self.account_number})"

    def get_absolute_url(self):
        return reverse("clients:detail", kwargs={"ref": self.ref})
```

```
    @property
    def is_kyc_approved(self):
        return self.kyc_status == self.KYCStatus.APPROVED
```

24–25, 30.

```
python manage.py startapp brokers     # add "brokers.apps.BrokersConfig" to
INSTALLED_APPS
```

```
# brokers/models.py
from decimal import Decimal

from django.core.validators import MaxValueValidator, MinValueValidator,
RegexValidator
from django.db import models


class Broker(models.Model):
    class Tier(models.IntegerChoices):
        TIER_1 = 1, "Tier 1"
        TIER_2 = 2, "Tier 2"
        TIER_3 = 3, "Tier 3"

    code = models.CharField(
# 25
        max_length=7, unique=True,
        validators=[RegexValidator(r"^BRF\d{4}$", "Format: BRF0000")],
    )
    name = models.CharField(max_length=120)
    email = models.EmailField(blank=True)
    commission_rate = models.DecimalField(
        max_digits=5, decimal_places=2, default=Decimal("0.00"),
        validators=[MinValueValidator(Decimal("0")),
MaxValueValidator(Decimal("100"))],
    )
    tier = models.PositiveSmallIntegerField(choices=Tier.choices,
default=Tier.TIER_3)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["code"]
```

```
    def __str__(self):
# 30
        return f"{self.code} — {self.name}"
```

27.

```
# clients/admin.py
from django.contrib import admin
from .models import Client

admin.site.register(Client)

# brokers/admin.py
from django.contrib import admin
from .models import Broker

admin.site.register(Broker)
```

28.

```
# python manage.py shell
from django.core.exceptions import ValidationError
from leads.models import Lead

bad = Lead(name="Test", phone="12345")
try:
    bad.full_clean()                # runs field validators + clean()
except ValidationError as e:
    print(e.message_dict)           # {'phone': ['Use UAE mobile format:
9715XXXXXXXX']}

bad.save()                          # no validation — the row is saved
bad.delete()
```

Forms (Day 13) call `full_clean()` for you; `save()` and `objects.create()` never do.

D. Challenge

32.

```
lead = Lead(name="Lost One", phone="971507770005", status=Lead.Status.LOST)
lead.full_clean()      # ValidationError: {'notes': ['Give a reason when
marking a lead lost.']}
```

33–36.

```
# 33: rename budget → expected_deposit in the model AND in the
CheckConstraint's Q(...)
python manage.py makemigrations leads
# Was lead.budget renamed to lead.expected_deposit (a DecimalField)? [y/N] y
python manage.py migrate

# 34: add country = models.CharField(max_length=2) with no default
python manage.py makemigrations leads
# 1) Provide a one-off default now ... Select an option: 1
# >>> "AE"
# then add default="AE" to the field and run makemigrations again

# 35
python manage.py showmigrations leads            # note the second-to-last
number
python manage.py migrate leads 000N              # roll back to it
python manage.py migrate leads                   # forward again

# 36: exits non-zero if a migration is missing
python manage.py makemigrations --check --dry-run
```

37–40.

```
# leads/tests.py (new class — TestCase, because it touches the database)
from decimal import Decimal

from django.core.exceptions import ValidationError
from django.db import IntegrityError, transaction
from django.test import TestCase

from brokers.models import Broker
from .models import Lead


class LeadModelTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="test lead",
phone="971509999001")

    def test_defaults_and_str(self):                              # 37
        self.assertEqual(self.lead.status, Lead.Status.NEW)
        self.assertEqual(str(self.lead), "Test Lead (New)")       # save()
title-cased it
        self.assertEqual(Lead._meta.ordering, ["-created_at"])
```

```
    def test_unique_phone(self):                                    # 38
        with self.assertRaises(IntegrityError), transaction.atomic():
            Lead.objects.create(name="Dup", phone="971509999001")

    def test_negative_deposit_rejected(self):                       # 39
        with self.assertRaises(IntegrityError), transaction.atomic():
            Lead.objects.create(name="Neg", phone="971509999002",
expected_deposit=Decimal("-1"))

    def test_validators(self):                                      # 40
        with self.assertRaises(ValidationError):
            Lead(name="Bad", phone="12345").full_clean()
        with self.assertRaises(ValidationError):
            Broker(code="XYZ1", name="Bad broker").full_clean()
```

`transaction.atomic()` inside `assertRaises` rolls back the failed insert so the rest of the test can keep using the database.

Day 6 — ORM CRUD and the shell

Goal: fluent create/read/update/delete in the shell, 200 realistic seeded leads, and the lead views running on the database instead of `LEADS`.

Do A and B in `python manage.py shell`.

A. Drills (1–10) — reading and writing

- [ ] **1.** Import `Lead` and print how many rows exist.
- [ ] **2.** Create a lead with `objects.create()`.
- [ ] **3.** Create one by instantiating `Lead(...)` and calling `save()`; print its `pk`.
- [ ] **4.** `get()` by pk; catch `Lead.DoesNotExist` for pk 99999.
- [ ] **5.** `filter()` new leads; print the count.
- [ ] **6.** `exclude()` lost leads.
- [ ] **7.** `first()`, `last()`, `latest("created_at")`, `earliest("created_at")` — note how `first()` follows Meta ordering.
- [ ] **8.** Check whether any lead has source `walk_in` with `exists()`.
- [ ] **9.** Five leads by name A→Z; five newest.
- [ ] **10.** `values()` (name, status) for 3 rows; `values_list("phone", flat=True)` for 3 rows.

B. Core (11–20) — updates, deletes, lookups

- [ ] **11.** Change one lead's status and save only that field (plus `updated_at`).
- [ ] **12.** Bulk-update every new Google lead to contacted with `queryset.update()`; print the row count.
- [ ] **13.** Delete one lead; delete a filtered queryset; read the returned tuples.
- [ ] **14.** `get_or_create` by phone with `defaults`.
- [ ] **15.** `update_or_create` by phone, changing status.
- [ ] **16.** `in_bulk()` by pk list and by phone.
- [ ] **17.** Distinct list of sources — then see why default ordering breaks `distinct()` and fix it.
- [ ] **18.** Lookups: `icontains`, `startswith`, `__date` = today, `gte` on deposit, `endswith` on phone, `__in` on status, last 7 days.
- [ ] **19.** Chain filter → exclude → order_by and print `qs.query` before it runs.
- [ ] **20.** `count()` vs `len()`; loop 200 rows with `.iterator(chunk_size=50)`.

C. Nexora CRM (21–30) — seed + switch views to the DB

- [ ] **21.** `leads/seed.py` with `run(n=200, seed=42)`: random names, unique UAE phones, weighted statuses, sources, deposits, notes for lost leads, `created_at` spread over 60 days. Use `bulk_create`.
- [ ] **22.** Run the seed in the shell; print the count per status by looping `Lead.Status.choices`.
- [ ] **23.** `LeadQueryMixin.get_leads()` returns a queryset (active leads, `source`, `status`, `q` via `icontains`).
- [ ] **24.** `LeadDetailView` uses `get_object_or_404`; detail page shows `get_source_display`.
- [ ] **25.** `DashboardView` counts per status from the DB; `status_count` tag takes a queryset; lead cards show the 6 newest.
- [ ] **26.** `LeadListJSONView` returns `values()` rows from the DB.
- [ ] **27.** `LeadStatusUpdateView` validates against `Lead.Status.values` and saves to the DB.
- [ ] **28.** `LeadDeleteView` deletes the row.
- [ ] **29.** `LeadCSVView` exports from the DB with `values_list`.
- [ ] **30.** `StaleLeadsView` at `leads/stale/`: new leads older than 7 days, oldest first, JSON.

D. Challenge (31–40)

- [ ] **31.** Clone a lead (new pk, new phone) by resetting `pk` and `_state.adding`.
- [ ] **32.** Show an in-memory object going stale after `queryset.update()`; fix with `refresh_from_db()`.
- [ ] **33.** Soft delete: `Lead.archive()` sets `is_active=False`; archived leads disappear from the list.
- [ ] **34.** Trigger and catch `Lead.MultipleObjectsReturned`.
- [ ] **35.** Count queries with `CaptureQueriesContext`: `len(qs)` twice vs `count()` twice on a fresh queryset.
- [ ] **36.** Show `qs[0]` hits the DB on every call until the queryset is evaluated.
- [ ] **37.** Raw SQL two ways: `Lead.objects.raw()` with params, and `connection.cursor()` for a GROUP BY.
- [ ] **38.** Update the Day 1–4 lead tests for the DB: `TestCase` base, escape test creates a real row.
- [ ] **39.** Test `update_status` changes the row (`refresh_from_db`).
- [ ] **40.** Test `delete` removes the row.

Solutions

Tip: run the seed (21–22) first if you want bigger result sets for A and B.

A. Drills

1–10.

```
# python manage.py shell
from leads.models import Lead

Lead.objects.count()                                                    # 1

Lead.objects.create(name="Zara Ahmed", phone="971507770011", source="google")
# 2

lead = Lead(name="Karim Aziz", phone="971507770012")                    # 3
lead.save()
lead.pk

try:                                                                    # 4
    Lead.objects.get(pk=99999)
except Lead.DoesNotExist:
    print("not found")

new = Lead.objects.filter(status=Lead.Status.NEW)                       # 5
new.count()

Lead.objects.exclude(status=Lead.Status.LOST).count()                   # 6

Lead.objects.first()                  # 7: newest — Meta ordering is -
created_at
Lead.objects.last()                   # oldest
Lead.objects.latest("created_at")
Lead.objects.earliest("created_at")

Lead.objects.filter(source=Lead.Source.WALK_IN).exists()                # 8

Lead.objects.order_by("name")[:5]                                       # 9
Lead.objects.order_by("-created_at")[:5]

Lead.objects.values("name", "status")[:3]                               #
10: list of dicts
Lead.objects.values_list("phone", flat=True)[:3]                        #
flat list
```

B. Core

11–20.

```
from datetime import timedelta
from django.utils import timezone

lead = Lead.objects.get(phone="971507770011")                                  # 11
lead.status = Lead.Status.CONTACTED
lead.save(update_fields=["status", "updated_at"])

n = (Lead.objects                                                              # 12
     .filter(source="google", status="new")
     .update(status="contacted", updated_at=timezone.now()))
print(n)          # rows changed — update() skips save(): no auto_now, no
normalising

Lead.objects.get(phone="971507770012").delete()                                # 13
→ (1, {'leads.Lead': 1})
Lead.objects.filter(name__startswith="Zara").delete()

lead, created = Lead.objects.get_or_create(                                    # 14
    phone="971507770013",
    defaults={"name": "Huda Saleh", "source": "referral"},
)

lead, created = Lead.objects.update_or_create(                                 # 15
    phone="971507770013",
    defaults={"status": "qualified"},          # Django 5.0+:
create_defaults={...} for create-only values
)

ids = list(Lead.objects.values_list("pk", flat=True)[:3])                      # 16
Lead.objects.in_bulk(ids)                                                      #
{pk: <Lead>, ...}
Lead.objects.in_bulk(["971501234567"], field_name="phone")                     #
field must be unique

list(Lead.objects.values_list("source", flat=True).distinct())                # 17:
duplicates!
# Meta ordering adds created_at to SELECT DISTINCT. Fix by clearing or
setting ordering:
list(Lead.objects.order_by().values_list("source", flat=True).distinct())
list(Lead.objects.order_by("source").values_list("source",
flat=True).distinct())

Lead.objects.filter(name__icontains="khan")                                    # 18
```

```
Lead.objects.filter(name__startswith="A")
Lead.objects.filter(created_at__date=timezone.localdate())
Lead.objects.filter(expected_deposit__gte=10_000)
Lead.objects.filter(phone__endswith="67")
Lead.objects.filter(status__in=["new", "contacted"])
Lead.objects.filter(created_at__gte=timezone.now() - timedelta(days=7))

qs = (Lead.objects                                                    # 19
      .filter(status="new")
      .exclude(source="manual")
      .order_by("name"))
print(qs.query)      # SQL text — nothing has run yet (querysets are lazy)
list(qs)             # now it runs

Lead.objects.count()           # 20: SELECT COUNT(*)
len(Lead.objects.all())        # loads every row, then counts in Python
for lead in Lead.objects.iterator(chunk_size=50):
    pass                       # streams rows, no result cache
```

C. Nexora CRM

21.

```
# leads/seed.py
import random
from datetime import timedelta
from decimal import Decimal

from django.utils import timezone

from .models import Lead

FIRST = ["Aisha", "Omar", "Priya", "Daniel", "Mei", "Sara", "Ali", "Fatima",
"Ravi",
         "Elena", "Yusuf", "Noor", "Arjun", "Lina", "Hassan", "Chen",
"Maria", "Tariq"]
LAST = ["Khan", "Haddad", "Nair", "Ortiz", "Lin", "Al Mansoori", "Sharma",
"Petrova",
        "Rahman", "Iyer", "Saleh", "Costa", "Wong", "Farouk"]
STATUS_WEIGHTS = [40, 30, 15, 15]          # new, contacted, qualified, lost


def run(n=200, seed=42):
    rng = random.Random(seed)
    used = set(Lead.objects.values_list("phone", flat=True))
    batch = []
```

```
    while len(batch) < n:
        phone = f"9715{rng.randint(0, 99_999_999):08d}"
        if phone in used:
            continue
        used.add(phone)
        status = rng.choices(Lead.Status.values, weights=STATUS_WEIGHTS)[0]
        batch.append(Lead(
            name=f"{rng.choice(FIRST)} {rng.choice(LAST)}",
            phone=phone,
            status=status,
            source=rng.choice(Lead.Source.values),
            expected_deposit=Decimal(rng.randrange(500, 50_000, 500)),
            notes="No response after 3 attempts" if status ==
Lead.Status.LOST else "",
        ))
    Lead.objects.bulk_create(batch)       # few INSERTs; skips save() and
clean()

    # auto_now_add overwrote created_at on insert → backdate afterwards
    now = timezone.now()
    created = list(Lead.objects.filter(phone__in=[l.phone for l in batch]))
    for lead in created:
        lead.created_at = now - timedelta(days=rng.randint(0, 60),
hours=rng.randint(0, 23))
    Lead.objects.bulk_update(created, ["created_at"], batch_size=100)
    return len(created)
```

22.

```
from leads.seed import run
run()            # 200 — each call adds n more

for value, label in Lead.Status.choices:
    print(f"{label:<10} {Lead.objects.filter(status=value).count()}")
```

23–24, 26–30.

```
# leads/views.py (DB versions — replace the LEADS-based classes)
from datetime import timedelta

from django.shortcuts import get_object_or_404
from django.utils import timezone

from .models import Lead

LEAD_FIELDS = ("id", "name", "phone", "status", "source")
```

```
class LeadQueryMixin:                                              # 23
    def get_leads(self):
        qs = Lead.objects.filter(is_active=True)                   # 33:
archived leads hidden
        params = self.request.GET
        if source := params.get("source"):
            qs = qs.filter(source=source)
        if status := params.get("status"):
            qs = qs.filter(status=status)
        if q := params.get("q", "").strip():
            qs = qs.filter(name__icontains=q)
        return qs


class LeadDetailView(PageTitleMixin, TemplateView):                # 24
    template_name = "leads/lead_detail.html"
    page_title = "Lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead_id = self.kwargs["lead_id"]
        lead = get_object_or_404(Lead, pk=lead_id)
        recent = self.request.session.get("recent_leads", [])
        if lead_id in recent:
            recent.remove(lead_id)
        recent.insert(0, lead_id)
        self.request.session["recent_leads"] = recent[:5]
        context["lead"] = lead
        return context


class LeadListJSONView(LeadQueryMixin, JsonResponseMixin, View):   # 26
    def get(self, request, *args, **kwargs):
        return
self.render_to_json(list(self.get_leads().values(*LEAD_FIELDS)))


class LeadCSVView(LeadQueryMixin, View):                           # 29
    def get(self, request):
        response = HttpResponse(content_type="text/csv")
        response["Content-Disposition"] = 'attachment; filename="leads.csv"'
        writer = csv.writer(response)
        writer.writerow(LEAD_FIELDS)
        writer.writerows(self.get_leads().values_list(*LEAD_FIELDS))
```

```
        return response


@method_decorator(csrf_exempt, name="dispatch")
class LeadStatusUpdateView(JsonMethodNotAllowedMixin, View):     # 27
    def post(self, request, lead_id):
        lead = get_object_or_404(Lead, pk=lead_id)
        status = request.POST.get("status")
        if status not in Lead.Status.values:
            return json_error("invalid status")
        lead.status = status
        lead.save(update_fields=["status", "updated_at"])
        return JsonResponse({"id": lead.pk, "status": lead.status})


@method_decorator(csrf_exempt, name="dispatch")
class LeadDeleteView(View):                                      # 28
    def delete(self, request, lead_id):
        get_object_or_404(Lead, pk=lead_id).delete()
        return HttpResponse(status=204)


class StaleLeadsView(JsonResponseMixin, View):                   # 30
    def get(self, request):
        cutoff = timezone.now() - timedelta(days=7)
        qs = (Lead.objects
              .filter(status=Lead.Status.NEW, created_at__lt=cutoff)
              .order_by("created_at"))
        return self.render_to_json({
            "count": qs.count(),
            "results": list(qs.values("id", "name", "phone", "created_at")
[:50]),
        })


# switch these too so nothing points at stale LEADS ids
class LeadCountView(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json({"total":
Lead.objects.filter(is_active=True).count()})


class RecentLeadsView(JsonResponseMixin, View):
    def get(self, request):
        ids = request.session.get("recent_leads", [])
        rows = {r["id"]: r for r in
```

```
Lead.objects.filter(pk__in=ids).values(*LEAD_FIELDS)}
        return self.render_to_json([rows[i] for i in ids if i in rows])   #
keep session order


class LatestLeadRedirectView(RedirectView):
    def get_redirect_url(self, *args, **kwargs):
        return Lead.objects.latest("created_at").get_absolute_url()

# leads/urls.py
path("stale/", views.StaleLeadsView.as_view(), name="stale"),
```

```
<!-- leads/templates/leads/lead_detail.html (24: change one line) -->
<dt>Source</dt><dd>{{ lead.get_source_display }}</dd>
```

25.

```
# core/views.py
from leads.models import Lead


class DashboardView(PageTitleMixin, TemplateView):
    template_name = "core/dashboard.html"
    page_title = "Dashboard"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        active = Lead.objects.filter(is_active=True)
        now = timezone.now()
        context.update(
            total=active.count(),
            counts={label: active.filter(status=value).count()     # 4
queries — Day 9 makes it 1
                    for value, label in Lead.Status.choices},
            all_leads=active,
            leads=active[:6],                                      # newest 6
            activity=[
                {"text": "Aisha Khan called back", "at": now -
timedelta(minutes=5)},
                {"text": "Priya Nair qualified", "at": now -
timedelta(hours=2)},
            ],
        )
        return context
```

```
# leads/templatetags/crm_filters.py — status_count now takes a queryset
@register.simple_tag
def status_count(leads, status):
    return leads.filter(status=status).count()
```

```
<!-- core/dashboard.html (changed line) — a sliced queryset can't be
filtered, so pass all_leads -->
{% status_count all_leads "new" as new_count %}
```

D. Challenge

31–34.

```
# 31: clone
clone = Lead.objects.first()
original_pk = clone.pk
clone.pk = None
clone._state.adding = True
clone.phone = "971507770020"
clone.save()
clone.pk != original_pk          # True

# 32: stale object
lead = Lead.objects.filter(status="new").first()
Lead.objects.filter(pk=lead.pk).update(status="qualified")
lead.status                      # 'new' — Python object didn't change
lead.refresh_from_db()
lead.status                      # 'qualified'

# 34
try:
    Lead.objects.get(status="new")
except Lead.MultipleObjectsReturned:
    print("More than one match — use filter() or a unique field")
```

```
# leads/models.py — 33
def archive(self):
    self.is_active = False
    self.save(update_fields=["is_active", "updated_at"])

# shell
Lead.objects.first().archive()
Lead.objects.filter(is_active=False).count()     # 1 — and it's gone from
/leads/
```

35–36.

```
from django.db import connection
from django.test.utils import CaptureQueriesContext

qs = Lead.objects.filter(status="new")
with CaptureQueriesContext(connection) as ctx:
    len(qs)
    len(qs)
print(len(ctx.captured_queries))     # 1 — second len() used the result cache

fresh = Lead.objects.filter(status="new")
with CaptureQueriesContext(connection) as ctx:
    fresh.count()
    fresh.count()
print(len(ctx.captured_queries))     # 2 — unevaluated queryset: count()
always queries
print(ctx.captured_queries[0]["sql"])

lazy = Lead.objects.all()            # 36
with CaptureQueriesContext(connection) as ctx:
    lazy[0]
    lazy[0]
print(len(ctx.captured_queries))     # 2 — each index is its own LIMIT 1
query
list(lazy)                           # evaluate once
with CaptureQueriesContext(connection) as ctx:
    lazy[0]
    lazy[0]
print(len(ctx.captured_queries))     # 0 — served from cache
```

37.

```
for lead in Lead.objects.raw(
    "SELECT * FROM crm_leads WHERE status = %s ORDER BY created_at DESC LIMIT
5", ["new"]
):
    print(lead.pk, lead.name)

with connection.cursor() as cursor:
    cursor.execute("SELECT status, COUNT(*) FROM crm_leads GROUP BY status
ORDER BY 2 DESC")
    print(cursor.fetchall())
# always pass values as params — never f-string user input into SQL
```

38–40.

```
# leads/tests.py
from django.test import TestCase
from .models import Lead


class LeadViewTests(TestCase):                # 38: was SimpleTestCase —
these views now query the DB
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="Badge Test",
phone="9715099999100")

    # ...keep the existing Day 1–3 tests...

    def test_lead_name_is_escaped(self):       # 38: replaces the
LEADS.append version
        Lead.objects.bulk_create([             # bulk_create skips save() →
name not title-cased
            Lead(name="<script>alert(1)</script>", phone="9715099999101"),
        ])
        response = self.client.get(reverse("leads:list"))
        self.assertNotContains(response, "<script>alert(1)</script>")
        self.assertContains(response,
"&lt;script&gt;alert(1)&lt;/script&gt;")

    def test_update_status_changes_db(self):   # 39
        response = self.client.post(
            reverse("leads:update_status", args=[self.lead.pk]), {"status":
"qualified"}
        )
        self.assertEqual(response.status_code, 200)
        self.lead.refresh_from_db()
        self.assertEqual(self.lead.status, Lead.Status.QUALIFIED)

    def test_delete_removes_row(self):         # 40
        lead = Lead.objects.create(name="Delete Me", phone="9715099999102")
        response = self.client.delete(reverse("leads:delete", args=
[lead.pk]))
        self.assertEqual(response.status_code, 204)
        self.assertFalse(Lead.objects.filter(pk=lead.pk).exists())


class ClassBasedViewTests(TestCase):          # 38: Day 4 class → TestCase
too
    ...
```

```
python manage.py test
git add -A && git commit -m "day 6"
```

Day 7 — Lookups, Q, F, ordering, values

Goal: express any lead question as a queryset — lookups, `Q` for OR/NOT, `F` for field-to-field maths — and package the CRM's everyday questions (search, stale, follow-up, burned, high value) into reusable query functions.

Do A and B in the shell.

A. Drills (1–10) — field lookups

- [ ] **1.** `exact` vs `iexact` on name.
- [ ] **2.** `contains` vs `icontains` — and why they behave the same on SQLite.
- [ ] **3.** `startswith`, `istartswith`, `endswith` on name and phone.
- [ ] **4.** `in` with a list of statuses, and with `Lead.PAID_SOURCES`.
- [ ] **5.** `gt`, `gte`, `lte` on `expected_deposit`; `range` between 5,000 and 20,000.
- [ ] **6.** Leads never contacted (`isnull`).
- [ ] **7.** Date parts: this year + month, Fridays (`week_day`), last 7 days with `__date__range`.
- [ ] **8.** `regex`: phones on the 050 prefix (`97150...`).
- [ ] **9.** Blank vs NULL: count leads with no email both ways; see which one works.
- [ ] **10.** Ordering: two fields, random (`?`), `last_contacted_at` desc with NULLs last.

B. Core (11–20) — Q, F, values

- [ ] **11.** OR: new leads OR referral leads.
- [ ] **12.** AND with `&`, NOT with `~`: new leads not from paid sources.
- [ ] **13.** Mix a `Q` with keyword filters in one `filter()` call (order matters).
- [ ] **14.** Build an OR `Q` dynamically from a list of sources (`reduce` or a loop with `|=`).
- [ ] **15.** Add 1 to `contact_attempts` for every contacted lead in one SQL statement.
- [ ] **16.** Leads updated more than a day after they were created (`F` + `timedelta`).
- [ ] **17.** `F()` on an instance: increment, save, see what the attribute holds, `refresh_from_db()`.
- [ ] **18.** `values()` with renamed keys via `F`.
- [ ] **19.** `values_list(..., named=True)` and read rows as named tuples.
- [ ] **20.** Database functions: names longer than 12 chars (`Length`), case-insensitive sort (`Lower`).

C. Nexora CRM (21–30) — `leads/queries.py`

- [ ] **21.** `search_q(term)` → a `Q` matching name or email (icontains) or phone digits (≥ 4 digits, ignoring `+` and spaces); `search_leads(term)` uses it.
- [ ] **22.** `stale_new_leads(days=7)`: new, older than N days, never contacted, oldest first.
- [ ] **23.** `follow_up_due(hours=48, max_attempts=3)`: contacted, last touch older than N hours, under the attempt cap.
- [ ] **24.** `burned_leads(max_attempts=3)`: still open (new/contacted) with ≥ 3 attempts.
- [ ] **25.** `high_value(threshold=20_000)`: deposit ≥ threshold, not lost, biggest first.
- [ ] **26.** `today_new()`: created today in Dubai time.
- [ ] **27.** Upgrade `LeadQueryMixin`: `q` uses `search_q`; add `finder`, `min_deposit`, `max_deposit`, `created_from`, `created_to`, and a whitelisted `sort`.
- [ ] **28.** `mark_burned_as_lost()`: one `update()` that sets status lost and appends `| auto-closed after 3 attempts` to notes.
- [ ] **29.** `LeadFinderView` at `leads/find/<finder>/` returning any finder as JSON (404 JSON for unknown names). Replaces Day 6's `StaleLeadsView`.
- [ ] **30.** Quick-filter chips above the lead table (All, Stale, Follow-up, Burned, High value, Sort by deposit) that keep the current search text.

D. Challenge (31–40)

- [ ] **31.** `get_object_or_404` with a `Q`: active leads only.
- [ ] **32.** Annotate a 2% commission estimate with `ExpressionWrapper`.
- [ ] **33.** Race-safe status change: `claim_new_lead(pk)` returns True only if *this* call moved it from new to contacted.
- [ ] **34.** Case-insensitive unique email (ignoring blanks) with a functional `UniqueConstraint(Lower("email"), condition=...)`.
- [ ] **35.** `explain()` a status filter and spot the Day 5 index in the plan.
- [ ] **36.** List every week that has leads with `datetimes("created_at", "week")`.
- [ ] **37.** Test `search_leads` matches by name, formatted phone and email, and not otherwise.
- [ ] **38.** Test `stale_new_leads` (backdate with `update()`).
- [ ] **39.** Test two `F()` increments give +2.
- [ ] **40.** Test `?sort=password` falls back to the default ordering.

Solutions

A. Drills

1–10.

```
# python manage.py shell
from datetime import timedelta
from django.db.models import F, Q
from django.db.models.functions import Length, Lower
from django.utils import timezone
from leads.models import Lead

Lead.objects.filter(name__exact="Aisha Khan")        # 1 — same as
name="Aisha Khan"
Lead.objects.filter(name__iexact="aisha khan")

Lead.objects.filter(name__contains="khan")           # 2 — SQLite's LIKE
ignores ASCII case,
Lead.objects.filter(name__icontains="khan")          #     so both match;
PostgreSQL won't

Lead.objects.filter(name__startswith="A")            # 3
Lead.objects.filter(name__istartswith="a")
Lead.objects.filter(phone__endswith="67")

Lead.objects.filter(status__in=["new", "contacted"]) # 4
Lead.objects.filter(source__in=Lead.PAID_SOURCES)

Lead.objects.filter(expected_deposit__gt=10_000)     # 5
Lead.objects.filter(expected_deposit__lte=1_000)
Lead.objects.filter(expected_deposit__range=(5_000, 20_000))   # inclusive
both ends

Lead.objects.filter(last_contacted_at__isnull=True)  # 6

now = timezone.localtime()                           # 7
Lead.objects.filter(created_at__year=now.year, created_at__month=now.month)
Lead.objects.filter(created_at__week_day=6)          # 1 = Sunday … 7 =
Saturday → 6 = Friday
today = timezone.localdate()
Lead.objects.filter(created_at__date__range=(today - timedelta(days=6),
today))

Lead.objects.filter(phone__regex=r"^97150")          # 8
```

```
Lead.objects.filter(email="").count()                # 9 — blank strings: the
real answer
Lead.objects.filter(email__isnull=True).count()      # 0 — EmailField has no
null=True
Lead.objects.exclude(email="")

Lead.objects.order_by("status", "-expected_deposit")                    #
10
Lead.objects.order_by("?")[:5]                       # random — slow on big
tables
Lead.objects.order_by(F("last_contacted_at").desc(nulls_last=True))
```

B. Core

11–20.

```
Lead.objects.filter(Q(status="new") | Q(source="referral"))                #
11

Lead.objects.filter(Q(status="contacted") & Q(contact_attempts__gte=2))    #
12
Lead.objects.filter(Q(status="new") & ~Q(source__in=Lead.PAID_SOURCES))

Lead.objects.filter(Q(status="new") | Q(status="contacted"), is_active=True)
# 13: Q first, kwargs after

import operator
# 14
from functools import reduce
sources = ["google", "facebook"]
cond = reduce(operator.or_, (Q(source=s) for s in sources))
Lead.objects.filter(cond)

cond = Q()                       # same thing with a loop; empty Q() matches
everything
for s in sources:
    cond |= Q(source=s)

Lead.objects.filter(status="contacted").update(
# 15
    contact_attempts=F("contact_attempts") + 1,     # UPDATE ... SET x = x +
1 — no race
)

Lead.objects.filter(updated_at__gt=F("created_at") + timedelta(days=1))
# 16
```

```
lead = Lead.objects.first()
# 17
lead.contact_attempts = F("contact_attempts") + 1
lead.save(update_fields=["contact_attempts", "updated_at"])
lead.contact_attempts            # <CombinedExpression ...> — not a number
yet
lead.refresh_from_db()
lead.contact_attempts            # real value

Lead.objects.values(lead_name=F("name"), lead_status=F("status"))[:3]       #
18

for row in Lead.objects.values_list("name", "status", named=True)[:3]:      #
19
    print(row.name, row.status)

Lead.objects.annotate(name_len=Length("name")).filter(name_len__gt=12)      #
20
Lead.objects.order_by(Lower("name"))
```

C. Nexora CRM

The seed has no call data until Day 8. To try 23–24 now, fake some:

```
# update() can't run on a slice, so collect pks first
ids = list(Lead.objects.filter(status="contacted").values_list("pk",
flat=True)[:20])
Lead.objects.filter(pk__in=ids).update(
    contact_attempts=3, last_contacted_at=timezone.now() - timedelta(days=3),
)
```

21–26, 28.

```
# leads/queries.py
import re
from datetime import timedelta

from django.db.models import F, Q, TextField, Value
from django.db.models.functions import Concat
from django.utils import timezone

from .models import Lead

OPEN_STATUSES = [Lead.Status.NEW, Lead.Status.CONTACTED]
```

```
def search_q(term):                                                    # 21
    term = term.strip()
    if not term:
        return Q()
    q = Q(name__icontains=term) | Q(email__icontains=term)
    digits = re.sub(r"\D", "", term)          # "+971 50 999" → "97150999"
    if len(digits) >= 4:
        q |= Q(phone__contains=digits)
    return q


def search_leads(term, qs=None):
    qs = Lead.objects.all() if qs is None else qs
    return qs.filter(search_q(term))


def stale_new_leads(days=7):                                           # 22
    cutoff = timezone.now() - timedelta(days=days)
    return Lead.objects.filter(
        status=Lead.Status.NEW, created_at__lt=cutoff,
last_contacted_at__isnull=True,
    ).order_by("created_at")


def follow_up_due(hours=48, max_attempts=3):                           # 23
    cutoff = timezone.now() - timedelta(hours=hours)
    return Lead.objects.filter(
        status=Lead.Status.CONTACTED,
        last_contacted_at__lt=cutoff,
        contact_attempts__lt=max_attempts,
    ).order_by("last_contacted_at")


def burned_leads(max_attempts=3):                                      # 24
    return Lead.objects.filter(status__in=OPEN_STATUSES,
contact_attempts__gte=max_attempts)


def high_value(threshold=20_000):                                      # 25
    return (Lead.objects
            .filter(expected_deposit__gte=threshold)
            .exclude(status=Lead.Status.LOST)
            .order_by("-expected_deposit"))
```

```
def today_new():                                                    # 26
    return Lead.objects.filter(created_at__date=timezone.localdate())   #
__date uses TIME_ZONE


def mark_burned_as_lost():                                          # 28
    return burned_leads().update(
        status=Lead.Status.LOST,
        notes=Concat(F("notes"), Value(" | auto-closed after 3 attempts"),
output_field=TextField()),
        updated_at=timezone.now(),
    )


FINDERS = {
    "stale": stale_new_leads,
    "follow_up": follow_up_due,
    "burned": burned_leads,
    "high_value": high_value,
    "today": today_new,
}
```

`Concat` needs `output_field` when mixing a TextField with a plain string value (28).

27.

```
# leads/views.py
from decimal import Decimal, InvalidOperation

from django.utils.dateparse import parse_date

from .queries import FINDERS, search_q

SORTABLE = {"name", "created_at", "expected_deposit", "status",
"contact_attempts"}


class LeadQueryMixin:
    default_sort = "-created_at"

    def get_leads(self):
        p = self.request.GET
        qs = Lead.objects.filter(is_active=True)
        if source := p.get("source"):
            qs = qs.filter(source=source)
        if status := p.get("status"):
            qs = qs.filter(status=status)
```

```
        if q := p.get("q", "").strip():
            qs = qs.filter(search_q(q))
        if finder := FINDERS.get(p.get("finder", "")):
            qs = qs.filter(pk__in=finder().values("pk"))          # subquery
        try:
            if min_dep := p.get("min_deposit"):
                qs = qs.filter(expected_deposit__gte=Decimal(min_dep))
            if max_dep := p.get("max_deposit"):
                qs = qs.filter(expected_deposit__lte=Decimal(max_dep))
        except InvalidOperation:
            pass                                                  # ignore
junk numbers
        if created_from := parse_date(p.get("created_from", "")):  # "2026-
10-01"
            qs = qs.filter(created_at__date__gte=created_from)
        if created_to := parse_date(p.get("created_to", "")):
            qs = qs.filter(created_at__date__lte=created_to)
        sort = p.get("sort", self.default_sort)
        if sort.lstrip("-") not in SORTABLE:                      # never
order by raw user input
            sort = self.default_sort
        return qs.order_by(sort)
```

29.

```
# leads/views.py
class LeadFinderView(JsonResponseMixin, View):
    def get(self, request, finder):
        fn = FINDERS.get(finder)
        if fn is None:
            return json_error(f"unknown finder: {finder}", status=404,
available=sorted(FINDERS))
        qs = fn()
        return self.render_to_json({
            "finder": finder,
            "count": qs.count(),
            "results": list(qs.values(*LEAD_FIELDS)[:100]),
        })

# leads/urls.py (replace the stale/ route)
path("find/<slug:finder>/", views.LeadFinderView.as_view(), name="finder"),
```

30.

```
<!-- leads/lead_list.html — above the table -->
{% with q=request.GET.q|urlencode %}
<nav class="chips">
  <a href="?q={{ q }}">All</a>
  <a href="?q={{ q }}&finder=stale">Stale</a>
  <a href="?q={{ q }}&finder=follow_up">Follow-up due</a>
  <a href="?q={{ q }}&finder=burned">3+ attempts</a>
  <a href="?q={{ q }}&finder=high_value">High value</a>
  <a href="?q={{ q }}&sort=-expected_deposit">Sort by deposit</a>
</nav>
{% endwith %}

<!-- inside the search form, so searching keeps the active chip -->
<input type="hidden" name="finder" value="{{ request.GET.finder }}">
```

```
/* static/css/crm.css */
.chips { display: flex; gap: .5rem; margin: 1rem 0; flex-wrap: wrap; }
.chips a { padding: .25rem .75rem; border: 1px solid #cbd5e1; border-radius:
999px; text-decoration: none; color: inherit; }
```

D. Challenge

31–33.

```
from decimal import Decimal
from django.db.models import DecimalField, ExpressionWrapper
from django.shortcuts import get_object_or_404

lead = get_object_or_404(Lead, Q(pk=1) & Q(is_active=True))                #
31

Lead.objects.annotate(
# 32
    commission_estimate=ExpressionWrapper(
        F("expected_deposit") * Decimal("0.02"),
        output_field=DecimalField(max_digits=12, decimal_places=2),
    )
).values("name", "expected_deposit", "commission_estimate")[:5]


# leads/queries.py
# 33
def claim_new_lead(lead_id):
    changed = (Lead.objects
               .filter(pk=lead_id, status=Lead.Status.NEW)    # condition
```

```
checked in the same UPDATE
                .update(status=Lead.Status.CONTACTED,
updated_at=timezone.now()))
    return changed == 1          # False → someone else already moved it
```

34.

```
# leads/models.py — Lead.Meta.constraints
from django.db.models.functions import Lower

models.UniqueConstraint(
    Lower("email"),
    name="lead_unique_email_ci",
    condition=~models.Q(email=""),      # blanks don't count as duplicates
),
```

```
python manage.py makemigrations leads && python manage.py migrate
```

35–36.

```
print(Lead.objects.filter(status="new").order_by("-created_at").explain())
# ... SEARCH crm_leads USING INDEX lead_status_created_idx (status=?)

Lead.objects.datetimes("created_at", "week")      # aware datetimes, one per
week with leads
```

37–40.

```
# leads/tests.py
from datetime import timedelta

from django.db.models import F
from django.utils import timezone

from .queries import search_leads, stale_new_leads


class LeadQueryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.a = Lead.objects.create(name="Search Target",
phone="9715099900001", email="target@nexora.test")
        cls.b = Lead.objects.create(name="Other Person",
phone="9715099900002")

    def test_search(self):                                          # 37
```

```
        self.assertIn(self.a, search_leads("search targ"))
        self.assertIn(self.a, search_leads("+971 50 999 0001"))
        self.assertIn(self.a, search_leads("TARGET@"))
        self.assertNotIn(self.b, search_leads("target"))

    def test_stale_new_leads(self):                                     # 38
        old = timezone.now() - timedelta(days=10)
        Lead.objects.filter(pk=self.a.pk).update(created_at=old)       #
update() skips auto_now_add
        Lead.objects.filter(pk=self.b.pk).update(created_at=old,
last_contacted_at=timezone.now())
        stale = stale_new_leads(days=7)
        self.assertIn(self.a, stale)
        self.assertNotIn(self.b, stale)

    def test_f_increments(self):                                        # 39
        qs = Lead.objects.filter(pk=self.a.pk)
        qs.update(contact_attempts=F("contact_attempts") + 1)
        qs.update(contact_attempts=F("contact_attempts") + 1)
        self.a.refresh_from_db()
        self.assertEqual(self.a.contact_attempts, 2)

    def test_unknown_sort_falls_back(self):                            # 40
        response = self.client.get(reverse("leads:list"), {"sort":
"password"})
        self.assertEqual(response.status_code, 200)
        expected = list(Lead.objects.filter(is_active=True).order_by("-
created_at"))
        self.assertEqual(list(response.context["leads"]), expected)
```

Day 8 — Relationships

Goal: the full Nexora data model — agents as a custom user, leads assigned to agents, call logs, brokers in a referral tree, clients converted from leads, trading accounts and deposits — seeded with realistic data.

Why the custom user lands today: Django's user model must be swapped before any table points at users. Today is the first FK to a user, so you reset the database once (exercise 1) and never again.

A. Drills (1–10) — users, FK, M2M

- [ ] **1.** `accounts` app with `CustomUser(AbstractUser)`; set `AUTH_USER_MODEL`; delete `db.sqlite3`, migrate, create a superuser, re-run the Day 6 seed.
- [ ] **2.** On `CustomUser`: `role` (agent, team_lead, manager, admin) and `team_lead` (self-FK, related name `team_members`).
- [ ] **3.** `CustomUserCreationForm` / `CustomUserChangeForm` and a `UserAdmin` subclass showing role and team lead.
- [ ] **4.** `Lead.assigned_to`: FK to the user model, nullable, `SET_NULL`, related name `leads`.
- [ ] **5.** In the shell: create agent `sara`, assign a lead, read it forward (`lead.assigned_to`) and backward (`sara.leads.all()`).
- [ ] **6.** `Lead.broker`: FK to `Broker`, nullable, `SET_NULL`, related name `leads`.
- [ ] **7.** `Tag` model (slug name, colour) and `Lead.tags` M2M; practise `add`, `remove`, `set`, `clear`.
- [ ] **8.** Reverse M2M (`tag.leads.all()`) and filtering leads by tag name(s) without duplicates.
- [ ] **9.** Filter across FKs: by agent username, agent role, broker tier, and unassigned leads.
- [ ] **10.** Reverse filter: agents who own at least one qualified lead (no duplicates).

B. Core (11–20) — the CRM data model

- [ ] **11.** `CallLog`: lead (CASCADE, `calls`), agent (PROTECT, `calls`), started_at (default now), duration_seconds, outcome (answered/no_answer/busy/voicemail), notes; index on lead + started_at.
- [ ] **12.** In the shell: delete a lead and watch its calls go (CASCADE); delete an agent with calls and catch `ProtectedError`.
- [ ] **13.** `Client.lead`: `OneToOneField` to `Lead` (`SET_NULL`, related name `client`); read both directions; check safely with `hasattr`.

- [ ] **14.** `Client.broker`: FK to `Broker`, related name `clients`.
- [ ] **15.** `TradingAccount`: client (CASCADE, `accounts`), unique 7-digit number, platform (MT4/MT5), currency (USD/AED), leverage, balance, opened_at.
- [ ] **16.** `Deposit`: account (PROTECT, `deposits`), amount (> 0 constraint), method (card/bank/crypto), created_at.
- [ ] **17.** `Broker.parent`: self-FK, related name `sub_brokers`. Build a 3-level tree in the shell.
- [ ] **18.** `Broker.ancestors()` (walk up) and `Broker.descendants()` (recursive walk down).
- [ ] **19.** `limit_choices_to` on `Lead.assigned_to` so only agents appear in forms and admin.
- [ ] **20.** Register `Tag`, `CallLog`, `TradingAccount`, `Deposit` in the admin; `CallLog` list shows lead, agent, outcome, duration, time.

C. Nexora CRM (21–30) — seed and pages

- [ ] **21.** `seed_people()`: 1 manager, 2 team leads, 8 agents (each under a team lead), 3 tier-1 brokers with 2 sub-brokers each, 5 tags.
- [ ] **22.** `assign_leads()`: round-robin every lead to an agent with `bulk_update`.
- [ ] **23.** Attach a broker to ~30% of leads; give leads 0–2 random tags by bulk-creating rows in the M2M through table.
- [ ] **24.** `seed_calls()`: 0–5 calls per assigned lead (weighted outcomes, talk time only when answered); update `contact_attempts` and `last_contacted_at`.
- [ ] **25.** `convert_qualified()`: each qualified lead becomes a `Client` (same broker) with 1–2 trading accounts and 1–4 deposits each. One transaction.
- [ ] **26.** Lead detail shows agent, broker, tags and a call-history table.
- [ ] **27.** Agent page `/team/agents/<pk>/`: the agent, their team lead, leads grouped by status.
- [ ] **28.** Broker page `/brokers/<code>/`: breadcrumb of ancestors, sub-broker links, client count.
- [ ] **29.** Client detail page from the DB: KYC, broker, source lead link, each account with its deposits.
- [ ] **30.** Lead list: `?agent=<id>` and `?tag=<name>` filters and an Agent column.

D. Challenge (31–40)

- [ ] **31.** M2M with extra data: `Campaign` ↔ `Lead` through `CampaignEnrollment` (enrolled_at, outcome, unique pair); add with `through_defaults`.
- [ ] **32.** Symmetrical self M2M: `Client.related_clients` (households). Add A→B and see B→A.
- [ ] **33.** Replace `PROTECT` on `CallLog.agent` with `SET(get_sentinel_agent)` so calls survive under a `deleted-agent` user.
- [ ] **34.** Multi-valued filters: one `filter()` with two `calls__` conditions vs two chained `filter()` calls — compare counts and explain.
- [ ] **35.** `Broker.descendants_sql()` with a `WITH RECURSIVE` raw query (one query for the whole tree).
- [ ] **36.** Tag usage counts by querying the M2M through model directly.
- [ ] **37.** Test: deleting a user leaves their leads unassigned.
- [ ] **38.** Test: deleting an agent moves their calls to the sentinel user.
- [ ] **39.** Test: deleting a trading account that has deposits raises `ProtectedError`.
- [ ] **40.** Test: `descendants()` and `ancestors()` on a 3-level tree.

Solutions

A. Drills

1–2.

```
python manage.py startapp accounts
```

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models


class CustomUser(AbstractUser):
    class Role(models.TextChoices):
        AGENT = "agent", "Agent"
        TEAM_LEAD = "team_lead", "Team lead"
        MANAGER = "manager", "Manager"
        ADMIN = "admin", "Admin"

    role = models.CharField(max_length=20, choices=Role.choices,
default=Role.AGENT)
    team_lead = models.ForeignKey(
```

```
        "self", null=True, blank=True, on_delete=models.SET_NULL,
        related_name="team_members", limit_choices_to={"role": "team_lead"},
    )
    phone_ext = models.CharField(max_length=6, blank=True)

    def __str__(self):
        return self.get_full_name() or self.username
```

```
# django_project/settings.py
INSTALLED_APPS = [..., "accounts.apps.AccountsConfig", ...]
AUTH_USER_MODEL = "accounts.CustomUser"
```

```
# stop runserver first
del db.sqlite3                     # PowerShell: Remove-Item db.sqlite3 ·
macOS: rm db.sqlite3
python manage.py makemigrations accounts
python manage.py migrate           # rebuilds every table; the Day 5 data
migration re-adds 5 leads
python manage.py createsuperuser
python manage.py shell -c "from leads.seed import run; print(run())"
```

The leads/clients/brokers migration files stay: none of them referenced `auth.User`, so only the database had to go.

3.

```
# accounts/forms.py
from django.contrib.auth.forms import UserChangeForm, UserCreationForm
from .models import CustomUser


class CustomUserCreationForm(UserCreationForm):
    class Meta(UserCreationForm.Meta):
        model = CustomUser
        fields = UserCreationForm.Meta.fields + ("email", "role",
"team_lead")


class CustomUserChangeForm(UserChangeForm):
    class Meta(UserChangeForm.Meta):
        model = CustomUser
        fields = "__all__"
```

```
# accounts/admin.py
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin
from .forms import CustomUserChangeForm, CustomUserCreationForm
from .models import CustomUser


@admin.register(CustomUser)
class CustomUserAdmin(UserAdmin):
    add_form = CustomUserCreationForm
    form = CustomUserChangeForm
    model = CustomUser
    list_display = ["username", "email", "role", "team_lead", "is_staff"]
    list_filter = UserAdmin.list_filter + ("role",)
    fieldsets = UserAdmin.fieldsets + (("CRM", {"fields": ("role",
"team_lead", "phone_ext")}),)
    add_fieldsets = UserAdmin.add_fieldsets + (("CRM", {"fields": ("email",
"role", "team_lead")}),)
```

4, 6–7, 19. New `Lead` fields + `Tag`.

```
# leads/models.py
from django.conf import settings


class Tag(models.Model):
# 7 (define above Lead)
    name = models.SlugField(max_length=30, unique=True)
    color = models.CharField(max_length=7, default="#64748b")

    class Meta:
        ordering = ["name"]

    def __str__(self):
        return self.name


class Lead(models.Model):
    ...
    assigned_to = models.ForeignKey(
# 4, 19
        settings.AUTH_USER_MODEL,          # never import CustomUser directly
here
        null=True, blank=True, on_delete=models.SET_NULL,
        related_name="leads", limit_choices_to={"role": "agent"},
    )
```

```
    broker = models.ForeignKey(
# 6
        "brokers.Broker", null=True, blank=True, on_delete=models.SET_NULL,
related_name="leads",
    )
    tags = models.ManyToManyField(Tag, blank=True, related_name="leads")
# 7
```

5, 7–10.

```
# python manage.py shell
from accounts.models import CustomUser
from leads.models import Lead, Tag

sara = CustomUser.objects.create_user("sara", password="nexora123",
role="agent")   # 5
lead = Lead.objects.first()
lead.assigned_to = sara
lead.save(update_fields=["assigned_to", "updated_at"])
lead.assigned_to.username          # forward
sara.leads.all()                   # reverse, via related_name

vip, _ = Tag.objects.get_or_create(name="vip")                                  #
7
callback, _ = Tag.objects.get_or_create(name="callback")
lead.tags.add(vip, callback)       # M2M changes save immediately — no
lead.save()
lead.tags.remove(callback)
lead.tags.set([vip, callback])
lead.tags.clear()
lead.tags.add(vip)

vip.leads.all()
# 8
Lead.objects.filter(tags__name="vip")
Lead.objects.filter(tags__name__in=["vip", "callback"]).distinct()   # a lead
with both tags would repeat

Lead.objects.filter(assigned_to__username="sara")                               #
9
Lead.objects.filter(assigned_to__role=CustomUser.Role.AGENT)
Lead.objects.filter(broker__tier=1)
Lead.objects.filter(assigned_to__isnull=True)

CustomUser.objects.filter(leads__status="qualified").distinct()             #
10
```

B. Core

11.

```
# leads/models.py (below Lead)
from django.utils import timezone


class CallLog(models.Model):
    class Outcome(models.TextChoices):
        ANSWERED = "answered", "Answered"
        NO_ANSWER = "no_answer", "No answer"
        BUSY = "busy", "Busy"
        VOICEMAIL = "voicemail", "Voicemail"

    lead = models.ForeignKey(Lead, on_delete=models.CASCADE,
related_name="calls")
    agent = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.PROTECT, related_name="calls")  # 33 changes this
    started_at = models.DateTimeField(default=timezone.now)     # default,
not auto_now_add → seedable
    duration_seconds = models.PositiveIntegerField(default=0)
    outcome = models.CharField(max_length=20, choices=Outcome.choices)
    notes = models.TextField(blank=True)

    class Meta:
        ordering = ["-started_at"]
        indexes = [models.Index(fields=["lead", "started_at"],
name="call_lead_started_idx")]

    def __str__(self):
        return f"{self.lead.name} · {self.get_outcome_display()} ·
{self.duration_seconds}s"
```

12.

```
from django.db.models import ProtectedError
from leads.models import CallLog

temp = Lead.objects.create(name="Temp", phone="971507770030")
CallLog.objects.create(lead=temp, agent=sara, outcome="answered",
duration_seconds=60)
temp.delete()                       # (2, {'leads.CallLog': 1, 'leads.Lead':
1}) — CASCADE

temp2 = Lead.objects.create(name="Temp Two", phone="971507770031")
CallLog.objects.create(lead=temp2, agent=sara, outcome="busy")
try:
    sara.delete()
except ProtectedError as e:
    print("blocked by:", e.protected_objects)
```

13–16.

```
# clients/models.py
from django.utils import timezone


class Client(models.Model):
    ...
    lead = models.OneToOneField(
# 13
        "leads.Lead", null=True, blank=True, on_delete=models.SET_NULL,
related_name="client",
    )
    broker = models.ForeignKey(
# 14
        "brokers.Broker", null=True, blank=True, on_delete=models.SET_NULL,
related_name="clients",
    )


class TradingAccount(models.Model):
# 15
    class Platform(models.TextChoices):
        MT4 = "mt4", "MetaTrader 4"
        MT5 = "mt5", "MetaTrader 5"

    class Currency(models.TextChoices):
        USD = "USD", "USD"
        AED = "AED", "AED"
```

```
    client = models.ForeignKey(Client, on_delete=models.CASCADE,
related_name="accounts")
    number = models.CharField(max_length=7, unique=True, validators=
[RegexValidator(r"^\d{7}$")])
    platform = models.CharField(max_length=3, choices=Platform.choices,
default=Platform.MT5)
    currency = models.CharField(max_length=3, choices=Currency.choices,
default=Currency.USD)
    leverage = models.PositiveSmallIntegerField(default=100)
    balance = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    opened_at = models.DateTimeField(default=timezone.now)

    def __str__(self):
        return f"{self.number} ({self.get_platform_display()})"


class Deposit(models.Model):
# 16
    class Method(models.TextChoices):
        CARD = "card", "Card"
        BANK = "bank", "Bank transfer"
        CRYPTO = "crypto", "Crypto"

    account = models.ForeignKey(TradingAccount, on_delete=models.PROTECT,
related_name="deposits")
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    method = models.CharField(max_length=10, choices=Method.choices)
    created_at = models.DateTimeField(default=timezone.now)

    class Meta:
        ordering = ["-created_at"]
        constraints = [
            models.CheckConstraint(condition=models.Q(amount__gt=0),
name="deposit_amount_positive"),
        ]
```

Deleting a client cascades to its accounts, then hits `PROTECT` on deposits → clients with money history can't be deleted.

```
# 13 in the shell
client.lead                 # forward
lead.client                 # reverse — raises
Lead.client.RelatedObjectDoesNotExist if none
hasattr(lead, "client")     # safe check (the exception subclasses
AttributeError)
```

17–18.

```
# brokers/models.py — add to Broker
parent = models.ForeignKey(
    "self", null=True, blank=True, on_delete=models.SET_NULL,
related_name="sub_brokers",
)

def ancestors(self):                    # nearest first
    chain, node = [], self.parent
    while node is not None:
        chain.append(node)
        node = node.parent
    return chain

def descendants(self):                  # one query per node — 35 does it in
one
    result = []
    for child in self.sub_brokers.all():
        result.append(child)
        result.extend(child.descendants())
    return result
```

```
# shell
from brokers.models import Broker

top = Broker.objects.create(code="BRF1000", name="Gulf Partners", tier=1)
mid = Broker.objects.create(code="BRF1100", name="Marina Desk", tier=2,
parent=top)
leaf = Broker.objects.create(code="BRF1110", name="JLT Intro", tier=3,
parent=mid)

top.sub_brokers.all()                       # direct children
Broker.objects.filter(parent__parent=top)   # grandchildren
leaf.ancestors()                            # [mid, top]
top.descendants()                           # [mid, leaf]
```

20.

```
# leads/admin.py
from django.contrib import admin
from .models import CallLog, Lead, Tag

admin.site.register(Lead)
admin.site.register(Tag)


@admin.register(CallLog)
class CallLogAdmin(admin.ModelAdmin):
    list_display = ["lead", "agent", "outcome", "duration_seconds",
"started_at"]

# clients/admin.py
from .models import Client, Deposit, TradingAccount

admin.site.register(Client)
admin.site.register(TradingAccount)
admin.site.register(Deposit)
```

C. Nexora CRM

21–25. Add to `leads/seed.py`; run `run_all()` once after the reset.

```
# leads/seed.py (add)
from django.contrib.auth import get_user_model
from django.db import transaction

from brokers.models import Broker
from clients.models import Client, Deposit, TradingAccount
from .models import CallLog, Lead, Tag

User = get_user_model()
TAGS = ["vip", "callback", "arabic-speaker", "high-risk", "whale"]


def _user(username, **fields):
    user, created = User.objects.get_or_create(username=username,
defaults=fields)
    if created:
        user.set_password("nexora123")
        user.save(update_fields=["password"])
    return user
```

```
def seed_people():
# 21
    _user("manager", role="manager", first_name="Rania")
    team_leads = [_user(f"tl{i}", role="team_lead", first_name=name)
                  for i, name in enumerate(["Karim", "Leila"], start=1)]
    agents = [_user(f"agent{i}", role="agent", first_name=f"Agent{i}",
team_lead=team_leads[i % 2])
              for i in range(1, 9)]
    brokers = []
    for i in range(1, 4):
        top, _ = Broker.objects.get_or_create(
            code=f"BRF{i}000", defaults={"name": f"Partner {i}", "tier": 1,
"commission_rate": 30})
        brokers.append(top)
        for j in range(1, 3):
            sub, _ = Broker.objects.get_or_create(
                code=f"BRF{i}{j}00",
                defaults={"name": f"Partner {i}.{j}", "tier": 2, "parent":
top, "commission_rate": 20})
            brokers.append(sub)
    tags = [Tag.objects.get_or_create(name=t)[0] for t in TAGS]
    return agents, brokers, tags


def assign_leads(agents):
# 22
    leads = list(Lead.objects.order_by("pk"))
    for i, lead in enumerate(leads):
        lead.assigned_to = agents[i % len(agents)]
    Lead.objects.bulk_update(leads, ["assigned_to"], batch_size=200)


def attach_brokers_and_tags(brokers, tags, seed=7):                            #
23
    rng = random.Random(seed)
    leads = list(Lead.objects.all())
    for lead in leads:
        lead.broker = rng.choice(brokers) if rng.random() < 0.3 else None
    Lead.objects.bulk_update(leads, ["broker"], batch_size=200)

    Through = Lead.tags.through                          # the auto-created
join table
    rows = [Through(lead_id=lead.pk, tag_id=tag.pk)
            for lead in leads
            for tag in rng.sample(tags, k=rng.randint(0, 2))]
    Through.objects.bulk_create(rows, ignore_conflicts=True)
```

```
def seed_calls(seed=11):
# 24
    rng = random.Random(seed)
    now = timezone.now()
    calls, touched = [], []
    for lead in Lead.objects.exclude(assigned_to=None):
        n = rng.randint(0, 5)
        if n == 0:
            continue
        times = sorted(
            min(lead.created_at + timedelta(hours=rng.randint(1, 24 * 20)),
now) for _ in range(n)
        )
        for t in times:
            outcome = rng.choices(CallLog.Outcome.values, weights=[45, 35,
10, 10])[0]
            calls.append(CallLog(
                lead=lead, agent_id=lead.assigned_to_id, started_at=t,
outcome=outcome,
                duration_seconds=rng.randint(30, 900) if outcome ==
"answered" else 0,
            ))
        lead.contact_attempts = n
        lead.last_contacted_at = times[-1]
        touched.append(lead)
    CallLog.objects.bulk_create(calls, batch_size=500)
    Lead.objects.bulk_update(touched, ["contact_attempts",
"last_contacted_at"], batch_size=200)


@transaction.atomic
# 25
def convert_qualified(seed=13):
    rng = random.Random(seed)
    next_number = 2_000_000 + TradingAccount.objects.count()
    for lead in Lead.objects.filter(status=Lead.Status.QUALIFIED,
client__isnull=True):
        client = Client.objects.create(
            lead=lead, broker=lead.broker, name=lead.name,
            email=lead.email or f"client{lead.pk}@example.com",
            account_number=f"{1_000_000 + lead.pk:07d}",
        )
        for _ in range(rng.randint(1, 2)):
            next_number += 1
```

```
            account = TradingAccount.objects.create(client=client,
number=str(next_number))
            for _ in range(rng.randint(1, 4)):
                Deposit.objects.create(
                    account=account,
                    amount=Decimal(rng.randrange(250, 20_000, 250)),
                    method=rng.choice(Deposit.Method.values),
                    created_at=timezone.now() - timedelta(days=rng.randint(0,
60)),
                )


def run_all():
    if not Lead.objects.exists():
        run(200)
    agents, brokers, tags = seed_people()
    assign_leads(agents)
    attach_brokers_and_tags(brokers, tags)
    seed_calls()          # not idempotent — run once per fresh database
    convert_qualified()
```

```
python manage.py makemigrations && python manage.py migrate
python manage.py shell -c "from leads.seed import run_all; run_all()"
```

26.

```
<!-- leads/lead_detail.html — add inside <dl> -->
<dt>Agent</dt><dd>{{ lead.assigned_to|default:"Unassigned" }}</dd>
<dt>Broker</dt><dd>{% if lead.broker %}<a href="{% url 'brokers:detail'
lead.broker.code %}">{{ lead.broker }}</a>{% else %}—{% endif %}</dd>
<dt>Tags</dt>
<dd>{% for tag in lead.tags.all %}<span class="badge" style="background:{{
tag.color }};color:#fff">{{ tag }}</span> {% empty %}—{% endfor %}</dd>

<!-- below </dl> -->
<h2>Calls ({{ lead.calls.count }})</h2>
<table class="table">
  <tr><th>When</th><th>Agent</th><th>Outcome</th><th>Talk time</th></tr>
  {% for call in lead.calls.all %}
    <tr>
      <td>{{ call.started_at|date:"d M H:i" }}</td>
      <td>{{ call.agent }}</td>
      <td>{{ call.get_outcome_display }}</td>
      <td>{{ call.duration_seconds }}s</td>
    </tr>
```

```
  {% empty %}
    <tr><td colspan="4">No calls yet.</td></tr>
  {% endfor %}
</table>
```

27.

```
# accounts/views.py
from django.contrib.auth import get_user_model
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView

from core.mixins import PageTitleMixin
from leads.models import Lead


class AgentDetailView(PageTitleMixin, TemplateView):
    template_name = "accounts/agent_detail.html"
    page_title = "Agent"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        agent = get_object_or_404(get_user_model(), pk=self.kwargs["pk"],
role="agent")
        context["agent"] = agent
        context["by_status"] = {label: agent.leads.filter(status=value)
                                for value, label in Lead.Status.choices}
        return context

# accounts/urls.py
from django.urls import path
from . import views

app_name = "accounts"
urlpatterns = [
    path("agents/<int:pk>/", views.AgentDetailView.as_view(),
name="agent_detail"),
]

# django_project/urls.py
path("team/", include("accounts.urls")),
```

```
<!-- accounts/templates/accounts/agent_detail.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ agent }}</h1>
<p>Team lead: {{ agent.team_lead|default:"—" }} · {{ agent.leads.count }}
leads</p>
{% for label, leads in by_status.items %}
  <h2>{{ label }} ({{ leads|length }})</h2>
  <ul>{% for lead in leads|slice:":10" %}<li><a href="{{
lead.get_absolute_url }}">{{ lead.name }}</a></li>{% endfor %}</ul>
{% endfor %}
{% endblock content %}
```

28.

```
# brokers/views.py
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView
from core.mixins import PageTitleMixin
from .models import Broker


class BrokerDetailView(PageTitleMixin, TemplateView):
    template_name = "brokers/broker_detail.html"
    page_title = "Broker"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        broker = get_object_or_404(Broker, code=self.kwargs["code"])
        context.update(
            broker=broker,
            ancestors=broker.ancestors()[::-1],       # top → parent
            children=broker.sub_brokers.all(),
            client_count=broker.clients.count(),
        )
        return context

# brokers/urls.py
app_name = "brokers"
urlpatterns = [path("<str:code>/", views.BrokerDetailView.as_view(),
name="detail")]

# django_project/urls.py
path("brokers/", include("brokers.urls")),
```

```
<!-- brokers/templates/brokers/broker_detail.html -->
{% extends "base.html" %}
{% block content %}
<p>{% for b in ancestors %}<a href="{% url 'brokers:detail' b.code %}">{{
b.code }}</a> › {% endfor %}{{ broker.code }}</p>
<h1>{{ broker.name }} <small>Tier {{ broker.tier }} · {{
broker.commission_rate }}%</small></h1>
<p>{{ client_count }} referred client{{ client_count|pluralize }}</p>
<h2>Sub-brokers</h2>
<ul>
  {% for child in children %}<li><a href="{% url 'brokers:detail' child.code
%}">{{ child }}</a></li>
  {% empty %}<li>None</li>{% endfor %}
</ul>
{% endblock content %}
```

29.

```
# clients/views.py — replaces the JSON ClientDetailView (URL name
clients:detail unchanged)
from django.shortcuts import get_object_or_404
from django.views.generic import TemplateView
from core.mixins import PageTitleMixin
from .models import Client


class ClientDetailView(PageTitleMixin, TemplateView):
    template_name = "clients/client_detail.html"
    page_title = "Client"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["client"] = get_object_or_404(Client, ref=self.kwargs["ref"])
        return context
```

```
<!-- clients/templates/clients/client_detail.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ client.name }} <small>{{ client.account_number }}</small></h1>
<p>KYC: {{ client.get_kyc_status_display }} · Broker: {{
client.broker|default:"Direct" }} ·
   Lead: {% if client.lead %}<a href="{{ client.lead.get_absolute_url }}">{{
client.lead.name }}</a>{% else %}—{% endif %}</p>
{% for account in client.accounts.all %}
  <h2>{{ account }} · {{ account.currency }} · 1:{{ account.leverage }}</h2>
  <table class="table">
    {% for dep in account.deposits.all %}
      <tr><td>{{ dep.created_at|date:"d M Y" }}</td><td>{{
dep.get_method_display }}</td><td>{{ dep.amount|floatformat:2 }}</td></tr>
    {% empty %}
      <tr><td colspan="3">No deposits.</td></tr>
    {% endfor %}
  </table>
{% endfor %}
{% endblock content %}
```

30.

```
# LeadQueryMixin.get_leads — add
if (agent := p.get("agent", "")).isdigit():
    qs = qs.filter(assigned_to_id=agent)
if tag := p.get("tag"):
    qs = qs.filter(tags__name=tag)          # one tag → no duplicate rows
```

```
<!-- lead_list.html: add <th>Agent</th> and -->
<td>{{ lead.assigned_to|default:"—" }}</td>
```

D. Challenge

31–33.

```
# leads/models.py
class Campaign(models.Model):
# 31
    name = models.CharField(max_length=100, unique=True)
    starts_on = models.DateField()
    leads = models.ManyToManyField(Lead, through="CampaignEnrollment",
related_name="campaigns")

    def __str__(self):
```

```
        return self.name


class CampaignEnrollment(models.Model):
    class Outcome(models.TextChoices):
        PENDING = "pending"
        INTERESTED = "interested"
        DECLINED = "declined"

    campaign = models.ForeignKey(Campaign, on_delete=models.CASCADE)
    lead = models.ForeignKey(Lead, on_delete=models.CASCADE)
    enrolled_at = models.DateTimeField(default=timezone.now)
    outcome = models.CharField(max_length=20, choices=Outcome.choices,
default=Outcome.PENDING)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["campaign", "lead"],
name="unique_campaign_lead")]


def get_sentinel_agent():
# 33
    from django.contrib.auth import get_user_model
    return get_user_model().objects.get_or_create(
        username="deleted-agent", defaults={"role": "agent", "is_active":
False},
    )[0]

# CallLog.agent becomes:
agent = models.ForeignKey(
    settings.AUTH_USER_MODEL, on_delete=models.SET(get_sentinel_agent),
related_name="calls",
)
```

```
# shell — 31
from datetime import date
from leads.models import Campaign, CampaignEnrollment

camp = Campaign.objects.create(name="Gold Q4", starts_on=date.today())
camp.leads.add(Lead.objects.first(), through_defaults={"outcome":
"interested"})
CampaignEnrollment.objects.filter(campaign=camp,
outcome="interested").count()
Lead.objects.first().campaigns.all()
```

```
# clients/models.py — 32
related_clients = models.ManyToManyField("self", blank=True,
symmetrical=True)

# shell
a, b = Client.objects.all()[:2]
a.related_clients.add(b)
b.related_clients.all()          # contains a — symmetrical adds both
directions
```

34.

```
from datetime import timedelta
week = timezone.now() - timedelta(days=7)

# ONE call that is both answered AND in the last 7 days
a = Lead.objects.filter(calls__outcome="answered",
calls__started_at__gte=week).distinct().count()

# SOME answered call (any time) AND SOME call (maybe another one) in the last
7 days
b =
Lead.objects.filter(calls__outcome="answered").filter(calls__started_at__gte=
week).distinct().count()

a <= b   # True — chained filters on a multi-valued relation each get their
own join
```

35–36.

```
# brokers/models.py — 35
def descendants_sql(self):
    return list(Broker.objects.raw("""
        WITH RECURSIVE tree(id) AS (
            SELECT id FROM brokers_broker WHERE parent_id = %s
            UNION ALL
            SELECT b.id FROM brokers_broker b JOIN tree t ON b.parent_id =
t.id
        )
        SELECT * FROM brokers_broker WHERE id IN (SELECT id FROM tree)
    """, [self.pk]))
```

```
# shell — 36
from django.db.models import Count
Through = Lead.tags.through
Through.objects.values("tag__name").annotate(n=Count("id")).order_by("-n")
```

37–40.

```
# leads/tests.py
from django.contrib.auth import get_user_model
from django.db.models import ProtectedError

from brokers.models import Broker
from clients.models import Client, Deposit, TradingAccount
from .models import CallLog


class RelationshipTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user("t_agent",
password="x", role="agent")
        cls.lead = Lead.objects.create(name="Rel Test", phone="971509990100",
assigned_to=cls.agent)

    def test_deleting_user_unassigns_leads(self):                    # 37
        self.agent.delete()
        self.lead.refresh_from_db()
        self.assertIsNone(self.lead.assigned_to)

    def test_deleted_agent_calls_move_to_sentinel(self):             # 38
        CallLog.objects.create(lead=self.lead, agent=self.agent,
outcome="answered")
        self.agent.delete()
        call = CallLog.objects.get(lead=self.lead)
        self.assertEqual(call.agent.username, "deleted-agent")

    def test_account_with_deposits_is_protected(self):               # 39
        client = Client.objects.create(name="C", email="c@x.test",
account_number="7000001")
        account = TradingAccount.objects.create(client=client,
number="7000001")
        Deposit.objects.create(account=account, amount=100, method="card")
        with self.assertRaises(ProtectedError):
            account.delete()

    def test_broker_tree(self):                                      # 40
```

```
        top = Broker.objects.create(code="BRF9000", name="Top", tier=1)
        mid = Broker.objects.create(code="BRF9100", name="Mid", tier=2,
parent=top)
        leaf = Broker.objects.create(code="BRF9110", name="Leaf", tier=3,
parent=mid)
        self.assertEqual(set(top.descendants()), {mid, leaf})
        self.assertEqual(leaf.ancestors(), [mid, top])
```

Day 9 — Aggregation and annotation

Goal: push every number the CRM reports into SQL — one query per report — and know the two traps (join fan-out and NULL sums). Ends with a `/reports/` page: agent leaderboard, broker deposits, source performance, SOP breaches.

Needs the Day 8 seed (`run_all()`). Do A and B in the shell.

A. Drills (1–10) — aggregate vs annotate

- [ ] **1.** One `aggregate()`: lead count, sum, average, min and max expected deposit.
- [ ] **2.** `aggregate()` with `filter=`: total, qualified and lost counts in one query.
- [ ] **3.** Leads per status (`values().annotate()` = GROUP BY), biggest first.
- [ ] **4.** Per source: count and average expected deposit.
- [ ] **5.** Annotate each agent with their lead count.
- [ ] **6.** Agents with more than 20 leads (filter on an annotation = HAVING).
- [ ] **7.** "New leads per agent" two ways: `filter()` before `annotate()` vs `Count(..., filter=Q(...))`. Which drops agents with zero?
- [ ] **8.** Leads per day with `TruncDate`.
- [ ] **9.** Calls per hour of day with `ExtractHour`.
- [ ] **10.** `Sum` of lost leads' deposits returning 0 instead of `None` (`default=`).

B. Core (11–20) — expressions, subqueries, windows

- [ ] **11.** Agent conversion rate (qualified ÷ total × 100) with `ExpressionWrapper`, best first.
- [ ] **12.** The fan-out trap: `Count("leads")` + `Count("calls")` on agents gives wrong numbers. Fix with `distinct=True`.
- [ ] **13.** `Case` / `When`: label each lead hot / warm / cold.
- [ ] **14.** Deposits by method in one row: `Sum(..., filter=)` and the `Case` / `When` version.
- [ ] **15.** One `update()` with `Case` / `When`: commission 30 / 20 / 10 by broker tier.
- [ ] **16.** `Subquery` + `OuterRef`: each lead's latest call outcome and time.
- [ ] **17.** `Exists`: open leads with no call in the last 7 days.
- [ ] **18.** Total deposits per client: `Subquery` sum vs direct `Sum("accounts__deposits__amount")`.
- [ ] **19.** `Window` + `Rank`: rank agents by qualified leads; keep the top 3.
- [ ] **20.** Running deposit total per trading account with `Window(Sum(...), partition_by=...)`.

C. Nexora CRM (21–30) — `reports` app

- [ ] **21.** Rewrite the dashboard's status counts as a single `aggregate()` query.
- [ ] **22.** `agent_leaderboard()`: leads, qualified, calls, answered calls, average talk time, conversion % — one query, correct counts.
- [ ] **23.** `broker_report()`: clients, leads, sub-brokers, total deposits (deposits via `Subquery` so the joins can't inflate them).
- [ ] **24.** `source_performance()`: leads, qualified, conversion %, average deposit per source.
- [ ] **25.** `daily_new_leads(days=30)`: date + count rows for a chart.
- [ ] **26.** `calls_by_hour()`: calls, answered, answer rate per hour.
- [ ] **27.** Lead list: annotate `call_count` and `last_call_at`; make both sortable with NULLs last.
- [ ] **28.** `sop_first_call_breaches(hours=24)`: leads not called within 24 h of creation (`Exists` + `OuterRef` to an annotated deadline).
- [ ] **29.** `/reports/` page (`ReportsView`) with tables for 22–24 and the first 20 SOP breaches; wire the sidebar link.
- [ ] **30.** JSON endpoints: `/reports/api/daily-leads/` and `/reports/api/calls-by-hour/`.

D. Challenge (31–40)

- [ ] **31.** Team rollup: leads and qualified per team lead (two-hop `values()`).
- [ ] **32.** Broker tree total: deposits for a broker plus all its descendants.
- [ ] **33.** `Cast` + `Round`: call duration in minutes to 1 decimal.
- [ ] **34.** Cohort table: leads per created-month × status, pivoted into a dict.
- [ ] **35.** `Lag` window: time between consecutive calls for each lead.
- [ ] **36.** Cross-check `agent_leaderboard()` against hand-written SQL with correlated subqueries.
- [ ] **37.** Test: the one-query dashboard counts equal separate `count()` calls.
- [ ] **38.** Test: the fan-out trap (2 leads × 3 calls → 6 and 6) and the `distinct=True` fix (2 and 3).
- [ ] **39.** Test: an agent with 1 qualified out of 2 leads shows 50.0% conversion.
- [ ] **40.** Test: `sop_first_call_breaches` flags a lead first called at 30 h but not one called at 2 h.

Solutions

`aggregate()` returns one dict for the whole queryset. `annotate()` adds a value per row; after `values()` it becomes GROUP BY.

A. Drills

1–10.

```
# python manage.py shell
from datetime import timedelta
from decimal import Decimal

from django.db.models import Avg, Count, F, Max, Min, Q, Sum
from django.db.models.functions import ExtractHour, TruncDate
from django.utils import timezone

from accounts.models import CustomUser
from leads.models import CallLog, Lead

Lead.objects.aggregate(                                                 # 1
    total=Count("id"),
    deposit_sum=Sum("expected_deposit"),
    deposit_avg=Avg("expected_deposit"),
    smallest=Min("expected_deposit"),
    largest=Max("expected_deposit"),
)

Lead.objects.aggregate(                                                 # 2
    total=Count("id"),
    qualified=Count("id", filter=Q(status="qualified")),
    lost=Count("id", filter=Q(status="lost")),
)

Lead.objects.values("status").annotate(n=Count("id")).order_by("-n")    # 3

(Lead.objects.values("source")                                          # 4
    .annotate(n=Count("id"), avg_deposit=Avg("expected_deposit"))
    .order_by("source"))

agents =
CustomUser.objects.filter(role="agent").annotate(lead_count=Count("leads"))
# 5
for a in agents:
    print(a.username, a.lead_count)
```

```
CustomUser.objects.annotate(lead_count=Count("leads")).filter(lead_count__gt=
20)        # 6 → HAVING

# 7: filter() first drops agents with zero new leads
CustomUser.objects.filter(leads__status="new").annotate(n=Count("leads"))
# filter= inside Count keeps every agent, 0 where none
CustomUser.objects.filter(role="agent").annotate(n=Count("leads",
filter=Q(leads__status="new")))

(Lead.objects.annotate(day=TruncDate("created_at"))                        # 8
— Dubai dates
    .values("day").annotate(n=Count("id")).order_by("day"))

(CallLog.objects.annotate(hour=ExtractHour("started_at"))                  # 9
    .values("hour").annotate(n=Count("id")).order_by("hour"))

Lead.objects.filter(status="lost").aggregate(total=Sum("expected_deposit",
default=0))   # 10
```

B. Core

11–15.

```
from django.db.models import Case, CharField, DecimalField,
ExpressionWrapper, FloatField, Value, When
from brokers.models import Broker
from clients.models import Deposit

board = (CustomUser.objects.filter(role="agent")                           #
11
    .annotate(
        total=Count("leads"),
        qualified=Count("leads", filter=Q(leads__status="qualified")),
    )
    .filter(total__gt=0)                                    # avoid divide-by-
zero
    .annotate(rate=ExpressionWrapper(F("qualified") * 100.0 / F("total"),
output_field=FloatField()))
    .order_by("-rate"))

# 12: each agent's leads JOIN each agent's calls → leads × calls rows → both
counts inflated
bad =
CustomUser.objects.filter(role="agent").annotate(leads_n=Count("leads"),
calls_n=Count("calls"))
good = CustomUser.objects.filter(role="agent").annotate(
```

```
    leads_n=Count("leads", distinct=True),
    calls_n=Count("calls", distinct=True),
)

Lead.objects.annotate(priority=Case(                                    #
13
    When(status="new", source__in=Lead.PAID_SOURCES, then=Value("hot")),
    When(status="contacted", then=Value("warm")),
    default=Value("cold"),
    output_field=CharField(),
)).values("name", "priority")[:10]

Deposit.objects.aggregate(                                              #
14
    card=Sum("amount", filter=Q(method="card"), default=0),
    bank=Sum("amount", filter=Q(method="bank"), default=0),
    crypto=Sum("amount", filter=Q(method="crypto"), default=0),
)
Deposit.objects.aggregate(        # Case/When form — what filter= becomes on
DBs without FILTER
    card=Sum(Case(When(method="card", then="amount"), default=Value(0),
output_field=DecimalField())),
)

Broker.objects.update(commission_rate=Case(                             #
15 — one UPDATE
    When(tier=1, then=Value(Decimal("30.00"))),
    When(tier=2, then=Value(Decimal("20.00"))),
    default=Value(Decimal("10.00")),
))
```

16–18.

```
from django.db.models import Exists, OuterRef, Subquery
from django.db.models.functions import Coalesce
from clients.models import Client

latest_call = CallLog.objects.filter(lead=OuterRef("pk")).order_by("-
started_at")   # 16
Lead.objects.annotate(
    last_outcome=Subquery(latest_call.values("outcome")[:1]),
    last_call_at=Subquery(latest_call.values("started_at")[:1]),
).values("name", "last_outcome", "last_call_at")[:10]

cutoff = timezone.now() - timedelta(days=7)
# 17
```

```
recent = CallLog.objects.filter(lead=OuterRef("pk"), started_at__gte=cutoff)
Lead.objects.filter(status__in=["new", "contacted"]).filter(~Exists(recent))

deposits = (Deposit.objects
# 18
    .filter(account__client=OuterRef("pk"))
    .order_by()
    .values("account__client")          # group by client inside the subquery
    .annotate(total=Sum("amount"))
    .values("total"))
Client.objects.annotate(total_deposits=Coalesce(
    Subquery(deposits), Value(Decimal("0")), output_field=DecimalField(),
))
Client.objects.annotate(total_deposits=Sum("accounts__deposits__amount",
default=0))   # same numbers here;
# the subquery stays correct even when you add other multi-valued joins to
the same query
```

19–20.

```
from django.db.models import Window
from django.db.models.functions import Rank

ranked = (CustomUser.objects.filter(role="agent")                          #
19
    .annotate(qualified=Count("leads", filter=Q(leads__status="qualified")))
    .annotate(rank=Window(expression=Rank(), order_by=F("qualified").desc()))
    .order_by("rank"))
ranked.filter(rank__lte=3)          # filtering on a window works from Django
4.2

running = (Deposit.objects                                                 #
20
    .annotate(running_total=Window(
        Sum("amount"), partition_by=F("account"),
order_by=F("created_at").asc(),
    ))
    .order_by("account", "created_at")
    .values("account__number", "created_at", "amount", "running_total")[:15])
```

C. Nexora CRM

21.

```
# core/views.py — DashboardView.get_context_data
stats = active.aggregate(
    total=Count("id"),
    **{value: Count("id", filter=Q(status=value)) for value in
Lead.Status.values},
)
context.update(
    total=stats["total"],
    counts={label: stats[value] for value, label in Lead.Status.choices},   #
1 query instead of 5
    ...
)
```

22–26, 28.

```
python manage.py startapp reports    # add "reports.apps.ReportsConfig" to
INSTALLED_APPS
```

```
# reports/queries.py
from datetime import timedelta
from decimal import Decimal

from django.contrib.auth import get_user_model
from django.db.models import (
    Avg, Case, Count, DateTimeField, DecimalField, Exists, ExpressionWrapper,
    F, FloatField, OuterRef, Q, Subquery, Sum, Value, When,
)
from django.db.models.functions import Coalesce, ExtractHour, TruncDate
from django.utils import timezone

from brokers.models import Broker
from clients.models import Deposit
from leads.models import CallLog, Lead


def pct(part, whole):
    """part / whole × 100 as a float expression (whole must be > 0)."""
    return ExpressionWrapper(F(part) * 100.0 / F(whole),
output_field=FloatField())


def agent_leaderboard():                                                #
22
    return (get_user_model().objects
        .filter(role="agent")
```

```
        .annotate(
            leads_n=Count("leads", distinct=True),
            qualified_n=Count("leads", filter=Q(leads__status="qualified"),
distinct=True),
            calls_n=Count("calls", distinct=True),
            answered_n=Count("calls", filter=Q(calls__outcome="answered"),
distinct=True),
            # Avg survives the fan-out (every call repeats equally); a Sum
here would not
            avg_talk=Avg("calls__duration_seconds",
filter=Q(calls__outcome="answered")),
        )
        .annotate(conversion=Case(
            When(leads_n=0, then=Value(0.0)),
            default=pct("qualified_n", "leads_n"),
            output_field=FloatField(),
        ))
        .order_by("-conversion", "-qualified_n"))


def broker_report():                                                    #
23
    deposits = (Deposit.objects
        .filter(account__client__broker=OuterRef("pk"))
        .order_by()
        .values("account__client__broker")
        .annotate(total=Sum("amount"))
        .values("total"))
    return (Broker.objects
        .annotate(
            clients_n=Count("clients", distinct=True),
            leads_n=Count("leads", distinct=True),
            sub_brokers_n=Count("sub_brokers", distinct=True),
            deposits_total=Coalesce(
                Subquery(deposits), Value(Decimal("0")),
                output_field=DecimalField(max_digits=14, decimal_places=2),
            ),
        )
        .order_by("-deposits_total"))


def source_performance():                                               #
24
    return (Lead.objects
        .values("source")
        .annotate(
```

```
            leads_n=Count("id"),
            qualified_n=Count("id", filter=Q(status="qualified")),
            avg_deposit=Avg("expected_deposit"),
        )
        .annotate(conversion=pct("qualified_n", "leads_n"))
        .order_by("-conversion"))


def daily_new_leads(days=30):                                                  #
25
    since = timezone.now() - timedelta(days=days)
    return list(Lead.objects
        .filter(created_at__gte=since)
        .annotate(day=TruncDate("created_at"))
        .values("day")
        .annotate(n=Count("id"))
        .order_by("day"))


def calls_by_hour():                                                           #
26
    return list(CallLog.objects
        .annotate(hour=ExtractHour("started_at"))
        .values("hour")
        .annotate(calls=Count("id"), answered=Count("id",
filter=Q(outcome="answered")))
        .annotate(answer_rate=pct("answered", "calls"))
        .order_by("hour"))


def sop_first_call_breaches(hours=24):                                         #
28
    window = timedelta(hours=hours)
    called_in_time = CallLog.objects.filter(
        lead=OuterRef("pk"), started_at__lte=OuterRef("deadline"),
    )
    return (Lead.objects
        .filter(created_at__lte=timezone.now() - window)          # deadline
already passed
        .annotate(deadline=ExpressionWrapper(F("created_at") + window,
output_field=DateTimeField()))
        .annotate(called_in_time=Exists(called_in_time))
        .filter(called_in_time=False)
        .select_related("assigned_to")
        .order_by("created_at"))
```

27.

```
# leads/views.py — LeadQueryMixin
from django.db.models import Count, F, Max

SORTABLE = {"name", "created_at", "expected_deposit", "status",
"contact_attempts",
            "call_count", "last_call_at"}

    # in get_leads(), replace the sorting block at the end:
        qs = qs.annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at"))
        sort = p.get("sort", self.default_sort)
        if sort.lstrip("-") not in SORTABLE:
            sort = self.default_sort
        field = sort.lstrip("-")
        order = F(field).desc(nulls_last=True) if sort.startswith("-") else
F(field).asc(nulls_last=True)
        return qs.order_by(order, "-pk")       # pk = stable tie-breaker
```

```
<!-- lead_list.html: two more columns -->
<th><a href="?sort=-call_count">Calls</a></th><th><a href="?sort=-
last_call_at">Last call</a></th>
...
<td>{{ lead.call_count }}</td><td>{{ lead.last_call_at|date:"d M
H:i"|default:"—" }}</td>
```

29–30.

```
# reports/views.py
from django.views import View
from django.views.generic import TemplateView

from core.mixins import JsonResponseMixin, PageTitleMixin
from . import queries


class ReportsView(PageTitleMixin, TemplateView):                              #
29
    template_name = "reports/index.html"
    page_title = "Reports"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.update(
            leaderboard=queries.agent_leaderboard(),
```

```
            brokers=queries.broker_report()[:10],
            sources=queries.source_performance(),
            breaches=queries.sop_first_call_breaches()[:20],
        )
        return context


class DailyLeadsAPI(JsonResponseMixin, View):                                  #
30
    def get(self, request):
        days = request.GET.get("days", "30")
        return self.render_to_json(queries.daily_new_leads(int(days) if
days.isdigit() else 30))


class CallsByHourAPI(JsonResponseMixin, View):
    def get(self, request):
        return self.render_to_json(queries.calls_by_hour())
```

```
# reports/urls.py
from django.urls import path
from . import views

app_name = "reports"
urlpatterns = [
    path("", views.ReportsView.as_view(), name="index"),
    path("api/daily-leads/", views.DailyLeadsAPI.as_view(),
name="daily_leads"),
    path("api/calls-by-hour/", views.CallsByHourAPI.as_view(),
name="calls_by_hour"),
]

# django_project/urls.py
path("reports/", include("reports.urls")),
```

```
<!-- templates/base.html — sidebar link -->
<a href="{% url 'reports:index' %}" class="{% if ns == 'reports' %}active{%
endif %}">Reports</a>
```

```
<!-- reports/templates/reports/index.html -->
{% extends "base.html" %}
{% block title %}Reports · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Reports</h1>
```

```
<h2>Agent leaderboard</h2>
<table class="table">
  <tr><th>Agent</th><th>Leads</th><th>Qualified</th><th>Conv.</th>
<th>Calls</th><th>Answered</th><th>Avg talk</th></tr>
  {% for a in leaderboard %}
  <tr>
    <td><a href="{% url 'accounts:agent_detail' a.pk %}">{{ a }}</a></td>
    <td>{{ a.leads_n }}</td><td>{{ a.qualified_n }}</td><td>{{
a.conversion|floatformat:1 }}%</td>
    <td>{{ a.calls_n }}</td><td>{{ a.answered_n }}</td>
    <td>{% if a.avg_talk %}{{ a.avg_talk|floatformat:0 }}s{% else %}—{% endif
%}</td>
  </tr>
  {% endfor %}
</table>

<h2>Top brokers by deposits</h2>
<table class="table">
  <tr><th>Broker</th><th>Clients</th><th>Leads</th><th>Sub-brokers</th>
<th>Deposits</th></tr>
  {% for b in brokers %}
  <tr>
    <td><a href="{% url 'brokers:detail' b.code %}">{{ b }}</a></td>
    <td>{{ b.clients_n }}</td><td>{{ b.leads_n }}</td><td>{{ b.sub_brokers_n
}}</td>
    <td>{{ b.deposits_total|floatformat:2 }}</td>
  </tr>
  {% endfor %}
</table>

<h2>Sources</h2>
<table class="table">
  <tr><th>Source</th><th>Leads</th><th>Qualified</th><th>Conv.</th><th>Avg
deposit</th></tr>
  {% for s in sources %}
  <tr><td>{{ s.source }}</td><td>{{ s.leads_n }}</td><td>{{ s.qualified_n }}
</td>
      <td>{{ s.conversion|floatformat:1 }}%</td><td>{{
s.avg_deposit|floatformat:0 }}</td></tr>
  {% endfor %}
</table>

<h2>SOP: no call within 24 h</h2>
<table class="table">
  <tr><th>Lead</th><th>Created</th><th>Agent</th></tr>
  {% for lead in breaches %}
```

```
  <tr><td><a href="{{ lead.get_absolute_url }}">{{ lead.name }}</a></td>
      <td>{{ lead.created_at|date:"d M H:i" }}</td><td>{{
lead.assigned_to|default:"—" }}</td></tr>
  {% empty %}
  <tr><td colspan="3">No breaches.</td></tr>
  {% endfor %}
</table>
{% endblock content %}
```

D. Challenge

31–35.

```
from collections import defaultdict
from django.db.models.functions import Cast, Lag, Round, TruncMonth

(Lead.objects                                                          #
31
    .values(team=F("assigned_to__team_lead__username"))
    .annotate(leads_n=Count("id"), qualified_n=Count("id",
filter=Q(status="qualified")))
    .order_by("team"))


def tree_deposits(broker):                                             #
32
    ids = [broker.pk, *(b.pk for b in broker.descendants())]
    return (Deposit.objects
            .filter(account__client__broker_id__in=ids)
            .aggregate(total=Sum("amount", default=0))["total"])

tree_deposits(Broker.objects.get(code="BRF1000"))

CallLog.objects.annotate(                                              #
33
    minutes=Round(Cast("duration_seconds", FloatField()) / 60, 1),
).values("duration_seconds", "minutes")[:5]

rows = (Lead.objects                                                   #
34
    .annotate(month=TruncMonth("created_at"))
    .values("month", "status")
    .annotate(n=Count("id"))
    .order_by("month", "status"))
pivot = defaultdict(dict)
for r in rows:
```

```
    pivot[r["month"].strftime("%Y-%m")][r["status"]] = r["n"]
dict(pivot)          # {'2026-09': {'contacted': 28, 'lost': 15, ...}, ...}

gaps = (CallLog.objects                                                    #
35
    .annotate(prev_call=Window(Lag("started_at"), partition_by=F("lead"),
order_by=F("started_at").asc()))
    .order_by("lead_id", "started_at")
    .values("lead_id", "started_at", "prev_call")[:20])
for g in gaps:
    if g["prev_call"]:
        print(g["lead_id"], g["started_at"] - g["prev_call"])
```

36.

```
from django.db import connection
from reports.queries import agent_leaderboard

SQL = """
SELECT u.username,
       (SELECT COUNT(*) FROM crm_leads l WHERE l.assigned_to_id = u.id)
AS leads_n,
       (SELECT COUNT(*) FROM crm_leads l WHERE l.assigned_to_id = u.id AND
l.status = 'qualified') AS qualified_n,
       (SELECT COUNT(*) FROM leads_calllog c WHERE c.agent_id = u.id)
AS calls_n
FROM accounts_customuser u
WHERE u.role = 'agent'
"""
with connection.cursor() as cur:
    cur.execute(SQL)
    raw = {row[0]: tuple(row[1:]) for row in cur.fetchall()}

orm = {a.username: (a.leads_n, a.qualified_n, a.calls_n) for a in
agent_leaderboard()}
assert raw == orm, (raw, orm)
```

37–40.

```
# reports/tests.py
from datetime import timedelta

from django.contrib.auth import get_user_model
from django.db.models import Count, Q
from django.test import TestCase
from django.utils import timezone
```

```
from leads.models import CallLog, Lead
from .queries import agent_leaderboard, sop_first_call_breaches


class ReportQueryTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user("r_agent",
password="x", role="agent")
        cls.l1 = Lead.objects.create(name="R One", phone="971509990201",
assigned_to=cls.agent, status="qualified")
        cls.l2 = Lead.objects.create(name="R Two", phone="971509990202",
assigned_to=cls.agent)
        for lead in (cls.l1, cls.l1, cls.l2):
            CallLog.objects.create(lead=lead, agent=cls.agent,
outcome="answered", duration_seconds=60)

    def test_dashboard_aggregate(self):                                    #
37
        stats = Lead.objects.aggregate(total=Count("id"), new=Count("id",
filter=Q(status="new")))
        self.assertEqual(stats["total"], Lead.objects.count())
        self.assertEqual(stats["new"],
Lead.objects.filter(status="new").count())

    def test_fanout_and_distinct(self):                                    #
38
        User = get_user_model()
        bad = User.objects.annotate(l=Count("leads"),
c=Count("calls")).get(pk=self.agent.pk)
        good = User.objects.annotate(
            l=Count("leads", distinct=True), c=Count("calls", distinct=True),
        ).get(pk=self.agent.pk)
        self.assertEqual((bad.l, bad.c), (6, 6))        # 2 leads × 3 calls
        self.assertEqual((good.l, good.c), (2, 3))

    def test_conversion_rate(self):                                        #
39
        row = agent_leaderboard().get(pk=self.agent.pk)
        self.assertEqual(row.qualified_n, 1)
        self.assertAlmostEqual(row.conversion, 50.0)

    def test_sop_breaches(self):                                           #
40
        old = timezone.now() - timedelta(days=3)
```

```
        Lead.objects.filter(pk__in=[self.l1.pk,
self.l2.pk]).update(created_at=old)
        CallLog.objects.filter(lead=self.l1).update(started_at=old +
timedelta(hours=2))
        CallLog.objects.filter(lead=self.l2).update(started_at=old +
timedelta(hours=30))
        breaches = sop_first_call_breaches(hours=24)
        self.assertNotIn(self.l1, breaches)
        self.assertIn(self.l2, breaches)
```

```
python manage.py test
git add -A && git commit -m "day 9"
```

Day 10 — Query performance and transactions

Goal: every CRM page runs a fixed, small number of queries no matter how many rows it shows, and every multi-step write is all-or-nothing.

Measure everything with `CaptureQueriesContext` (Day 6). Do A and B in the shell.

A. Drills (1–10) — loading related data

- [ ] **1.** Reproduce N+1: loop 20 leads printing `lead.assigned_to`; count the queries.
- [ ] **2.** Fix it with `select_related("assigned_to")`; count again.
- [ ] **3.** Two-hop `select_related`: `assigned_to__team_lead` and `broker__parent`.
- [ ] **4.** `prefetch_related("tags")`: print each lead's tag names in 2 queries total.
- [ ] **5.** Prefetch a reverse FK: `calls`.
- [ ] **6.** `Prefetch` object: only answered calls, stored on `answered_calls` via `to_attr`.
- [ ] **7.** Nested prefetch: clients → accounts → deposits.
- [ ] **8.** `only()` and `defer()`; touch a deferred field and watch the extra query.
- [ ] **9.** Same report three ways: model instances vs `values()` vs `values_list()` — compare speed for 200 rows.
- [ ] **10.** Second counting tool: `DEBUG=True` + `reset_queries()` + `connection.queries`.

B. Core (11–20) — bulk writes and transactions

- [ ] **11.** Time 200 `create()` calls vs one `bulk_create()` of 1,000 call logs (mark them `notes="perf-test"`).
- [ ] **12.** `bulk_update()` the perf-test calls; then do the same change with a single `update()`; delete them.
- [ ] **13.** Upsert tags with `bulk_create(update_conflicts=True, unique_fields=..., update_fields=...)`.
- [ ] **14.** `log_call()` service: one atomic function that creates a `CallLog`, bumps `contact_attempts` with `F()`, sets `last_contacted_at`, and moves new → contacted with `Case` / `When`.
- [ ] **15.** `atomic()` as a context manager: create a call, raise on purpose, prove nothing was saved.
- [ ] **16.** Savepoint: inner `atomic()` fails with `IntegrityError`, outer block carries on and commits.

- [ ] **17.** `transaction.on_commit()`: show the callback runs only after the outer block commits.
- [ ] **18.** `claim_lead(lead_id, agent)` with `select_for_update()` inside `atomic()`.
- [ ] **19.** Composite indexes: `CallLog(agent, started_at)` and `Lead(assigned_to, status)`; compare `explain()` before and after.
- [ ] **20.** Partial index: `created_at` only for new leads (`condition=Q(status="new")`).

C. Nexora CRM (21–30) — fix the pages

- [ ] **21.** `LeadQueryMixin`: `select_related("assigned_to", "broker")` + `prefetch_related("tags")`; show tags in the list.
- [ ] **22.** Lead detail: one queryset with `select_related` (agent, broker, client) + `Prefetch` of calls with their agents.
- [ ] **23.** Dashboard lead cards: load only the fields the card uses.
- [ ] **24.** Client detail: `prefetch_related("accounts__deposits")`.
- [ ] **25.** Agent page: replace the 4 per-status queries with one ordered query + `{% regroup %}`.
- [ ] **26.** `brokers/tree.py`: load all brokers in one query, build parent → children in Python; use it for ancestors/descendants on the broker page.
- [ ] **27.** `convert_lead_to_client(lead_id)` service: atomic, locks the lead, creates client + trading account, marks qualified, queues a "welcome email" with `on_commit`.
- [ ] **28.** `reassign_open_leads(from_agent, to_agent)`: one atomic `update()`, logs the count on commit.
- [ ] **29.** `upsert_leads(rows)`: insert new phones, update existing ones, in one `bulk_create` call.
- [ ] **30.** Query audit script: hit every CRM page with the test client and print query counts.

D. Challenge (31–40)

- [ ] **31.** Sliced `Prefetch` (Django 4.2+): each lead's 3 latest calls in one extra query.
- [ ] **32.** `prefetch_related_objects()` on a list you already loaded.
- [ ] **33.** "Leads with an answered call" three ways — `Exists`, `pk__in` subquery, join + `distinct()`; compare counts and plans.
- [ ] **34.** `atomic(durable=True)`: see the error when it's nested.
- [ ] **35.** `next_lead_to_dial(agent)` with `select_for_update(skip_locked=True)` — the work-queue pattern.
- [ ] **36.** `log_queries(label)` context manager printing query count and milliseconds.

- [ ] **37.** Test: lead list query count stays the same after adding 30 tagged, assigned leads.
- [ ] **38.** Test: `convert_lead_to_client` rolls back fully when account creation fails (`mock.patch`).
- [ ] **39.** Test: the welcome-email callback fires once (`captureOnCommitCallbacks`).
- [ ] **40.** Test: `upsert_leads` updates an existing phone and inserts a new one.

Solutions

Rule of thumb: FK / one-to-one you read forward → `select_related` (SQL JOIN). Reverse FK and M2M → `prefetch_related` (one extra query per relation, joined in Python).

A. Drills

1–10.

```
# python manage.py shell
import time
from django.db import connection, reset_queries
from django.db.models import Prefetch
from django.test.utils import CaptureQueriesContext

from clients.models import Client
from leads.models import CallLog, Lead

with CaptureQueriesContext(connection) as ctx:                          # 1
    for lead in Lead.objects.all()[:20]:
        print(lead.assigned_to)
print(len(ctx.captured_queries))        # 21 = 1 for leads + 1 per lead

with CaptureQueriesContext(connection) as ctx:                          # 2
    for lead in Lead.objects.select_related("assigned_to")[:20]:
        print(lead.assigned_to)
print(len(ctx.captured_queries))        # 1

qs = Lead.objects.select_related("assigned_to__team_lead", "broker__parent")
# 3
for lead in qs[:5]:
    print(lead.assigned_to.team_lead, lead.broker and lead.broker.parent)

with CaptureQueriesContext(connection) as ctx:                          # 4
    for lead in Lead.objects.prefetch_related("tags")[:20]:
        print(lead.name, [t.name for t in lead.tags.all()])
print(len(ctx.captured_queries))        # 2
```

```
for lead in Lead.objects.prefetch_related("calls")[:5]:                    # 5
    print(lead.name, len(lead.calls.all()))

answered = Prefetch(                                                       # 6
    "calls",

queryset=CallLog.objects.filter(outcome="answered").select_related("agent"),
    to_attr="answered_calls",            # a plain list, not a manager
)
for lead in Lead.objects.prefetch_related(answered)[:5]:
    print(lead.name, [c.agent.username for c in lead.answered_calls])

for client in Client.objects.prefetch_related("accounts__deposits")[:5]:   #
7 — 3 queries total
    for account in client.accounts.all():
        print(client.name, account.number, sum(d.amount for d in
account.deposits.all()))

slim = Lead.objects.only("id", "name", "status")[:5]                       # 8
with CaptureQueriesContext(connection) as ctx:
    for lead in slim:
        lead.notes                          # deferred → 1 extra query per
row
print(len(ctx.captured_queries))            # 6
Lead.objects.defer("notes", "email")

for label, qs in [                                                         # 9
    ("models", Lead.objects.all()),
    ("values", Lead.objects.values("name", "status")),
    ("values_list", Lead.objects.values_list("name", "status")),
]:
    t = time.perf_counter()
    list(qs)
    print(f"{label:<12} {(time.perf_counter() - t) * 1000:.1f} ms")

reset_queries()                           # 10 — only records when DEBUG=True
list(Lead.objects.all()[:5])
print(len(connection.queries), connection.queries[-1]["sql"])
```

B. Core

11–13.

```
from accounts.models import CustomUser
from leads.models import Tag

agent = CustomUser.objects.filter(role="agent").first()                  # 11
lead = Lead.objects.first()

t = time.perf_counter()
for _ in range(200):
    CallLog.objects.create(lead=lead, agent=agent, outcome="busy",
notes="perf-test")
print("200 x create:", time.perf_counter() - t)

t = time.perf_counter()
CallLog.objects.bulk_create(
    [CallLog(lead=lead, agent=agent, outcome="busy", notes="perf-test") for _
in range(1000)],
    batch_size=500,
)
print("1000 x bulk_create:", time.perf_counter() - t)

calls = list(CallLog.objects.filter(notes="perf-test")[:500])            # 12
for c in calls:
    c.duration_seconds = 5
CallLog.objects.bulk_update(calls, ["duration_seconds"], batch_size=250)
CallLog.objects.filter(notes="perf-test").update(duration_seconds=5)     #
same value → one UPDATE wins
CallLog.objects.filter(notes="perf-test").delete()

Tag.objects.bulk_create(                                                 # 13
    [Tag(name="vip", color="#b91c1c"), Tag(name="kyc-pending",
color="#0ea5e9")],
    update_conflicts=True,          # INSERT ... ON CONFLICT(name) DO UPDATE
    unique_fields=["name"],
    update_fields=["color"],
)
```

14, 18.

```
# leads/services.py
from django.db import transaction
from django.db.models import Case, F, Value, When
from django.utils import timezone

from .models import CallLog, Lead
```

```
@transaction.atomic                                                    #
14
def log_call(lead, agent, outcome, duration_seconds=0, notes=""):
    call = CallLog.objects.create(
        lead=lead, agent=agent, outcome=outcome,
        duration_seconds=duration_seconds, notes=notes,
started_at=timezone.now(),
    )
    Lead.objects.filter(pk=lead.pk).update(
        contact_attempts=F("contact_attempts") + 1,
        last_contacted_at=call.started_at,
        status=Case(
            When(status=Lead.Status.NEW, then=Value(Lead.Status.CONTACTED)),
            default=F("status"),
        ),
        updated_at=timezone.now(),
    )
    return call


@transaction.atomic                                                    #
18
def claim_lead(lead_id, agent):
    # row lock until commit on PostgreSQL; a no-op on SQLite (which locks the
whole file on write)
    lead = Lead.objects.select_for_update().get(pk=lead_id)
    if lead.assigned_to_id and lead.assigned_to_id != agent.pk:
        return False
    lead.assigned_to = agent
    lead.save(update_fields=["assigned_to", "updated_at"])
    return True
```

15–17.

```
from django.db import IntegrityError, transaction

lead = Lead.objects.exclude(assigned_to=None).first()                  # 15
before = CallLog.objects.count()
try:
    with transaction.atomic():
        CallLog.objects.create(lead=lead, agent=lead.assigned_to,
outcome="answered")
        raise RuntimeError("dialer crashed")
except RuntimeError:
    pass
CallLog.objects.count() == before        # True — rolled back
```

```
with transaction.atomic():                                              # 16
    Tag.objects.create(name="outer-ok")
    try:
        with transaction.atomic():           # savepoint
            Tag.objects.create(name="outer-ok")      # duplicate →
IntegrityError
    except IntegrityError:
        print("inner rolled back to the savepoint; outer continues")
    Tag.objects.create(name="outer-ok-2")
Tag.objects.filter(name__startswith="outer-ok").count()      # 2

with transaction.atomic():                                              # 17
    Tag.objects.create(name="commit-demo")
    transaction.on_commit(lambda: print("2. runs after COMMIT"))
    print("1. still inside the transaction")
```

Catch the `IntegrityError` *outside* the inner `atomic()`, never inside it — otherwise the transaction is broken for every following query (16).

19–20.

```
# leads/models.py
class Lead(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=["status", "created_at"],
name="lead_status_created_idx"),
            models.Index(fields=["assigned_to", "status"],
name="lead_agent_status_idx"),        # 19
            models.Index(fields=["created_at"], name="lead_new_created_idx",
# 20
                         condition=models.Q(status="new")),
        ]

class CallLog(models.Model):
    class Meta:
        indexes = [
            models.Index(fields=["lead", "started_at"],
name="call_lead_started_idx"),
            models.Index(fields=["agent", "started_at"],
name="call_agent_started_idx"),       # 19
        ]
```

```
# before and after makemigrations + migrate
print(CallLog.objects.filter(agent_id=3).order_by("-started_at").explain())
# after: SEARCH leads_calllog USING INDEX call_agent_started_idx (agent_id=?)
```

C. Nexora CRM

21.

```
# leads/views.py — first line of LeadQueryMixin.get_leads()
qs = (Lead.objects.filter(is_active=True)
      .select_related("assigned_to", "broker")
      .prefetch_related("tags"))
```

```
<!-- lead_list.html: add <th>Tags</th> and -->
<td>{% for tag in lead.tags.all %}<span class="badge">{{ tag }}</span> {%
endfor %}</td>
```

22–25.

```
# leads/views.py — LeadDetailView.get_context_data                          #
22
from django.db.models import Prefetch

lead = get_object_or_404(
    Lead.objects
        .select_related("assigned_to", "broker", "client")      # reverse
one-to-one works here too
        .prefetch_related(
            "tags",
            Prefetch("calls",
queryset=CallLog.objects.select_related("agent")),
        ),
    pk=lead_id,
)

# core/views.py — DashboardView                                            # 23
leads=active.only("id", "name", "phone", "status")[:6],

# clients/views.py — ClientDetailView                                      # 24
client = get_object_or_404(
    Client.objects.select_related("broker",
"lead").prefetch_related("accounts__deposits"),
    ref=self.kwargs["ref"],
)
```

```
# accounts/views.py — AgentDetailView                                    # 25
(replaces by_status)
context["leads"] = agent.leads.order_by("status", "-created_at")
```

```
<!-- accounts/agent_detail.html — 25 -->
{% regroup leads by get_status_display as groups %}
{% for group in groups %}
  <h2>{{ group.grouper }} ({{ group.list|length }})</h2>
  <ul>{% for lead in group.list|slice:":10" %}<li><a href="{{
lead.get_absolute_url }}">{{ lead.name }}</a></li>{% endfor %}</ul>
{% endfor %}
```

26.

```
# brokers/tree.py
from collections import defaultdict
from .models import Broker


def load_tree():
    """One query → (by_id, children) maps."""
    brokers = list(Broker.objects.order_by("code"))
    by_id = {b.pk: b for b in brokers}
    children = defaultdict(list)
    for b in brokers:
        children[b.parent_id].append(b)
    return by_id, children


def walk(children, parent_id=None, depth=0):
    """Yield (broker, depth) depth-first."""
    for b in children[parent_id]:
        yield b, depth
        yield from walk(children, b.pk, depth + 1)


def ancestors_of(broker, by_id):
    chain, pid = [], broker.parent_id
    while pid is not None:
        node = by_id[pid]
        chain.append(node)
        pid = node.parent_id
    return chain

# brokers/views.py — BrokerDetailView.get_context_data
```

```
by_id, children = load_tree()
broker = get_object_or_404(Broker, code=self.kwargs["code"])
context.update(
    broker=broker,
    ancestors=ancestors_of(broker, by_id)[::-1],
    children=children[broker.pk],
    descendant_count=sum(1 for _ in walk(children, broker.pk)),
    client_count=broker.clients.count(),
)
```

27.

```
# clients/services.py
from django.db import transaction

from leads.models import Lead
from .models import Client, TradingAccount


def next_account_number():
    last = TradingAccount.objects.order_by("-number").values_list("number",
flat=True).first()
    return str(int(last) + 1) if last else "2000001"


@transaction.atomic
def convert_lead_to_client(lead_id, platform="mt5", currency="USD"):
    lead = Lead.objects.select_for_update().get(pk=lead_id)
    if hasattr(lead, "client"):
        raise ValueError("Lead already converted")
    client = Client.objects.create(
        lead=lead, broker=lead.broker, name=lead.name,
        email=lead.email or f"client{lead.pk}@example.com",
        account_number=f"{1_000_000 + lead.pk:07d}",
    )
    TradingAccount.objects.create(
        client=client, number=next_account_number(), platform=platform,
currency=currency,
    )
    lead.status = Lead.Status.QUALIFIED
    lead.save(update_fields=["status", "updated_at"])
    transaction.on_commit(lambda: print(f"Welcome email queued for
{client.email}"))   # Day 23: real email
    return client
```

28–29.

```
# leads/services.py (add)
from .models import normalize_uae_phone
from .queries import OPEN_STATUSES


@transaction.atomic                                                    #
28
def reassign_open_leads(from_agent, to_agent):
    n = (Lead.objects
         .filter(assigned_to=from_agent, status__in=OPEN_STATUSES)
         .update(assigned_to=to_agent, updated_at=timezone.now()))
    transaction.on_commit(lambda: print(f"Reassigned {n} leads: {from_agent}
→ {to_agent}"))
    return n


def upsert_leads(rows):                                                #
29
    objs = [
        Lead(
            name=r["name"].strip().title(),
            phone=normalize_uae_phone(r["phone"]),     # bulk_create skips
save() → normalise here
            source=r.get("source", Lead.Source.MANUAL),
            email=r.get("email", ""),
        )
        for r in rows
    ]
    return Lead.objects.bulk_create(
        objs,
        update_conflicts=True,
        unique_fields=["phone"],                       # needs the unique
index on phone
        update_fields=["name", "source", "email", "updated_at"],
        batch_size=500,
    )
```

30.

```
# python manage.py shell
from django.db import connection
from django.test import Client as HttpClient          # not
clients.models.Client
from django.test.utils import CaptureQueriesContext, setup_test_environment

from clients.models import Client
```

```
from leads.models import Lead

setup_test_environment()
http = HttpClient()
lead = Lead.objects.first()
client = Client.objects.first()
agent = lead.assigned_to

urls = ["/leads/", lead.get_absolute_url(), "/dashboard/",
client.get_absolute_url(),
        f"/team/agents/{agent.pk}/", "/brokers/BRF1000/", "/reports/"]
for url in urls:
    with CaptureQueriesContext(connection) as ctx:
        status = http.get(url).status_code
    print(f"{len(ctx.captured_queries):>4} queries  {status}  {url}")
```

D. Challenge

31–36.

```
from django.db import transaction
from django.db.models import Exists, F, OuterRef, prefetch_related_objects

recent = Prefetch(                                                  #
31
    "calls", queryset=CallLog.objects.order_by("-started_at")[:3],
to_attr="recent_calls",
)
for lead in Lead.objects.prefetch_related(recent)[:10]:
    print(lead.name, [c.outcome for c in lead.recent_calls])

leads = list(Lead.objects.filter(status="qualified")[:20])              # 32
prefetch_related_objects(leads, "tags", "calls")

answered = CallLog.objects.filter(outcome="answered")                   # 33
a = Lead.objects.filter(Exists(answered.filter(lead=OuterRef("pk"))))
b = Lead.objects.filter(pk__in=answered.values("lead_id"))
c = Lead.objects.filter(calls__outcome="answered").distinct()
for qs in (a, b, c):
    print(qs.count())
    print(qs.explain())

with transaction.atomic():                                              # 34
    try:
        with transaction.atomic(durable=True):
            pass
```

```
    except RuntimeError as e:
        print(e)      # A durable atomic block cannot be nested within
another atomic block.


# leads/services.py                                                    # 35
def next_lead_to_dial(agent):
    with transaction.atomic():
        lead = (Lead.objects
                .select_for_update(skip_locked=True)    # other agents'
locked rows are skipped (PostgreSQL)
                .filter(assigned_to=agent, status__in=OPEN_STATUSES)
                .order_by(F("last_contacted_at").asc(nulls_first=True),
"created_at")
                .first())
        if lead:

Lead.objects.filter(pk=lead.pk).update(last_contacted_at=timezone.now())
        return lead
```

```
# core/debug.py                                                        # 36
import time
from contextlib import contextmanager

from django.db import connection
from django.test.utils import CaptureQueriesContext


@contextmanager
def log_queries(label="block"):
    start = time.perf_counter()
    with CaptureQueriesContext(connection) as ctx:
        yield ctx
    ms = (time.perf_counter() - start) * 1000
    print(f"[{label}] {len(ctx.captured_queries)} queries in {ms:.1f} ms")

# usage
with log_queries("leaderboard"):
    list(agent_leaderboard())
```

37–40.

```
# leads/tests.py
from unittest import mock

from django.db import IntegrityError, connection
```

```
from django.test.utils import CaptureQueriesContext

from clients.models import Client
from clients.services import convert_lead_to_client
from .models import Tag
from .services import upsert_leads


class PerformanceAndTransactionTests(TestCase):
    def _count(self, url):
        with CaptureQueriesContext(connection) as ctx:
            self.client.get(url)
        return len(ctx.captured_queries)

    def test_lead_list_query_count_is_constant(self):                   # 37
        agent = get_user_model().objects.create_user("qc_agent",
password="x", role="agent")
        tag = Tag.objects.create(name="qc")
        before = self._count(reverse("leads:list"))
        Lead.objects.bulk_create([
            Lead(name=f"QC {i}", phone=f"9715098{i:05d}", assigned_to=agent)
for i in range(30)
        ])
        for lead in Lead.objects.filter(name__startswith="QC"):
            lead.tags.add(tag)
        self.assertEqual(self._count(reverse("leads:list")), before)

    def test_conversion_rolls_back(self):                               # 38
        lead = Lead.objects.create(name="Rollback", phone="971509990501")
        with mock.patch("clients.services.next_account_number",
side_effect=IntegrityError("boom")):
            with self.assertRaises(IntegrityError):
                convert_lead_to_client(lead.pk)
        self.assertFalse(Client.objects.filter(lead=lead).exists())   #
client insert undone
        lead.refresh_from_db()
        self.assertEqual(lead.status, Lead.Status.NEW)

    def test_welcome_email_on_commit(self):                             # 39
        lead = Lead.objects.create(name="Commit", phone="971509990502")
        with self.captureOnCommitCallbacks(execute=True) as callbacks:
            convert_lead_to_client(lead.pk)
        self.assertEqual(len(callbacks), 1)

    def test_upsert(self):                                              # 40
        Lead.objects.create(name="Old Name", phone="971509990601")
```

```
        upsert_leads([
            {"name": "new name", "phone": "+971 50 999 0601", "source":
"google"},
            {"name": "fresh lead", "phone": "971509990602", "source":
"referral"},
        ])
        self.assertEqual(Lead.objects.get(phone="971509990601").name, "New
Name")
        self.assertTrue(Lead.objects.filter(phone="971509990602").exists())
```

`TestCase` wraps each test in a transaction, so `on_commit` callbacks never fire on their own — `captureOnCommitCallbacks(execute=True)` runs them (39).

Day 11 — Admin customization

Goal: turn `/admin/` into Nexora's back office — searchable, filterable lists for every model, inline history, bulk actions, and guard rails on who can edit what.

Replace each `admin.site.register(Model)` line from Days 5 and 8 with a full `@admin.register` class as you go (registering a model twice raises `AlreadyRegistered`).

A. Drills (1–10) — `LeadAdmin` list page

- [ ] **1.** `list_display`: name, phone, status, source, agent, created.
- [ ] **2.** `list_filter` (status, source, agent, tags) + `date_hierarchy` on `created_at`.
- [ ] **3.** `search_fields`: name, phone, email, agent username.
- [ ] **4.** `ordering`, `list_per_page = 50`, and `status` editable straight from the list (`list_editable`).
- [ ] **5.** `list_display_links` on name; `readonly_fields` for timestamps and dialer counters.
- [ ] **6.** `fieldsets`: Contact, Pipeline, Dialer (collapsed), Notes, Timestamps (collapsed).
- [ ] **7.** `autocomplete_fields` for agent and broker (their admins need `search_fields`).
- [ ] **8.** `filter_horizontal` for tags.
- [ ] **9.** Computed column `masked_phone` with `@admin.display(description=...)`, reusing the Day 3 filter.
- [ ] **10.** Boolean column `hot` from `Lead.is_hot` (`boolean=True`).

B. Core (11–20) — inlines, actions, hooks

- [ ] **11.** `CallLogInline` (tabular, read-only, newest first, link to each call) on the lead page.
- [ ] **12.** `TradingAccountInline` (stacked) on clients; `DepositInline` (tabular) on trading accounts.
- [ ] **13.** Action "Mark selected as contacted" with a success message.
- [ ] **14.** Reusable `export_as_csv` action in `core/admin_actions.py`; attach it to leads.
- [ ] **15.** `SimpleListFilter` "Contact state": never contacted / no touch in 48 h / touched in 48 h.
- [ ] **16.** `get_queryset`: `select_related` + annotate a call count; sortable "Calls" column.
- [ ] **17.** Site header, site title and index title.
- [ ] **18.** `save_model`: a new lead created by an agent with no assignee is assigned to that agent.

- [ ] **19.** `get_readonly_fields`: agents can't edit `expected_deposit`.
- [ ] **20.** `formfield_for_foreignkey`: agent dropdown lists active agents only.

C. Nexora CRM (21–30) — the other models

- [ ] **21.** `BrokerAdmin`: code, name, tier, parent, commission, client count; filter tier; search; autocomplete parent.
- [ ] **22.** Sub-brokers inline on the broker page (`fk_name="parent"`).
- [ ] **23.** `ClientAdmin`: KYC filter, accounts inline, read-only `ref`, total deposits column, "Approve KYC" action.
- [ ] **24.** `TradingAccountAdmin`: deposits inline and a deposits-total column.
- [ ] **25.** `DepositAdmin`: date hierarchy, method filter, search by account number or client name, client column.
- [ ] **26.** `CallLogAdmin`: `list_select_related`, filters for outcome and agent, date hierarchy, search.
- [ ] **27.** `CustomUserAdmin`: lead-count column, role/team-lead filters, read-only team-members inline.
- [ ] **28.** `TagAdmin`: colour swatch with `format_html` and a lead count.
- [ ] **29.** Lead list column linking to the converted client's admin page (`reverse("admin:...")`).
- [ ] **30.** Attach `export_as_csv` to clients and deposits too.

D. Challenge (31–40)

- [ ] **31.** Intermediate-page action "Reassign to agent…": form to pick the agent, then apply.
- [ ] **32.** Custom admin page `/admin/leads/lead/stats/` via `get_urls()`, showing the Day 9 reports.
- [ ] **33.** Make `CallLog` read-only in the admin (no add, change, delete) — it's audit data.
- [ ] **34.** Second `AdminSite` at `/backoffice/` with only leads, calls, users and brokers.
- [ ] **35.** Override the lead change-list template to show "N new leads today" above the table.
- [ ] **36.** `get_search_results`: a phone typed in any format (`+971 50 …`) finds the lead.
- [ ] **37.** Test: lead change list loads for a superuser with filters applied.
- [ ] **38.** Test: the "mark contacted" action updates the row.
- [ ] **39.** Test: the CallLog add page returns 403.
- [ ] **40.** Test: searching `+971 50 999 0301` finds the lead.

Solutions

Shared action (14, 30)

```
# core/admin_actions.py
import csv

from django.contrib import admin
from django.http import HttpResponse


@admin.action(description="Export selected to CSV")
def export_as_csv(modeladmin, request, queryset):
    meta = modeladmin.model._meta
    fields = [f.name for f in meta.concrete_fields]
    response = HttpResponse(content_type="text/csv")
    response["Content-Disposition"] = f'attachment; filename="
{meta.model_name}s.csv"'
    writer = csv.writer(response)
    writer.writerow(fields)
    for obj in queryset:
        writer.writerow([getattr(obj, f) for f in fields])
    return response              # returning a response sends it instead of
reloading the list
```

A + B + D. Final `leads/admin.py` (1–20, 29, 31–33, 35–36)

```
# leads/admin.py
import re
from datetime import timedelta

from django import forms
from django.contrib import admin, messages
from django.contrib.admin import helpers
from django.contrib.auth import get_user_model
from django.db.models import Count
from django.shortcuts import render
from django.template.response import TemplateResponse
from django.urls import path, reverse
from django.utils import timezone
from django.utils.html import format_html

from core.admin_actions import export_as_csv
from reports.queries import agent_leaderboard, source_performance
from .models import CallLog, Lead, Tag
```

```
from .templatetags.crm_filters import mask_phone


class CallLogInline(admin.TabularInline):                                       #
11, 33
    model = CallLog
    extra = 0
    fields = ["started_at", "agent", "outcome", "duration_seconds"]
    readonly_fields = fields
    ordering = ["-started_at"]
    show_change_link = True
    can_delete = False

    def has_add_permission(self, request, obj=None):
        return False


class ContactStateFilter(admin.SimpleListFilter):                               #
15
    title = "contact state"
    parameter_name = "contact"

    def lookups(self, request, model_admin):
        return [("never", "Never contacted"), ("overdue", "No touch in 48
h"), ("recent", "Touched in 48 h")]

    def queryset(self, request, queryset):
        cutoff = timezone.now() - timedelta(hours=48)
        if self.value() == "never":
            return queryset.filter(last_contacted_at__isnull=True)
        if self.value() == "overdue":
            return queryset.filter(last_contacted_at__lt=cutoff)
        if self.value() == "recent":
            return queryset.filter(last_contacted_at__gte=cutoff)
        return queryset


class ReassignForm(forms.Form):                                                 #
31
    agent = forms.ModelChoiceField(
        queryset=get_user_model().objects.filter(role="agent",
is_active=True),
    )


@admin.register(Lead)
```

```
class LeadAdmin(admin.ModelAdmin):
    list_display = ["name", "masked_phone", "status", "source",
"assigned_to",          # 1, 9-10, 16, 29
                    "call_count", "hot", "client_link", "created_at"]
    list_display_links = ["name"]
# 5
    list_editable = ["status"]
# 4
    list_filter = ["status", "source", ContactStateFilter, "assigned_to",
"tags"]       # 2, 15
    date_hierarchy = "created_at"
    search_fields = ["name", "phone", "email", "assigned_to__username"]
# 3
    ordering = ["-created_at"]
    list_per_page = 50
    readonly_fields = ["created_at", "updated_at", "contact_attempts",
"last_contacted_at"]
    autocomplete_fields = ["assigned_to", "broker"]
# 7
    filter_horizontal = ["tags"]
# 8
    inlines = [CallLogInline]
    actions = ["mark_contacted", "reassign", export_as_csv]
# 13, 31, 14
    fieldsets = [
# 6
        ("Contact", {"fields": ["name", "phone", "email", "country"]}),
        ("Pipeline", {"fields": ["status", "source", "assigned_to", "broker",
"tags", "expected_deposit"]}),
        ("Dialer", {"fields": ["contact_attempts", "last_contacted_at"],
"classes": ["collapse"]}),
        ("Notes", {"fields": ["notes", "is_active"]}),
        ("Timestamps", {"fields": ["created_at", "updated_at"], "classes":
["collapse"]}),
    ]

    # ---- queryset & columns ---------------------------------------------
--
    def get_queryset(self, request):
# 16
        return (super().get_queryset(request)
                .select_related("assigned_to", "broker", "client")
                .annotate(_call_count=Count("calls")))

    @admin.display(description="Phone")
    def masked_phone(self, obj):
```

```
        return mask_phone(obj.phone)

    @admin.display(description="Calls", ordering="_call_count")
    def call_count(self, obj):
        return obj._call_count

    @admin.display(description="Hot", boolean=True)
    def hot(self, obj):
        return obj.is_hot

    @admin.display(description="Client")
# 29
    def client_link(self, obj):
        if not hasattr(obj, "client"):
            return "—"
        url = reverse("admin:clients_client_change", args=[obj.client.pk])
        return format_html('<a href="{}">{}</a>', url,
obj.client.account_number)

    # ---- actions ----------------------------------------------------------
----
    @admin.action(description="Mark selected as contacted")
# 13
    def mark_contacted(self, request, queryset):
        pks = list(queryset.values_list("pk", flat=True))     # drop the
Count annotation
        n =
Lead.objects.filter(pk__in=pks).update(status=Lead.Status.CONTACTED,
updated_at=timezone.now())
        self.message_user(request, f"{n} lead(s) marked contacted.",
messages.SUCCESS)

    @admin.action(description="Reassign selected to an agent…")
# 31
    def reassign(self, request, queryset):
        if "apply" in request.POST:
            form = ReassignForm(request.POST)
            if form.is_valid():
                agent = form.cleaned_data["agent"]
                pks = list(queryset.values_list("pk", flat=True))
                n = Lead.objects.filter(pk__in=pks).update(assigned_to=agent,
updated_at=timezone.now())
                self.message_user(request, f"{n} lead(s) reassigned to
{agent}.", messages.SUCCESS)
                return None                    # back to the change list
        else:
```

```
            form = ReassignForm()
        return render(request, "admin/leads/reassign.html", {
            **self.admin_site.each_context(request),
            "title": "Reassign leads",
            "form": form,
            "leads": queryset,
            "action_checkbox_name": helpers.ACTION_CHECKBOX_NAME,
        })

    # ---- hooks ----------------------------------------------------------
----
    def save_model(self, request, obj, form, change):
# 18
        if not change and obj.assigned_to is None and getattr(request.user,
"role", "") == "agent":
            obj.assigned_to = request.user
        super().save_model(request, obj, form, change)

    def get_readonly_fields(self, request, obj=None):
# 19
        fields = list(super().get_readonly_fields(request, obj))
        if not request.user.is_superuser and getattr(request.user, "role",
"") == "agent":
            fields.append("expected_deposit")
        return fields

    def formfield_for_foreignkey(self, db_field, request, **kwargs):
# 20
        if db_field.name == "assigned_to":
            kwargs["queryset"] =
get_user_model().objects.filter(role="agent", is_active=True)
        return super().formfield_for_foreignkey(db_field, request, **kwargs)

    def get_search_results(self, request, queryset, search_term):
# 36
        digits = re.sub(r"\D", "", search_term)
        if len(digits) >= 4 and not re.search(r"[A-Za-z]", search_term):
            return queryset.filter(phone__contains=digits), False
        return super().get_search_results(request, queryset, search_term)

    def changelist_view(self, request, extra_context=None):
# 35
        extra_context = {
            **(extra_context or {}),
            "today_count":
Lead.objects.filter(created_at__date=timezone.localdate()).count(),
```

```
        }
        return super().changelist_view(request, extra_context=extra_context)

    # ---- custom page ------------------------------------------------------
----
    def get_urls(self):
# 32
        custom = [
            path("stats/", self.admin_site.admin_view(self.stats_view),
name="leads_lead_stats"),
        ]
        return custom + super().get_urls()     # custom first, or
<path:object_id>/ swallows it

    def stats_view(self, request):
        return TemplateResponse(request, "admin/leads/stats.html", {
            **self.admin_site.each_context(request),
            "title": "Lead stats",
            "sources": source_performance(),
            "leaderboard": agent_leaderboard(),
        })


@admin.register(CallLog)
class CallLogAdmin(admin.ModelAdmin):                                   #
26, 33
    list_display = ["started_at", "lead", "agent", "outcome",
"duration_seconds"]
    list_filter = ["outcome", "agent"]
    date_hierarchy = "started_at"
    search_fields = ["lead__name", "lead__phone", "agent__username"]
    list_select_related = ["lead", "agent"]

    def has_add_permission(self, request):
        return False

    def has_change_permission(self, request, obj=None):
        return False                  # view permission still lets staff read
it

    def has_delete_permission(self, request, obj=None):
        return False


@admin.register(Tag)
class TagAdmin(admin.ModelAdmin):                                       #
```

```
28
    list_display = ["name", "swatch", "lead_count"]
    search_fields = ["name"]

    def get_queryset(self, request):
        return super().get_queryset(request).annotate(_leads=Count("leads"))

    @admin.display(description="Colour")
    def swatch(self, obj):
        return format_html(
            '<span style="display:inline-block;width:1rem;height:1rem;border-
radius:4px;'
            'background:{}"></span> {}', obj.color, obj.color,
        )

    @admin.display(description="Leads", ordering="_leads")
    def lead_count(self, obj):
        return obj._leads
```

`format_html` escapes its arguments — never build admin HTML with f-strings (28, 29).

17.

```
# django_project/urls.py
admin.site.site_header = "Nexora CRM back office"
admin.site.site_title = "Nexora admin"
admin.site.index_title = "Operations"
```

C. Nexora CRM

21–22.

```
# brokers/admin.py
from django.contrib import admin
from django.db.models import Count
from .models import Broker


class SubBrokerInline(admin.TabularInline):
    model = Broker
    fk_name = "parent"              # the self-FK this inline follows
    fields = ["code", "name", "tier", "commission_rate", "is_active"]
    extra = 0
    show_change_link = True
    verbose_name_plural = "Sub-brokers"
```

```
@admin.register(Broker)
class BrokerAdmin(admin.ModelAdmin):
    list_display = ["code", "name", "tier", "parent", "commission_rate",
"clients_n", "is_active"]
    list_filter = ["tier", "is_active"]
    search_fields = ["code", "name"]
    autocomplete_fields = ["parent"]
    list_select_related = ["parent"]
    inlines = [SubBrokerInline]

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_clients=Count("clients",
distinct=True))

    @admin.display(description="Clients", ordering="_clients")
    def clients_n(self, obj):
        return obj._clients
```

23–25, 30.

```
# clients/queries.py
from decimal import Decimal

from django.db.models import DecimalField, OuterRef, Subquery, Sum, Value
from django.db.models.functions import Coalesce

from .models import Deposit


def deposits_total():
    """Annotation: total deposits for each Client row (join-safe)."""
    per_client = (Deposit.objects
        .filter(account__client=OuterRef("pk"))
        .order_by()
        .values("account__client")
        .annotate(t=Sum("amount"))
        .values("t"))
    return Coalesce(Subquery(per_client), Value(Decimal("0")),
                    output_field=DecimalField(max_digits=14,
decimal_places=2))
```

```
# clients/admin.py
from django.contrib import admin, messages
from django.db.models import Sum
from django.urls import reverse
from django.utils.html import format_html

from core.admin_actions import export_as_csv
from .models import Client, Deposit, TradingAccount
from .queries import deposits_total


class TradingAccountInline(admin.StackedInline):
    model = TradingAccount
    extra = 0
    show_change_link = True


class DepositInline(admin.TabularInline):
    model = Deposit
    extra = 1
    fields = ["amount", "method", "created_at"]
    can_delete = False             # money records are never deleted from
here


@admin.register(Client)
class ClientAdmin(admin.ModelAdmin):
# 23
    list_display = ["name", "account_number", "account_type", "kyc_status",
"broker",
                    "total_deposits", "date_joined"]
    list_filter = ["kyc_status", "account_type", "broker"]
    search_fields = ["name", "email", "account_number", "accounts__number"]
    readonly_fields = ["ref", "date_joined", "lead_link"]
    autocomplete_fields = ["broker", "lead"]
    list_select_related = ["broker"]
    inlines = [TradingAccountInline]
    actions = ["approve_kyc", export_as_csv]
# 30

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_deposits=deposits_total())

    @admin.display(description="Deposits", ordering="_deposits")
    def total_deposits(self, obj):
```

```
        return f"{obj._deposits:,.2f}"

    @admin.display(description="Source lead")
    def lead_link(self, obj):
        if not obj.lead_id:
            return "—"
        url = reverse("admin:leads_lead_change", args=[obj.lead_id])
        return format_html('<a href="{}">{}</a>', url, obj.lead.name)

    @admin.action(description="Approve KYC")
    def approve_kyc(self, request, queryset):
        n =
Client.objects.filter(pk__in=queryset.values("pk")).update(kyc_status=Client.
KYCStatus.APPROVED)
        self.message_user(request, f"{n} client(s) approved.",
messages.SUCCESS)


@admin.register(TradingAccount)
class TradingAccountAdmin(admin.ModelAdmin):
# 24
    list_display = ["number", "client", "platform", "currency", "leverage",
"deposits_sum"]
    list_filter = ["platform", "currency"]
    search_fields = ["number", "client__name"]
    list_select_related = ["client"]
    inlines = [DepositInline]

    def get_queryset(self, request):
        return
super().get_queryset(request).annotate(_total=Sum("deposits__amount",
default=0))

    @admin.display(description="Deposits", ordering="_total")
    def deposits_sum(self, obj):
        return f"{obj._total:,.2f}"


@admin.register(Deposit)
class DepositAdmin(admin.ModelAdmin):
# 25
    list_display = ["created_at", "amount", "method", "account",
"client_name"]
    list_filter = ["method"]
    date_hierarchy = "created_at"
    search_fields = ["account__number", "account__client__name"]
```

```
    list_select_related = ["account__client"]
    actions = [export_as_csv]
# 30

    @admin.display(description="Client", ordering="account__client__name")
    def client_name(self, obj):
        return obj.account.client.name
```

27.

```
# accounts/admin.py (extend Day 8's CustomUserAdmin)
from django.db.models import Count


class TeamMemberInline(admin.TabularInline):
    model = CustomUser
    fk_name = "team_lead"
    fields = ["username", "first_name", "role", "is_active"]
    readonly_fields = fields       # create users through the normal add form
(password hashing)
    extra = 0
    max_num = 0
    can_delete = False
    show_change_link = True
    verbose_name_plural = "Team members"


@admin.register(CustomUser)
class CustomUserAdmin(UserAdmin):
    ...
    list_display = ["username", "email", "role", "team_lead", "lead_count",
"is_active"]
    list_filter = UserAdmin.list_filter + ("role", "team_lead")
    inlines = [TeamMemberInline]

    def get_queryset(self, request):
        return super().get_queryset(request).annotate(_leads=Count("leads",
distinct=True))

    @admin.display(description="Leads", ordering="_leads")
    def lead_count(self, obj):
        return obj._leads
```

D. Challenge — templates and the second site

31.

```
<!-- templates/admin/leads/reassign.html -->
{% extends "admin/base_site.html" %}
{% block content %}
<form method="post">{% csrf_token %}
  <p>Reassign {{ leads|length }} lead{{ leads|length|pluralize }}:</p>
  <ul>{% for lead in leads|slice:":20" %}<li>{{ lead }}</li>{% endfor %}</ul>
  {% for lead in leads %}<input type="hidden" name="{{ action_checkbox_name
}}" value="{{ lead.pk }}">{% endfor %}
  {{ form.as_p }}
  <input type="hidden" name="action" value="reassign">
  <input type="submit" name="apply" value="Reassign">
</form>
{% endblock content %}
```

32.

```
<!-- templates/admin/leads/stats.html  → /admin/leads/lead/stats/ -->
{% extends "admin/base_site.html" %}
{% block content %}
<h2>Sources</h2>
<table>
  <tr><th>Source</th><th>Leads</th><th>Qualified</th><th>Conversion</th></tr>
  {% for s in sources %}<tr><td>{{ s.source }}</td><td>{{ s.leads_n }}</td>
<td>{{ s.qualified_n }}</td><td>{{ s.conversion|floatformat:1 }}%</td></tr>{%
endfor %}
</table>
<h2>Agents</h2>
<table>
  <tr><th>Agent</th><th>Leads</th><th>Conversion</th></tr>
  {% for a in leaderboard %}<tr><td>{{ a }}</td><td>{{ a.leads_n }}</td><td>
{{ a.conversion|floatformat:1 }}%</td></tr>{% endfor %}
</table>
{% endblock content %}
```

34.

```
# core/admin_site.py
from django.contrib.admin import AdminSite


class BackOfficeSite(AdminSite):
    site_header = "Nexora back office"
    site_title = "Back office"
    index_title = "Daily operations"
```

```
backoffice = BackOfficeSite(name="backoffice")

# at the bottom of each app's admin.py — autocomplete targets must be
registered on this site too
from core.admin_site import backoffice
backoffice.register(Lead, LeadAdmin)            # leads/admin.py
backoffice.register(CallLog, CallLogAdmin)
backoffice.register(CustomUser, CustomUserAdmin)   # accounts/admin.py
backoffice.register(Broker, BrokerAdmin)           # brokers/admin.py

# django_project/urls.py
from core.admin_site import backoffice
path("backoffice/", backoffice.urls),
```

35.

```
<!-- templates/admin/leads/lead/change_list.html  (the admin finds this path
automatically) -->
{% extends "admin/change_list.html" %}
{% block content_title %}
  {{ block.super }}
  <p><strong>{{ today_count }}</strong> new lead{{ today_count|pluralize }}
today</p>
{% endblock content_title %}
```

37–40.

```
# leads/tests.py
class LeadAdminTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.boss = get_user_model().objects.create_superuser(
            "boss", "boss@nexora.test", "pass12345", role="admin",
        )
        cls.lead = Lead.objects.create(name="Admin Test",
phone="971509990301")

    def setUp(self):
        self.client.force_login(self.boss)

    def test_changelist_with_filters(self):                              # 37
        response = self.client.get(
            reverse("admin:leads_lead_changelist"), {"status__exact": "new",
"contact": "never"},
        )
        self.assertEqual(response.status_code, 200)
```

```
        self.assertContains(response, "Admin Test")

    def test_mark_contacted_action(self):                                # 38
        response = self.client.post(reverse("admin:leads_lead_changelist"), {
            "action": "mark_contacted",
            "_selected_action": [self.lead.pk],
            "index": 0,
        })
        self.assertEqual(response.status_code, 302)
        self.lead.refresh_from_db()
        self.assertEqual(self.lead.status, Lead.Status.CONTACTED)

    def test_calllog_add_forbidden(self):                                # 39

self.assertEqual(self.client.get(reverse("admin:leads_calllog_add")).status_c
ode, 403)

    def test_search_by_formatted_phone(self):                            # 40
        response = self.client.get(reverse("admin:leads_lead_changelist"),
{"q": "+971 50 999 0301"})
        self.assertContains(response, "Admin Test")
```

Day 12 — ListView, DetailView, pagination

Goal: replace the hand-rolled `TemplateView` pages with Django's generic display views — paginated lists that keep their filters, detail pages that load in a few queries, and date-based archives for calls and deposits.

Django 5.1+ ships `{% querystring %}`, which builds pagination links that keep every other GET parameter.

A. Drills (1–10) — ListView and DetailView basics

- [ ] **1.** `LeadListView(ListView)`: `context_object_name = "leads"`, same template, `get_queryset()` returns `self.get_leads()`.
- [ ] **2.** `paginate_by = 25`; in the template use `page_obj`, `paginator`, `is_paginated`.
- [ ] **3.** Pagination partial `templates/partials/_pagination.html`: First / Prev / "Page x of y" / Next / Last.
- [ ] **4.** Use `{% querystring page=... %}` so `q`, `status`, `sort` survive page changes.
- [ ] **5.** `?per_page=` with `get_paginate_by()` (whitelist 10/25/50/100).
- [ ] **6.** `LeadDetailView(DetailView)` with `pk_url_kwarg = "lead_id"` and `context_object_name = "lead"`.
- [ ] **7.** `BrokerDetailView(DetailView)` looked up by code: `slug_field` + `slug_url_kwarg`.
- [ ] **8.** Move the detail page extras (tree, counts, session "recent") into `get_context_data` using `self.object`.
- [ ] **9.** `get_queryset()` on detail views with the Day 10 `select_related` / `prefetch_related`.
- [ ] **10.** In the shell: `Paginator` — `page()`, `get_page()` with junk input, `num_pages`, `page_range`, `get_elided_page_range()`.

B. Core (11–20) — more list pages

- [ ] **11.** `ClientListView`: search (name, account, trading account number), KYC filter, total deposits column, 25 per page.
- [ ] **12.** `ClientDetailView(DetailView)` looked up by UUID `ref`.
- [ ] **13.** `AgentDetailView(DetailView)` whose queryset only allows agents.
- [ ] **14.** `CallLogListView` at `/leads/calls/`: 50 per page, filters outcome / agent / day, `ordering` attribute.

- [ ] **15.** Paginate the call history inside lead detail with its own `?calls_page=`.
- [ ] **16.** `allow_empty = False` on a strict calls route — empty result → 404.
- [ ] **17.** Try `?page=999` (404) and `?page=last` (last page).
- [ ] **18.** `get_ordering()` on calls from `?sort=` with a whitelist.
- [ ] **19.** Delete `template_name` from `ClientListView` and rely on the default `clients/client_list.html`.
- [ ] **20.** Pass `queryset = ...` as a class attribute (instead of `model`) on a view that needs no request data.

C. Nexora CRM (21–30)

- [ ] **21.** Lead list on `ListView` keeps everything: filters, chips, sort, annotations, tags.
- [ ] **22.** One pagination partial used by leads, clients and calls.
- [ ] **23.** "Showing 26–50 of 213" under each table.
- [ ] **24.** Lead detail: Older / Newer links (`get_previous_by_created_at` / `get_next_by_created_at`).
- [ ] **25.** Lead detail sidebar: the 5 recently viewed leads from the session.
- [ ] **26.** Clients page wired into the sidebar; deposits via `clients/queries.deposits_total()`.
- [ ] **27.** Broker list page `/brokers/`: whole tree, indented by depth, one query (`brokers/tree.py`).
- [ ] **28.** Agent call queue `/team/agents/<pk>/queue/`: their follow-up-due + stale leads, never-called first, paginated.
- [ ] **29.** `/leads/api/page/?page=`: JSON with page, num_pages, count, has_next, results.
- [ ] **30.** `?partial=1` returns only the table rows (ready for "load more" on Day 27).

D. Challenge (31–40)

- [ ] **31.** Keyset pagination for calls: `/leads/api/calls/?after=<timestamp>,<id>` — no OFFSET, stable under inserts.
- [ ] **32.** `FilteredListView` base: a `filters = {param: lookup}` dict drives filtering; use it for calls.
- [ ] **33.** `paginate_orphans = 5` and see the last page absorb a short tail.
- [ ] **34.** `ArchiveIndexView` + `DayArchiveView` for calls: `/leads/calls/archive/` and `/leads/calls/2026/10/15/`.
- [ ] **35.** `MonthArchiveView` for deposits with a month total.

- [ ] **36.** Check a paginated page runs the same number of queries on page 1 and page 3.
- [ ] **37.** Test: page 1 has 25 leads and a next page.
- [ ] **38.** Test: the next-page link keeps `q`.
- [ ] **39.** Test: detail page Older / Newer leads match the model methods.
- [ ] **40.** Test: two keyset pages don't overlap.

Solutions

A + C. Leads

1–2, 5, 6, 8–9, 15, 21, 24–25, 30.

```
# leads/views.py
from django.core.paginator import Paginator
from django.views.generic import DetailView, ListView


@method_decorator(log_request, name="dispatch")
class LeadListView(PageTitleMixin, LeadQueryMixin, ListView):           # 1–
2, 5, 21, 30
    template_name = "leads/lead_list.html"
    context_object_name = "leads"          # the current page's rows
    paginate_by = 25
    page_title = "Leads"
    PER_PAGE = {10, 25, 50, 100}

    def get_queryset(self):
        return self.get_leads()            # filters, sort, annotations,
select/prefetch from Days 7–10

    def get_paginate_by(self, queryset):                                  # 5
        value = self.request.GET.get("per_page", "")
        return int(value) if value.isdigit() and int(value) in self.PER_PAGE
else self.paginate_by

    def get_template_names(self):                                         #
30
        if self.request.GET.get("partial") == "1":
            return ["leads/partials/_lead_rows.html"]
        if self.request.GET.get("compact") == "1":
            return ["leads/lead_list_compact.html"]
        return [self.template_name]
```

```
class LeadDetailView(PageTitleMixin, DetailView):                          # 6,
8-9, 15, 24-25
    template_name = "leads/lead_detail.html"
    context_object_name = "lead"
    pk_url_kwarg = "lead_id"               # URL is <int:lead_id>, not
<int:pk>
    page_title = "Lead"

    def get_queryset(self):                                                 # 9
        return (Lead.objects.filter(is_active=True)
                .select_related("assigned_to", "broker", "client")
                .prefetch_related("tags"))

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        lead = self.object

        recent = [pk for pk in self.request.session.get("recent_leads", [])
if pk != lead.pk]   # 25
        found = Lead.objects.only("id", "name").in_bulk(recent)
        context["recent_leads"] = [found[pk] for pk in recent if pk in found]
        self.request.session["recent_leads"] = [lead.pk, *recent][:5]

        calls = lead.calls.select_related("agent")                          #
15
        context["calls_page"] = Paginator(calls,
10).get_page(self.request.GET.get("calls_page"))

        context["older_lead"] =
self._neighbour(lead.get_previous_by_created_at)   # 24
        context["newer_lead"] = self._neighbour(lead.get_next_by_created_at)
        return context

    @staticmethod
    def _neighbour(method):
        try:
            return method(is_active=True)      # extra kwargs filter the
neighbour search
        except Lead.DoesNotExist:
            return None
```

The Day 3 "By status" `{% regroup %}` block on the lead list now only sees one page — delete it.

3-4, 22-23.

```
<!-- templates/partials/_pagination.html -->
{% if page_obj.paginator.num_pages > 1 %}
<nav class="pagination">
  {% if page_obj.has_previous %}
    <a href="{% querystring page=1 %}">« First</a>
    <a href="{% querystring page=page_obj.previous_page_number %}">‹ Prev</a>
  {% endif %}
  <span>Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
</span>
  {% if page_obj.has_next %}
    <a href="{% querystring page=page_obj.next_page_number %}">Next ›</a>
    <a href="{% querystring page=page_obj.paginator.num_pages %}">Last »</a>
  {% endif %}
</nav>
{% endif %}
{% if page_obj.paginator.count %}
  <p class="muted">Showing {{ page_obj.start_index }}–{{ page_obj.end_index
}} of {{ page_obj.paginator.count }}</p>
{% endif %}
```

`{% querystring page=3 %}` copies the current `request.GET`, sets `page`, and outputs `?q=khan&status=new&page=3`.

```
<!-- leads/lead_list.html — table body + pagination -->
<tbody>
  {% include "leads/partials/_lead_rows.html" %}
</tbody>
</table>
{% include "partials/_pagination.html" %}
```

```
<!-- leads/templates/leads/partials/_lead_rows.html  (30: returned alone for
?partial=1) -->
{% load crm_filters %}
{% for lead in leads %}
<tr class="{% cycle 'row-light' 'row-dark' %}">
  <td><a href="{{ lead.get_absolute_url }}">{{ lead.name }}</a></td>
  <td>{{ lead.phone|mask_phone }}</td>
  <td>{% include "partials/_badge.html" with status=lead.status only %}</td>
  <td>{{ lead.get_source_display }}</td>
  <td>{{ lead.assigned_to|default:"—" }}</td>
  <td>{% for tag in lead.tags.all %}<span class="badge">{{ tag }}</span> {%
endfor %}</td>
  <td>{{ lead.call_count }}</td>
  <td>{{ lead.last_call_at|date:"d M H:i"|default:"—" }}</td>
</tr>
{% empty %}
<tr><td colspan="8">No leads match.</td></tr>
{% endfor %}
```

```
<!-- leads/lead_detail.html — additions (15, 24, 25) -->
<p>
  {% if older_lead %}<a href="{{ older_lead.get_absolute_url }}">‹ Older: {{
older_lead.name }}</a>{% endif %}
  {% if newer_lead %}<a href="{{ newer_lead.get_absolute_url }}">Newer: {{
newer_lead.name }} ›</a>{% endif %}
</p>

<h2>Calls ({{ calls_page.paginator.count }})</h2>
<table class="table">
  {% for call in calls_page %}
    <tr><td>{{ call.started_at|date:"d M H:i" }}</td><td>{{ call.agent }}
</td>
        <td>{{ call.get_outcome_display }}</td><td>{{ call.duration_seconds
}}s</td></tr>
  {% empty %}<tr><td colspan="4">No calls yet.</td></tr>{% endfor %}
</table>
{% if calls_page.has_previous %}<a href="{% querystring
calls_page=calls_page.previous_page_number %}">‹ Newer calls</a>{% endif %}
{% if calls_page.has_next %}<a href="{% querystring
calls_page=calls_page.next_page_number %}">Older calls ›</a>{% endif %}

<aside>
```

```
  <h3>Recently viewed</h3>
  <ul>{% for r in recent_leads %}<li><a href="{{ r.get_absolute_url }}">{{
r.name }}</a></li>{% empty %}<li>—</li>{% endfor %}</ul>
</aside>
```

10.

```
# python manage.py shell
from django.core.paginator import EmptyPage, Paginator
from leads.models import Lead

p = Paginator(Lead.objects.all(), 25)
p.count, p.num_pages, list(p.page_range)
page = p.page(2)
page.has_next(), page.start_index(), page.end_index(), len(page.object_list)
p.get_page("abc").number          # 1 — junk falls back to the first page
p.get_page(999).number            # last page
try:
    p.page(999)                   # strict version
except EmptyPage:
    print("no such page")
list(p.get_elided_page_range(5, on_each_side=1, on_ends=1))   # [1, '…', 4,
5, 6, '…', 9]
```

Brokers (7, 27)

```
# brokers/views.py
from django.views.generic import DetailView, TemplateView
from .tree import ancestors_of, load_tree, walk


class BrokerDetailView(PageTitleMixin, DetailView):                    # 7
    model = Broker                    # context name defaults to "broker"
    slug_field = "code"               # model field
    slug_url_kwarg = "code"           # URL kwarg
    template_name = "brokers/broker_detail.html"
    page_title = "Broker"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        by_id, children = load_tree()
        broker = self.object
        context.update(
            ancestors=ancestors_of(broker, by_id)[::-1],
            children=children[broker.pk],
            descendant_count=sum(1 for _ in walk(children, broker.pk)),
```

```
            client_count=broker.clients.count(),
        )
        return context


class BrokerListView(PageTitleMixin, TemplateView):                    # 27
    template_name = "brokers/broker_list.html"
    page_title = "Brokers"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        _, children = load_tree()
        context["rows"] = list(walk(children))      # [(broker, depth), ...]
        return context

# brokers/urls.py
urlpatterns = [
    path("", views.BrokerListView.as_view(), name="list"),
    path("<str:code>/", views.BrokerDetailView.as_view(), name="detail"),
]
```

```
<!-- brokers/templates/brokers/broker_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Broker network</h1>
{% for broker, depth in rows %}
  <div class="depth-{{ depth }}">
    <a href="{% url 'brokers:detail' broker.code %}">{{ broker }}</a> · Tier
{{ broker.tier }} · {{ broker.commission_rate }}%
  </div>
{% endfor %}
{% endblock content %}
```

```
.depth-1 { padding-left: 1.5rem; }
.depth-2 { padding-left: 3rem; }
.depth-3 { padding-left: 4.5rem; }
```

B. Clients, agents, calls

11–12, 19, 26.

```
# clients/views.py
from django.db.models import Q
from django.views.generic import DetailView, ListView


```

```
from core.mixins import PageTitleMixin
from .models import Client
from .queries import deposits_total


class ClientListView(PageTitleMixin, ListView):                              #
11, 19, 26
    context_object_name = "clients"
    paginate_by = 25
    page_title = "Clients"
    extra_context = {"kyc_choices": Client.KYCStatus.choices}
    # no template_name → Django uses clients/client_list.html
(<app>/<model>_list.html)

    def get_queryset(self):
        qs =
Client.objects.select_related("broker").annotate(total_deposits=deposits_tota
l())
        p = self.request.GET
        if kyc := p.get("kyc"):
            qs = qs.filter(kyc_status=kyc)
        if q := p.get("q", "").strip():
            qs = qs.filter(
                Q(name__icontains=q) | Q(account_number__startswith=q) |
Q(accounts__number=q)
            ).distinct()
        return qs.order_by("name")


class ClientDetailView(PageTitleMixin, DetailView):                          # 12
    slug_field = "ref"
    slug_url_kwarg = "ref"
    context_object_name = "client"
    template_name = "clients/client_detail.html"
    page_title = "Client"

    def get_queryset(self):
        return Client.objects.select_related("broker",
"lead").prefetch_related("accounts__deposits")

# clients/urls.py
path("", views.ClientListView.as_view(), name="list"),
path("<uuid:ref>/", views.ClientDetailView.as_view(), name="detail"),
```

```
<!-- clients/templates/clients/client_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Clients</h1>
<form method="get">
  <input name="q" value="{{ request.GET.q }}" placeholder="Name or account
number">
  <select name="kyc">
    <option value="">Any KYC</option>
    {% for value, label in kyc_choices %}
      <option value="{{ value }}"{% if request.GET.kyc == value %} selected{%
endif %}>{{ label }}</option>
    {% endfor %}
  </select>
  <button type="submit">Filter</button>
</form>
<table class="table">
  <tr><th>Name</th><th>Client no.</th><th>KYC</th><th>Broker</th>
<th>Deposits</th></tr>
  {% for c in clients %}
  <tr>
    <td><a href="{{ c.get_absolute_url }}">{{ c.name }}</a></td><td>{{
c.account_number }}</td>
    <td>{{ c.get_kyc_status_display }}</td><td>{{ c.broker|default:"Direct"
}}</td>
    <td>{{ c.total_deposits|floatformat:2 }}</td>
  </tr>
  {% empty %}<tr><td colspan="5">No clients.</td></tr>{% endfor %}
</table>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

13, 28.

```
# accounts/views.py
from django.db.models import F, Q
from django.views.generic import DetailView, ListView
from leads.queries import follow_up_due, stale_new_leads


class AgentDetailView(PageTitleMixin, DetailView):                        # 13
    template_name = "accounts/agent_detail.html"
    context_object_name = "agent"
    page_title = "Agent"

    def get_queryset(self):            # non-agents → 404
```

```
        return
get_user_model().objects.filter(role="agent").select_related("team_lead")

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["leads"] = self.object.leads.order_by("status", "-
created_at")
        return context


class AgentQueueView(PageTitleMixin, ListView):                          # 28
    template_name = "accounts/agent_queue.html"
    context_object_name = "leads"
    paginate_by = 20
    page_title = "Call queue"

    def get_queryset(self):
        self.agent = get_object_or_404(get_user_model(),
pk=self.kwargs["pk"], role="agent")
        due = follow_up_due().values("pk")
        stale = stale_new_leads(days=1).values("pk")
        return (self.agent.leads
                .filter(Q(pk__in=due) | Q(pk__in=stale))
                .order_by(F("last_contacted_at").asc(nulls_first=True),
"created_at"))

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["agent"] = self.agent
        return context

# accounts/urls.py
path("agents/<int:pk>/", views.AgentDetailView.as_view(),
name="agent_detail"),
path("agents/<int:pk>/queue/", views.AgentQueueView.as_view(),
name="agent_queue"),
```

14, 16, 18, 20, 32–33.

```
# core/views_generic.py                                                 # 32
from django.core.exceptions import ValidationError
from django.views.generic import ListView


class FilteredListView(ListView):
    filters = {}                        # {"GET param": "ORM lookup"}

    def get_queryset(self):
        qs = super().get_queryset()    # applies get_ordering()
        for param, lookup in self.filters.items():
            if value := self.request.GET.get(param):
                try:
                    qs = qs.filter(**{lookup: value})
                except (ValueError, ValidationError):   # e.g. agent=abc,
day=not-a-date
                    pass
        return qs
```

```
# leads/views.py
from core.views_generic import FilteredListView


class CallLogListView(PageTitleMixin, FilteredListView):                # 14
    queryset = CallLog.objects.select_related("lead", "agent")           #
20: cloned per request
    template_name = "leads/call_list.html"
    context_object_name = "calls"
    paginate_by = 50
    paginate_orphans = 5               # 33: a final page of ⩽ 5 rows merges
into the previous one
    ordering = "-started_at"
    page_title = "Calls"
    filters = {"outcome": "outcome", "agent": "agent_id", "day":
"started_at__date"}
    SORTABLE = {"started_at", "duration_seconds", "outcome"}

    def get_ordering(self):                                              #
18
        sort = self.request.GET.get("sort", "")
        return sort if sort.lstrip("-") in self.SORTABLE else self.ordering
```

```
# leads/urls.py
path("calls/", views.CallLogListView.as_view(), name="calls"),
path("calls/strict/", views.CallLogListView.as_view(allow_empty=False),
name="calls_strict"),   # 16
```

```
<!-- leads/templates/leads/call_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Calls</h1>
<form method="get">
  <select name="outcome">
    <option value="">Any outcome</option>
    <option value="answered">Answered</option><option value="no_answer">No
answer</option>
    <option value="busy">Busy</option><option
value="voicemail">Voicemail</option>
  </select>
  <input type="date" name="day" value="{{ request.GET.day }}">
  <button type="submit">Filter</button>
</form>
<table class="table">
  <tr><th><a href="{% querystring sort='-started_at' %}">When</a></th>
<th>Lead</th><th>Agent</th>
      <th>Outcome</th><th><a href="{% querystring sort='-duration_seconds'
%}">Talk</a></th></tr>
  {% for call in calls %}
  <tr><td>{{ call.started_at|date:"d M H:i" }}</td>
      <td><a href="{{ call.lead.get_absolute_url }}">{{ call.lead.name }}</a>
</td>
      <td>{{ call.agent }}</td><td>{{ call.get_outcome_display }}</td><td>{{
call.duration_seconds }}s</td></tr>
  {% empty %}<tr><td colspan="5">No calls.</td></tr>{% endfor %}
</table>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

17. `/leads/?page=999` → 404 (`ListView` raises it for out-of-range pages). `/leads/?page=last` → last page — "last" is a built-in keyword.

C. JSON endpoints (29, 31)

```
# leads/views.py
from django.utils.dateparse import parse_datetime
```

```
class LeadPageAPI(LeadQueryMixin, JsonResponseMixin, View):              # 29
    def get(self, request):
        paginator = Paginator(self.get_leads().values(*LEAD_FIELDS), 25)
        page = paginator.get_page(request.GET.get("page"))
        return self.render_to_json({
            "page": page.number,
            "num_pages": paginator.num_pages,
            "count": paginator.count,
            "has_next": page.has_next(),
            "results": list(page.object_list),
        })


class CallFeedAPI(JsonResponseMixin, View):                              # 31
    page_size = 50

    def get(self, request):
        qs = CallLog.objects.select_related("lead", "agent").order_by("-
started_at", "-id")
        if after := request.GET.get("after"):
            ts_raw, _, last_id = after.rpartition(",")
            ts = parse_datetime(ts_raw)
            if ts is None or not last_id.isdigit():
                return json_error("bad cursor")
            # rows strictly after the cursor in (started_at DESC, id DESC)
order
            qs = qs.filter(Q(started_at__lt=ts) | Q(started_at=ts,
id__lt=int(last_id)))
        rows = list(qs[: self.page_size + 1])          # fetch one extra to
know if there's more
        has_more = len(rows) > self.page_size
        rows = rows[: self.page_size]
        last = rows[-1] if rows else None
        return self.render_to_json({
            "results": [
                {"id": c.id, "lead": c.lead.name, "agent": str(c.agent),
                 "outcome": c.outcome, "started_at": c.started_at}
                for c in rows
            ],
            "next": f"{last.started_at.isoformat()},{last.id}" if has_more
else None,
        })

# leads/urls.py
path("api/page/", views.LeadPageAPI.as_view(), name="api_page"),
path("api/calls/", views.CallFeedAPI.as_view(), name="call_feed"),
```

URL-encode the cursor when you send it — the `+` in `+00:00` would otherwise arrive as a space (31).

D. Date-based views (34–35)

```
# leads/views.py                                                    # 34
from django.views.generic.dates import ArchiveIndexView, DayArchiveView


class CallArchiveIndexView(PageTitleMixin, ArchiveIndexView):
    queryset = CallLog.objects.select_related("lead", "agent")
    date_field = "started_at"
    date_list_period = "day"          # date_list = days that have calls
    paginate_by = 50
    template_name = "leads/call_archive.html"
    page_title = "Call archive"


class CallDayArchiveView(PageTitleMixin, DayArchiveView):
    queryset = CallLog.objects.select_related("lead", "agent")
    date_field = "started_at"
    month_format = "%m"               # numeric months in the URL
    allow_empty = True
    template_name = "leads/call_archive_day.html"
    page_title = "Calls by day"

# leads/urls.py
path("calls/archive/", views.CallArchiveIndexView.as_view(),
name="call_archive"),
path("calls/<int:year>/<int:month>/<int:day>/",
views.CallDayArchiveView.as_view(), name="call_day"),
```

```
<!-- leads/templates/leads/call_archive.html -->
{% extends "base.html" %}
{% block content %}
<h1>Call archive</h1>
<p>{% for d in date_list %}<a href="{% url 'leads:call_day' d.year d.month
d.day %}">{{ d|date:"d M" }}</a> {% endfor %}</p>
<ul>{% for call in latest %}<li>{{ call.started_at|date:"d M H:i" }} · {{
call.lead.name }} · {{ call.get_outcome_display }}</li>{% endfor %}</ul>
{% include "partials/_pagination.html" %}
{% endblock content %}
```

```
<!-- leads/templates/leads/call_archive_day.html -->
{% extends "base.html" %}
{% block content %}
<h1>Calls on {{ day|date:"D d M Y" }} ({{ object_list|length }})</h1>
<p>
  {% if previous_day %}<a href="{% url 'leads:call_day' previous_day.year
previous_day.month previous_day.day %}">‹ {{ previous_day|date:"d M" }}</a>{%
endif %}
  {% if next_day %}<a href="{% url 'leads:call_day' next_day.year
next_day.month next_day.day %}">{{ next_day|date:"d M" }} ›</a>{% endif %}
</p>
<ul>{% for call in object_list %}<li>{{ call.started_at|date:"H:i" }} · {{
call.lead.name }} · {{ call.agent }} · {{ call.get_outcome_display }}</li>{%
endfor %}</ul>
{% endblock content %}
```

```
# clients/views.py                                                    # 35
from django.db.models import Sum
from django.views.generic.dates import MonthArchiveView
from .models import Deposit


class DepositMonthView(PageTitleMixin, MonthArchiveView):
    queryset = Deposit.objects.select_related("account__client")
    date_field = "created_at"
    month_format = "%m"
    allow_empty = True
    template_name = "clients/deposit_month.html"
    page_title = "Deposits"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["total"] = self.object_list.aggregate(t=Sum("amount",
default=0))["t"]
        return context

# clients/urls.py
path("deposits/<int:year>/<int:month>/", views.DepositMonthView.as_view(),
name="deposit_month"),
```

```
<!-- clients/templates/clients/deposit_month.html -->
{% extends "base.html" %}
{% block content %}
<h1>Deposits · {{ month|date:"F Y" }} · {{ total|floatformat:2 }}</h1>
<p>
  {% if previous_month %}<a href="{% url 'clients:deposit_month'
previous_month.year previous_month.month %}">‹ {{ previous_month|date:"M Y"
}}</a>{% endif %}
  {% if next_month %}<a href="{% url 'clients:deposit_month' next_month.year
next_month.month %}">{{ next_month|date:"M Y" }} ›</a>{% endif %}
</p>
<table class="table">
  {% for d in object_list %}
    <tr><td>{{ d.created_at|date:"d M" }}</td><td>{{ d.account.client.name }}
</td><td>{{ d.get_method_display }}</td><td>{{ d.amount|floatformat:2 }}</td>
</tr>
  {% empty %}<tr><td colspan="4">No deposits this month.</td></tr>{% endfor
%}
</table>
{% endblock content %}
```

36.

```
from django.db import connection
from django.test import Client as HttpClient
from django.test.utils import CaptureQueriesContext, setup_test_environment

setup_test_environment()
http = HttpClient()
for page in (1, 3):
    with CaptureQueriesContext(connection) as ctx:
        http.get("/leads/", {"page": page})
    print(page, len(ctx.captured_queries))     # same number: COUNT + page
rows + tags prefetch
```

Tests (37–40)

```
# leads/tests.py
import re
from datetime import timedelta


class ListDetailPaginationTests(TestCase):
    @classmethod
    def setUpTestData(cls):
```

```
        Lead.objects.bulk_create([
            Lead(name=f"Page Lead {i}", phone=f"9715099{i:05d}") for i in
range(60)
        ])

    def test_first_page(self):                                          # 37
        response = self.client.get(reverse("leads:list"))
        self.assertEqual(len(response.context["leads"]), 25)
        self.assertTrue(response.context["page_obj"].has_next())

    def test_next_link_keeps_query(self):                               # 38
        response = self.client.get(reverse("leads:list"), {"q": "page lead"})
        self.assertRegex(response.content.decode(),
r"q=page\+lead(&amp;|&)page=2")

    def test_older_newer_links(self):                                   # 39
        leads = list(Lead.objects.order_by("created_at", "pk"))
        middle = leads[len(leads) // 2]
        response = self.client.get(middle.get_absolute_url())
        self.assertEqual(response.context["older_lead"],
middle.get_previous_by_created_at(is_active=True))
        self.assertEqual(response.context["newer_lead"],
middle.get_next_by_created_at(is_active=True))


class CallFeedTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        agent = get_user_model().objects.create_user("feed_agent",
password="x", role="agent")
        lead = Lead.objects.create(name="Feed", phone="971509990401")
        base = timezone.now() - timedelta(days=1)
        CallLog.objects.bulk_create([
            CallLog(lead=lead, agent=agent, outcome="busy", started_at=base +
timedelta(minutes=i))
            for i in range(120)
        ])

    def test_keyset_pages_do_not_overlap(self):                         # 40
        first = self.client.get(reverse("leads:call_feed")).json()
        second = self.client.get(reverse("leads:call_feed"), {"after":
first["next"]}).json()  # dict → URL-encoded
        ids1 = {r["id"] for r in first["results"]}
```

```
        ids2 = {r["id"] for r in second["results"]}
        self.assertEqual(len(ids1), 50)
        self.assertEqual(len(ids2), 50)
        self.assertFalse(ids1 & ids2)
```

The Day 7 sort test now sees one page: compare against `expected[:25]`.

```
python manage.py test
git add -A && git commit -m "day 12"
```

Day 13 — Forms and validation

Goal: every input into the CRM goes through a Django form — normalised, validated, with friendly errors — and the lead list filters run off a form instead of raw `request.GET`.

Files: `leads/forms.py`, `leads/validators.py`, `clients/forms.py`. Do A in the shell.

A. Drills (1–10) — `forms.Form`

- [ ] **1.** `LeadSearchForm` (q, status, source — all optional, blank choice first). Print it, `as_div()`, and one field.
- [ ] **2.** Bind it to data: `is_bound`, `is_valid()`, `cleaned_data` (note the stripped `q`).
- [ ] **3.** `FieldTypesDemoForm` with CharField, EmailField, IntegerField (0–10), DecimalField, DateField, BooleanField, ChoiceField, MultipleChoiceField.
- [ ] **4.** Invalid data: read `errors`, `errors.as_json()`, one field's errors.
- [ ] **5.** Field options: `label`, `help_text`, `initial`, widget `attrs` (placeholder, type="date").
- [ ] **6.** `ContactForm.clean_phone()` normalising `+971 50 ...` to `9715...`.
- [ ] **7.** `DateRangeForm.clean()`: start must not be after end.
- [ ] **8.** Attach that error to `date_to` with `add_error()`.
- [ ] **9.** Reusable validator `validate_uae_mobile` with an error `code` and `params`.
- [ ] **10.** `has_changed()` and `changed_data` against `initial`.

B. Core (11–20) — `ModelForm`

- [ ] **11.** `LeadForm(ModelForm)` with an explicit field list: name, phone, email, source, assigned_to, expected_deposit, tags, notes.
- [ ] **12.** `Meta.widgets`, `labels`, `help_texts` (tags as checkboxes, notes 3 rows).
- [ ] **13.** Why never `fields = "__all__"` on user-facing forms — list what it would expose.
- [ ] **14.** Custom message for the duplicate-phone error via `Meta.error_messages`.
- [ ] **15.** `LeadStatusForm` (status + notes) — watch `Lead.clean()` (lost needs notes) fire through the form.
- [ ] **16.** `LeadForm.clean_phone()` normalising before the uniqueness check runs.
- [ ] **17.** `save(commit=False)` → set `assigned_to` → `save()` → `save_m2m()`.
- [ ] **18.** Edit with `instance=`: the unique check skips the row itself.
- [ ] **19.** In `__init__`, limit the agent dropdown to active agents and make it optional.

- [ ] **20.** `__init__(..., user=None)`: agents don't see `expected_deposit` or `assigned_to`.

C. Nexora CRM (21–30)

- [ ] **21.** Function view `lead_create` (GET empty form / POST validate → save → redirect) at `/leads/new/`.
- [ ] **22.** `leads/lead_form.html`: CSRF token, non-field errors, hidden fields, each visible field with `as_field_group`.
- [ ] **23.** Error styling: red border and message on fields with errors.
- [ ] **24.** `lead_edit` at `/leads/<id>/edit/` reusing the same form and template.
- [ ] **25.** `LogCallForm` (outcome, talk seconds, notes; talk time only for answered calls) shown on lead detail and posted to `/leads/<id>/log-call/`, saved with Day 10 `log_call()`.
- [ ] **26.** `DepositForm`: amount > 0, ⩽ 1,000,000, crypto capped at 50,000 (SOP rule).
- [ ] **27.** `ConvertLeadForm` (platform, currency, leverage) + `/leads/<id>/convert/` calling `convert_lead_to_client` (add a `leverage` argument to it).
- [ ] **28.** `LeadFilterForm` (q, status, source, agent, tag, finder, deposit range, date range, sort) with cross-field checks; `LeadQueryMixin` reads `cleaned_data`.
- [ ] **29.** Render the filter form above the lead list (GET, values kept, Reset link).
- [ ] **30.** Quick-add 5 leads at once with `formset_factory`; reject the same phone twice in one batch.

D. Challenge (31–40)

- [ ] **31.** `inlineformset_factory(Client, TradingAccount)` page to edit a client's accounts.
- [ ] **32.** Custom field `UAEPhoneField(forms.CharField)`: `to_python` normalises, validator checks.
- [ ] **33.** Custom widget `ColorInput` (`input_type = "color"`) in a `TagForm`.
- [ ] **34.** Dynamic form: `TagPickerForm` builds one checkbox per tag in `__init__`.
- [ ] **35.** `clean_email()`: no other active lead with the same email (case-insensitive), excluding itself.
- [ ] **36.** `Meta.field_classes` so `LeadForm.phone` uses `UAEPhoneField` automatically (replaces 16).
- [ ] **37.** Test: `LeadForm` accepts `+971 50 123 9999` and saves `971501239999`.
- [ ] **38.** Test: duplicate phone shows your custom message.

- [ ] **39.** Test: `DateRangeForm` rejects from > to.
- [ ] **40.** Test: `DepositForm` crypto cap error.

Solutions

Validation order for one form: each field's `clean()` (to_python → validate → validators) → `clean_<field>()` → form `clean()` → for a ModelForm, the model's `full_clean()` (field validators, `Model.clean()`, unique and constraint checks).

Validator + custom field (9, 32)

```
# leads/validators.py
import re
from django.core.exceptions import ValidationError

UAE_MOBILE = re.compile(r"^9715\d{8}$")


def validate_uae_mobile(value):
    if not UAE_MOBILE.match(value or ""):
        raise ValidationError(
            "%(value)s is not a UAE mobile (9715XXXXXXXX).",
            code="invalid_uae_mobile",
            params={"value": value},
        )
```

```
# leads/fields.py
from django import forms
from .models import normalize_uae_phone
from .validators import validate_uae_mobile


class UAEPhoneField(forms.CharField):
    default_validators = [validate_uae_mobile]

    def __init__(self, **kwargs):
        kwargs["max_length"] = 20          # room for "+971 50 123 4567"
before normalising
        kwargs.setdefault("widget", forms.TextInput(attrs={"placeholder":
"+971 50 123 4567", "inputmode": "tel"}))
        super().__init__(**kwargs)

    def to_python(self, value):
        value = super().to_python(value)   # strips whitespace
        return normalize_uae_phone(value) if value else value
```

Final `leads/forms.py` (1–8, 10–20, 25, 28, 30, 33–36)

```
# leads/forms.py
from django import forms
from django.contrib.auth import get_user_model
from django.core.exceptions import ValidationError

from clients.models import TradingAccount
from .fields import UAEPhoneField
from .models import CallLog, Lead, Tag, normalize_uae_phone
from .queries import FINDERS
from .validators import validate_uae_mobile

DATE = forms.DateInput(attrs={"type": "date"})


# ---------- A: plain forms ----------------------------------------------
---
class LeadSearchForm(forms.Form):
# 1, 5
    q = forms.CharField(required=False, label="Search",
                        widget=forms.TextInput(attrs={"placeholder": "Name,
email or phone"}))
    status = forms.ChoiceField(required=False, choices=[("", "Any status"),
*Lead.Status.choices])
```

```
    source = forms.ChoiceField(required=False, choices=[("", "Any source"),
*Lead.Source.choices])


class FieldTypesDemoForm(forms.Form):
# 3
    name = forms.CharField(max_length=50, help_text="Full name as on ID")
    email = forms.EmailField()
    attempts = forms.IntegerField(min_value=0, max_value=10, initial=0)
    deposit = forms.DecimalField(max_digits=12, decimal_places=2,
min_value=0)
    callback_on = forms.DateField(widget=DATE)
    vip = forms.BooleanField(required=False)
    source = forms.ChoiceField(choices=Lead.Source.choices)
    channels = forms.MultipleChoiceField(
        choices=[("call", "Call"), ("sms", "SMS"), ("email", "Email")],
        widget=forms.CheckboxSelectMultiple,
    )


class ContactForm(forms.Form):
# 6
    name = forms.CharField(max_length=120)
    phone = forms.CharField(max_length=20)

    def clean_phone(self):
        phone = normalize_uae_phone(self.cleaned_data["phone"])
        validate_uae_mobile(phone)
# 9
        return phone                        # the returned value replaces
cleaned_data["phone"]


class DateRangeForm(forms.Form):
# 7-8
    date_from = forms.DateField(required=False, widget=DATE)
    date_to = forms.DateField(required=False, widget=DATE)

    def clean(self):
        cleaned = super().clean()
        start, end = cleaned.get("date_from"), cleaned.get("date_to")   #
.get: a field may have failed
        if start and end and start > end:
            self.add_error("date_to", "End date must be on or after the start
date.")
            # non-field version: raise ValidationError("...")
```

```
        return cleaned


# ---------- B: model forms ----------------------------------------------
---
class LeadForm(forms.ModelForm):
    class Meta:
        model = Lead
        fields = ["name", "phone", "email", "source", "assigned_to",
# 11, 13
                  "expected_deposit", "tags", "notes"]
        field_classes = {"phone": UAEPhoneField}
# 36
        widgets = {
# 12
            "notes": forms.Textarea(attrs={"rows": 3}),
            "expected_deposit": forms.NumberInput(attrs={"step": "500",
"min": "0"}),
            "tags": forms.CheckboxSelectMultiple,
        }
        labels = {"expected_deposit": "Expected first deposit (USD)"}
        help_texts = {"email": "Optional — used for KYC and statements."}
        error_messages = {"phone": {"unique": "A lead with this phone number
already exists."}}   # 14

    def __init__(self, *args, user=None, **kwargs):
# 19-20
        super().__init__(*args, **kwargs)
        self.fields["assigned_to"].queryset =
get_user_model().objects.filter(role="agent", is_active=True)
        self.fields["assigned_to"].required = False
        if user is not None and getattr(user, "role", "") == "agent":
            del self.fields["expected_deposit"]
            del self.fields["assigned_to"]

    def clean_name(self):
        return self.cleaned_data["name"].strip().title()

    def clean_email(self):
# 35
        email = self.cleaned_data.get("email", "").strip().lower()
        if email:
            clash = Lead.objects.filter(email__iexact=email, is_active=True)
            if self.instance.pk:
                clash = clash.exclude(pk=self.instance.pk)
            if clash.exists():
```

```
                raise ValidationError("Another active lead already uses this
email.")
        return email

    def clean_expected_deposit(self):
        value = self.cleaned_data.get("expected_deposit")
        if value is not None and value < 0:
            raise ValidationError("Deposit can't be negative.")
        return value


class LeadStatusForm(forms.ModelForm):
# 15
    class Meta:
        model = Lead
        fields = ["status", "notes"]
        widgets = {"notes": forms.Textarea(attrs={"rows": 2})}
    # Lead.clean() runs during is_valid(): "lost" without notes → error on
the notes field


class LogCallForm(forms.ModelForm):
# 25
    class Meta:
        model = CallLog
        fields = ["outcome", "duration_seconds", "notes"]
        labels = {"duration_seconds": "Talk time (seconds)"}
        widgets = {"notes": forms.Textarea(attrs={"rows": 2})}

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["duration_seconds"].required = False

    def clean_duration_seconds(self):
        return self.cleaned_data.get("duration_seconds") or 0

    def clean(self):
        cleaned = super().clean()
        if cleaned.get("outcome") != CallLog.Outcome.ANSWERED and
cleaned.get("duration_seconds"):
            self.add_error("duration_seconds", "Only answered calls have talk
time.")
        return cleaned


class ConvertLeadForm(forms.Form):
```

```
# 27
    platform = forms.ChoiceField(choices=TradingAccount.Platform.choices,
initial="mt5")
    currency = forms.ChoiceField(choices=TradingAccount.Currency.choices,
initial="USD")
    leverage = forms.TypedChoiceField(
        choices=[(50, "1:50"), (100, "1:100"), (200, "1:200"), (500,
"1:500")],
        coerce=int, initial=100,
    )

    def __init__(self, *args, lead=None, **kwargs):
        self.lead = lead
        super().__init__(*args, **kwargs)

    def clean(self):
        cleaned = super().clean()
        if self.lead is not None and hasattr(self.lead, "client"):
            raise ValidationError("This lead is already a client.")
        return cleaned


# ---------- C: filter form ----------------------------------------------
---
class LeadFilterForm(forms.Form):
# 28
    SORT_CHOICES = [
        ("-created_at", "Newest"), ("created_at", "Oldest"), ("name", "Name
A–Z"),
        ("-expected_deposit", "Deposit high → low"), ("-call_count", "Most
calls"),
        ("-last_call_at", "Recently called"),
    ]
    q = forms.CharField(required=False, label="Search")
    status = forms.ChoiceField(required=False, choices=[("", "Any status"),
*Lead.Status.choices])
    source = forms.ChoiceField(required=False, choices=[("", "Any source"),
*Lead.Source.choices])
    agent = forms.ModelChoiceField(required=False, empty_label="Any agent",

queryset=get_user_model().objects.filter(role="agent"))
    tag = forms.ModelChoiceField(required=False, empty_label="Any tag",
                                 queryset=Tag.objects.all(),
to_field_name="name")   # ?tag=vip keeps working
    finder = forms.ChoiceField(required=False,
                               choices=[("", "—"), *[(k, k.replace("_", "
```

```
").title()) for k in FINDERS]])
    min_deposit = forms.DecimalField(required=False, min_value=0)
    max_deposit = forms.DecimalField(required=False, min_value=0)
    created_from = forms.DateField(required=False, widget=DATE)
    created_to = forms.DateField(required=False, widget=DATE)
    sort = forms.ChoiceField(required=False, choices=SORT_CHOICES)

    def clean(self):
        cleaned = super().clean()
        lo, hi = cleaned.get("min_deposit"), cleaned.get("max_deposit")
        if lo is not None and hi is not None and lo > hi:
            self.add_error("max_deposit", "Max must be at least the min.")
        start, end = cleaned.get("created_from"), cleaned.get("created_to")
        if start and end and start > end:
            self.add_error("created_to", "End date must be on or after the
start date.")
        return cleaned


# ---------- C/D: formset, tag forms ----------------------------------------
-----
class QuickLeadForm(forms.Form):
# 30
    name = forms.CharField(max_length=120)
    phone = UAEPhoneField()
    source = forms.ChoiceField(choices=Lead.Source.choices,
initial=Lead.Source.MANUAL)


class BaseQuickLeadFormSet(forms.BaseFormSet):
    def clean(self):
        if any(self.errors):
            return
        phones = [f.cleaned_data["phone"] for f in self.forms if
f.cleaned_data]
        if len(phones) != len(set(phones)):
            raise ValidationError("The same phone appears twice in this
batch.")   # → non_form_errors


QuickLeadFormSet = forms.formset_factory(QuickLeadForm,
formset=BaseQuickLeadFormSet, extra=5)


class ColorInput(forms.TextInput):
# 33
```

```
    input_type = "color"


class TagForm(forms.ModelForm):
    class Meta:
        model = Tag
        fields = ["name", "color"]
        widgets = {"color": ColorInput}


class TagPickerForm(forms.Form):
# 34
    def __init__(self, *args, lead=None, **kwargs):
        super().__init__(*args, **kwargs)
        current = set(lead.tags.values_list("pk", flat=True)) if lead else
set()
        for tag in Tag.objects.all():
            self.fields[f"tag_{tag.pk}"] = forms.BooleanField(
                required=False, label=tag.name, initial=tag.pk in current,
            )

    def selected_tag_ids(self):
        return [int(name.removeprefix("tag_")) for name, on in
self.cleaned_data.items() if on]

# usage: form = TagPickerForm(request.POST, lead=lead); form.is_valid();
lead.tags.set(form.selected_tag_ids())
```

16 (before 36 replaced it):

```
# inside LeadForm
def clean_phone(self):
    phone = normalize_uae_phone(self.cleaned_data["phone"])
    validate_uae_mobile(phone)
    return phone          # the ModelForm's unique check then runs on the
normalised value
```

13. `"__all__"` would expose `status`, `is_active`, `contact_attempts`, `last_contacted_at`, `country` — any field you add later appears in the form automatically. Always list fields.

A + B in the shell (1–4, 10, 17–18)

```
# python manage.py shell
from accounts.models import CustomUser
from leads.forms import FieldTypesDemoForm, LeadForm, LeadSearchForm
from leads.models import Lead, Tag

form = LeadSearchForm()                                          # 1
print(form)                          # Django 5+: <div>-based rendering
print(form.as_div())
print(form["q"])

bound = LeadSearchForm({"q": "  khan ", "status": "new"})  # 2
bound.is_bound, bound.is_valid(), bound.cleaned_data
# (True, True, {'q': 'khan', 'status': 'new', 'source': ''})

FieldTypesDemoForm({"name": "Ali", "email": "ali@x.test", "attempts": 3,
"deposit": "2500.50",   # 3
                    "callback_on": "2026-10-20", "source": "google",
"channels": ["call", "sms"]}).is_valid()

bad = LeadSearchForm({"status": "won"})                          # 4
bad.is_valid()                       # False
bad.errors                           # {'status': ['Select a valid choice. won
is not one of ...']}
bad.errors.as_json()
bad["status"].errors

f = LeadSearchForm({"q": "khan", "status": "new", "source": ""}, initial=
{"status": "new"})   # 10
f.has_changed(), f.changed_data     # (True, ['q'])

vip, _ = Tag.objects.get_or_create(name="vip")                   # 17
form = LeadForm({"name": "zara ahmed", "phone": "+971 50 777 0101", "source":
"google", "tags": [vip.pk]})
form.is_valid()
lead = form.save(commit=False)      # instance built, nothing written
lead.assigned_to = CustomUser.objects.filter(role="agent").first()
lead.save()
form.save_m2m()                      # tags need the saved pk — only after
commit=False

form = LeadForm({"name": lead.name, "phone": lead.phone, "source":
lead.source}, instance=lead)   # 18
form.is_valid()                      # True — unique check excludes this row
```

C. Views and templates

21, 24–25, 27, 30.

```
# leads/views.py
from django.shortcuts import get_object_or_404, redirect, render
from django.views.decorators.http import require_POST

from clients.services import convert_lead_to_client
from .forms import ConvertLeadForm, LeadForm, LogCallForm, QuickLeadFormSet
from .services import log_call, upsert_leads


def lead_create(request):
# 21
    if request.method == "POST":
        form = LeadForm(request.POST, user=request.user)
        if form.is_valid():
            lead = form.save()
            return redirect(lead)          # → lead.get_absolute_url()
    else:
        form = LeadForm(user=request.user)
    return render(request, "leads/lead_form.html", {"form": form,
"page_title": "New lead"})


def lead_edit(request, lead_id):
# 24
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = LeadForm(request.POST or None, instance=lead, user=request.user)
    if request.method == "POST" and form.is_valid():
        form.save()
        return redirect(lead)
    return render(request, "leads/lead_form.html", {"form": form,
"page_title": f"Edit {lead.name}"})


@require_POST
def log_call_view(request, lead_id):
# 25
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = LogCallForm(request.POST)
    if form.is_valid():
        agent = request.user if request.user.is_authenticated else
lead.assigned_to   # Day 15: request.user only
        log_call(lead, agent, **form.cleaned_data)
        return redirect(lead)
```

```
    return render(request, "leads/log_call.html", {"form": form, "lead":
lead})


def convert_lead(request, lead_id):
# 27
    lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
    form = ConvertLeadForm(request.POST or None, lead=lead)
    if request.method == "POST" and form.is_valid():
        client = convert_lead_to_client(lead.pk, **form.cleaned_data)
        return redirect(client)
    return render(request, "leads/convert.html", {"form": form, "lead":
lead})


def quick_add_many(request):
# 30
    formset = QuickLeadFormSet(request.POST or None)
    if request.method == "POST" and formset.is_valid():
        rows = [f.cleaned_data for f in formset if f.cleaned_data]   #
untouched extra rows are empty
        upsert_leads(rows)
        return redirect("leads:list")
    return render(request, "leads/quick_add_many.html", {"formset": formset})
```

```
# clients/services.py — 27: accept leverage
def convert_lead_to_client(lead_id, platform="mt5", currency="USD",
leverage=100):
    ...
    TradingAccount.objects.create(client=client,
number=next_account_number(),
                                  platform=platform, currency=currency,
leverage=leverage)

# leads/views.py — LeadDetailView.get_context_data (25)
context["log_call_form"] = LogCallForm()

# leads/urls.py
path("new/", views.lead_create, name="new"),
path("quick-add-many/", views.quick_add_many, name="quick_add_many"),
path("<int:lead_id>/edit/", views.lead_edit, name="edit"),
path("<int:lead_id>/log-call/", views.log_call_view, name="log_call"),
path("<int:lead_id>/convert/", views.convert_lead, name="convert"),
```

`request.POST or None` is unbound on GET; a POST with no fields would also look unbound, which is fine for these forms (24).

22–23.

```
<!-- leads/templates/leads/lead_form.html (also used by convert.html / log_call.html with other headings) -->
{% extends "base.html" %}
{% block content %}
<h1>{{ page_title }}</h1>
<form method="post" novalidate>      {# novalidate: let Django's errors show while you practise #}
  {% csrf_token %}
  {{ form.non_field_errors }}
  {% for hidden in form.hidden_fields %}{{ hidden }}{{ hidden.errors }}{% endfor %}
  {% for field in form.visible_fields %}
    <div class="field{% if field.errors %} has-error{% endif %}">
      {{ field.as_field_group }}     {# label + help text + errors + widget (Django 5.0+) #}
    </div>
  {% endfor %}
  <button type="submit">Save</button>
  <a href="{% url 'leads:list' %}">Cancel</a>
</form>
{% endblock content %}
```

```
<!-- the long form of as_field_group, if you want full control -->
<label for="{{ field.id_for_label }}">{{ field.label }}</label>
{{ field }}
{% if field.help_text %}<small class="muted">{{ field.help_text }}</small>{% endif %}
{{ field.errors }}
```

```
/* static/css/crm.css — 23 */
.field { margin-bottom: .75rem; display: grid; gap: .25rem; }
.has-error input, .has-error select, .has-error textarea { border: 1px solid #dc2626; }
.errorlist { color: #dc2626; list-style: none; padding: 0; margin: 0; font-size: .875rem; }
```

```
<!-- leads/lead_detail.html — 25 -->
<h2>Log a call</h2>
<form method="post" action="{% url 'leads:log_call' lead.pk %}">
  {% csrf_token %}
  {{ log_call_form.as_div }}
  <button type="submit">Log call</button>
</form>
```

```
<!-- leads/templates/leads/quick_add_many.html — 30 -->
{% extends "base.html" %}
{% block content %}
<h1>Quick add</h1>
<form method="post">
  {% csrf_token %}
  {{ formset.management_form }}     {# TOTAL_FORMS etc. — required #}
  {{ formset.non_form_errors }}
  <table class="table">
    <tr><th>Name</th><th>Phone</th><th>Source</th></tr>
    {% for form in formset %}
      <tr>
        <td>{{ form.name }}{{ form.name.errors }}</td>
        <td>{{ form.phone }}{{ form.phone.errors }}</td>
        <td>{{ form.source }}</td>
      </tr>
    {% endfor %}
  </table>
  <button type="submit">Add leads</button>
</form>
{% endblock content %}
```

26.

```
# clients/forms.py
from decimal import Decimal

from django import forms

from .models import Client, Deposit, TradingAccount

CRYPTO_CAP = Decimal("50000")


class DepositForm(forms.ModelForm):
    class Meta:
        model = Deposit
        fields = ["amount", "method"]

    def clean_amount(self):
        amount = self.cleaned_data["amount"]
        if amount <= 0:
            raise forms.ValidationError("Amount must be positive.")
        if amount > Decimal("1000000"):
            raise forms.ValidationError("Amounts over 1,000,000 need manual
approval.")
```

```
        return amount

    def clean(self):
        cleaned = super().clean()
        if cleaned.get("method") == Deposit.Method.CRYPTO and
cleaned.get("amount", 0) > CRYPTO_CAP:
            self.add_error("amount", f"Crypto deposits are capped at
{CRYPTO_CAP:,.0f}.")
        return cleaned


TradingAccountFormSet = forms.inlineformset_factory(
# 31
    Client, TradingAccount,
    fields=["number", "platform", "currency", "leverage"],
    extra=1, can_delete=False,
)
```

28–29.

```
# leads/views.py — LeadQueryMixin now reads the form
from django.db.models import Count, F, Max
from .forms import LeadFilterForm


class LeadQueryMixin:
    def get_filter_form(self):
        if not hasattr(self, "_filter_form"):
            self._filter_form = LeadFilterForm(self.request.GET or None)
        return self._filter_form

    def get_leads(self):
        qs = (Lead.objects.filter(is_active=True)
              .select_related("assigned_to", "broker")
              .prefetch_related("tags")
              .annotate(call_count=Count("calls", distinct=True),
last_call_at=Max("calls__started_at")))
        form = self.get_filter_form()
        form.is_valid()
        data = getattr(form, "cleaned_data", {})     # valid fields only;
invalid ones are left out
        if q := data.get("q"):
            qs = qs.filter(search_q(q))
        if status := data.get("status"):
            qs = qs.filter(status=status)
        if source := data.get("source"):
```

```
            qs = qs.filter(source=source)
        if agent := data.get("agent"):
            qs = qs.filter(assigned_to=agent)
        if tag := data.get("tag"):
            qs = qs.filter(tags=tag)
        if finder := data.get("finder"):
            qs = qs.filter(pk__in=FINDERS[finder]().values("pk"))
        if data.get("min_deposit") is not None:
            qs = qs.filter(expected_deposit__gte=data["min_deposit"])
        if data.get("max_deposit") is not None:
            qs = qs.filter(expected_deposit__lte=data["max_deposit"])
        if data.get("created_from"):
            qs = qs.filter(created_at__date__gte=data["created_from"])
        if data.get("created_to"):
            qs = qs.filter(created_at__date__lte=data["created_to"])
        sort = data.get("sort") or "-created_at"
        field = sort.lstrip("-")
        order = F(field).desc(nulls_last=True) if sort.startswith("-") else
F(field).asc(nulls_last=True)
        return qs.order_by(order, "-pk")


# LeadListView — expose the form
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["filter_form"] = self.get_filter_form()
    return context
```

```
<!-- leads/lead_list.html — 29 (replaces the Day 3 search box) -->
<form method="get" class="filters">
  {% for field in filter_form %}{{ field.as_field_group }}{% endfor %}
  <button type="submit">Apply</button>
  <a href="{% url 'leads:list' %}">Reset</a>
</form>
```

```
.filters { display: grid; grid-template-columns: repeat(auto-fill,
minmax(160px, 1fr)); gap: .75rem; align-items: end; }
```

D. Challenge

31.

```
# clients/views.py
from .forms import TradingAccountFormSet


def client_accounts(request, ref):
    client = get_object_or_404(Client, ref=ref)
    formset = TradingAccountFormSet(request.POST or None, instance=client)
    if request.method == "POST" and formset.is_valid():
        formset.save()                    # sets account.client for new rows
        return redirect(client)
    return render(request, "clients/client_accounts.html", {"client": client,
"formset": formset})

# clients/urls.py
path("<uuid:ref>/accounts/", views.client_accounts, name="accounts"),
```

```
<!-- clients/templates/clients/client_accounts.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ client.name }} · trading accounts</h1>
<form method="post">{% csrf_token %}
  {{ formset.management_form }}
  {% for form in formset %}<fieldset>{{ form.as_div }}</fieldset>{% endfor %}
  <button type="submit">Save accounts</button>
</form>
{% endblock content %}
```

Tests (37–40)

```
# leads/tests.py
from clients.forms import DepositForm
from .forms import DateRangeForm, LeadForm


class FormTests(TestCase):
    def test_leadform_normalises_phone(self):                        # 37
        form = LeadForm({"name": "zara ahmed", "phone": "+971 50 123 9999",
"source": "google"})
        self.assertTrue(form.is_valid(), form.errors)
        lead = form.save()
        self.assertEqual(lead.phone, "971501239999")
        self.assertEqual(lead.name, "Zara Ahmed")

    def test_duplicate_phone_message(self):                          # 38
        Lead.objects.create(name="First", phone="971501239998")
```

```
        form = LeadForm({"name": "Second", "phone": "0501239998", "source":
"google"})
        self.assertFalse(form.is_valid())
        self.assertIn("already exists", form.errors["phone"][0])

    def test_date_range(self):                                          # 39
        form = DateRangeForm({"date_from": "2026-10-10", "date_to": "2026-10-
01"})
        self.assertFalse(form.is_valid())
        self.assertIn("date_to", form.errors)

    def test_crypto_cap(self):                                          # 40
        form = DepositForm({"amount": "60000", "method": "crypto"})
        self.assertFalse(form.is_valid())
        self.assertIn("capped", form.errors["amount"][0])
```

Day 14 — Editing views and messages

Goal: rebuild Day 13's function views as generic editing views, give every action feedback through the messages framework, and add the CRM's day-to-day write actions: status changes, conversion, deposits, bulk updates, archive with undo, and an audit trail.

URL names stay the same as Day 13 (`new`, `edit`, `log_call`, `convert`), so templates keep working.

A. Drills (1–10) — the editing views

- [ ] **1.** `LeadCreateView(CreateView)` with `form_class = LeadForm`; redirect to the lead (via `get_absolute_url`).
- [ ] **2.** `LeadUpdateView(UpdateView)` with `pk_url_kwarg`, same template, active leads only.
- [ ] **3.** `LeadConfirmDeleteView(DeleteView)` with a confirm page at `/leads/<id>/delete/confirm/`.
- [ ] **4.** `UserFormKwargsMixin.get_form_kwargs()` passing `user=request.user` to the form.
- [ ] **5.** `form_valid()`: an agent creating a lead with no assignee gets it.
- [ ] **6.** `form_invalid()`: add an error message, then return the normal response.
- [ ] **7.** Messages block in `base.html` (CSS per level).
- [ ] **8.** `SuccessMessageMixin`: "Lead Zara Ahmed created."
- [ ] **9.** "Save and add another" button via `get_success_url()`.
- [ ] **10.** `LogCallView(FormView)` replacing Day 13's `log_call_view`.

B. Core (11–20)

- [ ] **11.** `LeadStatusView(UpdateView)` with `LeadStatusForm`; message shows the new status label.
- [ ] **12.** `ConvertLeadView(FormView)`: load the lead in `setup()`, call the service, turn `ValueError` into a form error.
- [ ] **13.** `ClientUpdateView` editing the client and its trading accounts (inline formset) atomically.
- [ ] **14.** `DepositCreateView` at `/clients/accounts/<number>/deposits/new/`; the account comes from the URL.
- [ ] **15.** Full Tag CRUD with four generic views and a shared mixin.

- [ ] **16.** `LeadArchiveView`: POST-only soft delete.
- [ ] **17.** `MESSAGE_TAGS` (error → `danger`) and `extra_tags`.
- [ ] **18.** Info message on lead detail when a new lead has waited 7+ days without a call.
- [ ] **19.** Delete confirm page lists what goes with the lead (call count, client record).
- [ ] **20.** `ClientDeleteView` catches `ProtectedError` (clients with deposits) and shows an error instead of a 500.

C. Nexora CRM (21–30)

- [ ] **21.** Action bar on lead detail: Edit, Change status, Convert (hidden if already a client), Archive.
- [ ] **22.** Bulk actions on the lead list: checkboxes + "mark contacted / assign to agent / archive" (`LeadBulkForm` + `LeadBulkActionView`).
- [ ] **23.** Reassign a single lead from its detail page by reusing the bulk view.
- [ ] **24.** `get_initial()`: `/leads/new/?source=referral&agent=3` pre-fills the form.
- [ ] **25.** Duplicate phone on create → redirect to the existing lead with a warning instead of an error.
- [ ] **26.** Every bulk action reports how many rows changed.
- [ ] **27.** Edit conflict guard: hidden `loaded_at`; reject the save if the lead changed since the form was opened.
- [ ] **28.** Log-call form on detail posts to `LogCallView` and comes back with a success message (Post/Redirect/Get).
- [ ] **29.** Saving a deposit also adds it to `TradingAccount.balance` with `F()`, in the same transaction.
- [ ] **30.** Archive message with an Undo link to a restore page (`LeadRestoreView`, POST to restore).

D. Challenge (31–40)

- [ ] **31.** `ActivityLog` model with a `GenericForeignKey` + `AuditMixin` that records create/update with changed fields; show the last 10 on lead detail.
- [ ] **32.** `get_form_class()` returning a slimmer `modelform_factory` form for agents.
- [ ] **33.** Two forms on one page with `prefix` (edit lead + log call) and one `post()` that tells them apart.
- [ ] **34.** `success_url` with a format string: `"/leads/tags/?created={id}"`.
- [ ] **35.** `JsonFormMixin`: the same create view answers JSON when `Accept: application/json`.

- [ ] **36.** `safe_next()` helper using `url_has_allowed_host_and_scheme`; use it for `?next=` redirects.
- [ ] **37.** Test: create view redirects to the new lead and shows the success message.
- [ ] **38.** Test: a stale `loaded_at` is rejected.
- [ ] **39.** Test: deleting a client with deposits keeps the row and shows the error message.
- [ ] **40.** Test: bulk "mark contacted" updates both selected leads.

Solutions

Generic editing flow: GET → `get_form()` → render. POST → `get_form()` → `is_valid()` → `form_valid()` (save + redirect to `get_success_url()`) or `form_invalid()` (re-render with errors). Override those hooks, always calling `super()`.

Helpers (4, 31, 36)

```
# core/utils.py — 36
from django.utils.http import url_has_allowed_host_and_scheme


def safe_next(request, default="/"):
    nxt = request.POST.get("next") or request.GET.get("next")
    if nxt and url_has_allowed_host_and_scheme(
        nxt, allowed_hosts={request.get_host()},
require_https=request.is_secure(),
    ):
        return nxt
    return default          # never redirect to another site from user input
```

```
# core/models.py — 31
from django.conf import settings
from django.contrib.contenttypes.fields import GenericForeignKey
from django.contrib.contenttypes.models import ContentType
from django.db import models


class ActivityLog(models.Model):
    actor = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                              on_delete=models.SET_NULL,
related_name="activity")
    verb = models.CharField(max_length=20)                    # created /
updated / deleted
    content_type = models.ForeignKey(ContentType, on_delete=models.CASCADE)
```

```
    object_id = models.CharField(max_length=64)                # str: works
for int and UUID pks
    target = GenericForeignKey("content_type", "object_id")      # the object
itself, any model
    object_repr = models.CharField(max_length=200)
    changes = models.JSONField(default=dict, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["content_type", "object_id"],
name="activity_target_idx")]

    def __str__(self):
        return f"{self.actor or 'system'} {self.verb} {self.object_repr}"
```

```
# core/mixins.py — 4, 31 (add)
from django.contrib.contenttypes.models import ContentType
from core.models import ActivityLog


class UserFormKwargsMixin:                                          # 4
    def get_form_kwargs(self):
        kwargs = super().get_form_kwargs()
        kwargs["user"] = self.request.user
        return kwargs


class AuditMixin:                                                   # 31 — for
Create/UpdateView
    audit_verb = "updated"

    def form_valid(self, form):
        response = super().form_valid(form)                         # self.object
is saved now
        ActivityLog.objects.create(
            actor=self.request.user if self.request.user.is_authenticated
else None,
            verb=self.audit_verb,
            content_type=ContentType.objects.get_for_model(self.object),
            object_id=str(self.object.pk),
            object_repr=str(self.object)[:200],
```

```
            changes={name: str(form.cleaned_data.get(name)) for name in
form.changed_data},
        )
        return response
```

```
python manage.py makemigrations core && python manage.py migrate
```

Forms added today (22, 27)

```
# leads/forms.py (add)
from django.utils import timezone


class LeadEditForm(LeadForm):                                    # 27
    loaded_at = forms.DateTimeField(widget=forms.HiddenInput)

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        if self.instance.pk and not self.is_bound:
            self.initial["loaded_at"] = self.instance.updated_at.isoformat()

    def clean(self):
        cleaned = super().clean()
        loaded = cleaned.get("loaded_at")
        if loaded and self.instance.pk and self.instance.updated_at > loaded:
            raise ValidationError("Someone else changed this lead after you
opened it. Reload and try again.")
        return cleaned


class LeadBulkForm(forms.Form):                                  # 22
    ACTIONS = [("contacted", "Mark contacted"), ("assign", "Assign to
agent"), ("archive", "Archive")]
    leads =
forms.ModelMultipleChoiceField(queryset=Lead.objects.filter(is_active=True),
                                           widget=forms.MultipleHiddenInput)
    action = forms.ChoiceField(choices=ACTIONS)
    agent = forms.ModelChoiceField(required=False, empty_label="— agent —",

queryset=get_user_model().objects.filter(role="agent", is_active=True))

    def clean(self):
```

```
        cleaned = super().clean()
        if cleaned.get("action") == "assign" and not cleaned.get("agent"):
            self.add_error("agent", "Pick an agent to assign to.")
        return cleaned
```

Lead views (1–12, 16, 18–19, 21–25, 28, 30–36)

```
# leads/views.py
from datetime import timedelta

from django.contrib import messages
from django.contrib.messages.views import SuccessMessageMixin
from django.forms import modelform_factory
from django.http import JsonResponse
from django.shortcuts import get_object_or_404, redirect
from django.urls import reverse, reverse_lazy
from django.utils import timezone
from django.utils.html import format_html
from django.views import View
from django.views.generic import TemplateView
from django.views.generic.edit import CreateView, DeleteView, FormView,
UpdateView

from clients.services import convert_lead_to_client
from core.mixins import AuditMixin, PageTitleMixin, UserFormKwargsMixin
from core.utils import safe_next
from .forms import ConvertLeadForm, LeadBulkForm, LeadEditForm, LeadForm,
LeadStatusForm, LogCallForm
from .models import Lead, normalize_uae_phone
from .services import log_call


class JsonFormMixin:                                                    #
35
    def wants_json(self):
        return "application/json" in self.request.headers.get("Accept", "")

    def form_invalid(self, form):
        if self.wants_json():
            return JsonResponse({"errors": form.errors.get_json_data()},
status=400)
        return super().form_invalid(form)

    def form_valid(self, form):
        response = super().form_valid(form)
        if self.wants_json():
```

```
            return JsonResponse({"id": self.object.pk, "url":
self.object.get_absolute_url()}, status=201)
        return response


class LeadCreateView(JsonFormMixin, AuditMixin, PageTitleMixin,
UserFormKwargsMixin,
                     SuccessMessageMixin, CreateView):
# 1, 4–6, 8–9, 24–25, 31, 35
    model = Lead
    form_class = LeadForm
    template_name = "leads/lead_form.html"
    page_title = "New lead"
    success_message = "Lead %(name)s created."        # %(field)s comes from
cleaned_data
    audit_verb = "created"

    def get_initial(self):                                                  #
24
        initial = super().get_initial()
        if source := self.request.GET.get("source"):
            initial["source"] = source
        if (agent := self.request.GET.get("agent", "")).isdigit():
            initial["assigned_to"] = agent
        return initial

    def form_valid(self, form):                                             #
5
        if form.instance.assigned_to is None and getattr(self.request.user,
"role", "") == "agent":
            form.instance.assigned_to = self.request.user
        return super().form_valid(form)

    def form_invalid(self, form):                                           #
6, 25
        if form.has_error("phone", code="unique"):
            existing = Lead.objects.filter(
                phone=normalize_uae_phone(form.data.get("phone", "")),
is_active=True,
            ).first()
            if existing:
                messages.warning(self.request, f"{existing.name} already has
this number — opened their record.")
                return redirect(existing)
        messages.error(self.request, "Please fix the errors below.")
        return super().form_invalid(form)
```

```
    def get_success_url(self):                                          #
9
        if "_addanother" in self.request.POST:
            return reverse("leads:new")
        return self.object.get_absolute_url()


class LeadUpdateView(AuditMixin, PageTitleMixin, UserFormKwargsMixin,
                     SuccessMessageMixin, UpdateView):
# 2, 27, 31
    model = Lead
    form_class = LeadEditForm
    pk_url_kwarg = "lead_id"
    template_name = "leads/lead_form.html"
    page_title = "Edit lead"
    success_message = "Saved %(name)s."

    def get_queryset(self):
        return Lead.objects.filter(is_active=True)

    def get_form_class(self):                                           #
32
        if getattr(self.request.user, "role", "") == "agent":
            return modelform_factory(Lead, form=LeadEditForm,
                                     fields=["name", "phone", "email",
"source", "tags", "notes"])
        return super().get_form_class()


class LeadStatusView(PageTitleMixin, SuccessMessageMixin, UpdateView):     #
11
    model = Lead
    form_class = LeadStatusForm
    pk_url_kwarg = "lead_id"
    template_name = "leads/lead_form.html"
    page_title = "Change status"

    def get_success_message(self, cleaned_data):
        return f"Status changed to {self.object.get_status_display()}."


class LeadConfirmDeleteView(PageTitleMixin, SuccessMessageMixin, DeleteView):
# 3, 19, 36
    model = Lead
    pk_url_kwarg = "lead_id"
```

```
    template_name = "leads/lead_confirm_delete.html"
    success_message = "Lead deleted."
    page_title = "Delete lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["call_count"] = self.object.calls.count()
        context["is_client"] = hasattr(self.object, "client")
        return context

    def get_success_url(self):
        return safe_next(self.request, default=reverse("leads:list"))


class LogCallView(SuccessMessageMixin, FormView):                               #
10, 28
    form_class = LogCallForm
    template_name = "leads/log_call.html"
    success_message = "Call logged."

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)

    def form_valid(self, form):
        agent = self.request.user if self.request.user.is_authenticated else
self.lead.assigned_to
        log_call(self.lead, agent, **form.cleaned_data)
        return super().form_valid(form)

    def get_success_url(self):
        return self.lead.get_absolute_url()

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "lead": self.lead}


class ConvertLeadView(PageTitleMixin, FormView):                                #
12
    form_class = ConvertLeadForm
    template_name = "leads/convert.html"
    page_title = "Convert to client"

    def setup(self, request, *args, **kwargs):     # runs before dispatch —
good place to load objects
```

```
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)

    def get_form_kwargs(self):
        return {**super().get_form_kwargs(), "lead": self.lead}

    def form_valid(self, form):
        try:
            client = convert_lead_to_client(self.lead.pk,
**form.cleaned_data)
        except ValueError as exc:
            form.add_error(None, str(exc))
            return self.form_invalid(form)
        messages.success(self.request, f"{self.lead.name} is now client
{client.account_number}.")
        return redirect(client)

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "lead": self.lead}


class LeadArchiveView(View):                                          #
16, 30
    http_method_names = ["post"]

    def post(self, request, lead_id):
        lead = get_object_or_404(Lead, pk=lead_id, is_active=True)
        lead.archive()
        undo = reverse("leads:restore", args=[lead.pk])
        messages.success(request, format_html('{} archived. <a href="
{}">Undo</a>', lead.name, undo))
        return redirect("leads:list")


class LeadRestoreView(PageTitleMixin, TemplateView):
# 30
    template_name = "leads/lead_restore.html"
    page_title = "Restore lead"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["lead"] = get_object_or_404(Lead, pk=self.kwargs["lead_id"],
is_active=False)
        return context
```

```
    def post(self, request, lead_id):              # state changes only on
POST
        lead = get_object_or_404(Lead, pk=lead_id, is_active=False)
        Lead.objects.filter(pk=lead.pk).update(is_active=True,
updated_at=timezone.now())
        messages.success(request, f"{lead.name} restored.")
        return redirect(lead)


class LeadBulkActionView(FormView):                                        #
22-23, 26
    form_class = LeadBulkForm
    http_method_names = ["post"]

    def form_valid(self, form):
        qs = form.cleaned_data["leads"]               # a queryset of the
ticked leads
        action, now = form.cleaned_data["action"], timezone.now()
        if action == "contacted":
            n = qs.update(status=Lead.Status.CONTACTED, updated_at=now)
        elif action == "assign":
            n = qs.update(assigned_to=form.cleaned_data["agent"],
updated_at=now)
        else:
            n = qs.update(is_active=False, updated_at=now)
        messages.success(self.request, f"{n} lead{'s' if n != 1 else ''}
updated.")
        return redirect(safe_next(self.request,
default=reverse("leads:list")))

    def form_invalid(self, form):
        errors = "; ".join(e for errs in form.errors.values() for e in errs)
        messages.error(self.request, f"Bulk action failed: {errors}")
        return redirect(safe_next(self.request,
default=reverse("leads:list")))


class LeadWorkspaceView(PageTitleMixin, TemplateView):                     #
33
    template_name = "leads/lead_workspace.html"
    page_title = "Workspace"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.lead = get_object_or_404(Lead, pk=kwargs["lead_id"],
is_active=True)
```

```
    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.setdefault("lead_form", LeadForm(instance=self.lead,
prefix="lead", user=self.request.user))
        context.setdefault("call_form", LogCallForm(prefix="call"))
        context["lead"] = self.lead
        return context

    def post(self, request, *args, **kwargs):
        if "save_lead" in request.POST:
            form = LeadForm(request.POST, instance=self.lead, prefix="lead",
user=request.user)
            if form.is_valid():
                form.save()
                messages.success(request, "Lead saved.")
                return redirect(request.path)
            return
self.render_to_response(self.get_context_data(lead_form=form))
        form = LogCallForm(request.POST, prefix="call")
        if form.is_valid():
            log_call(self.lead, request.user, **form.cleaned_data)
            messages.success(request, "Call logged.")
            return redirect(request.path)
        return self.render_to_response(self.get_context_data(call_form=form))
```

```
# LeadDetailView.get_context_data — 18, 31 (add)
from django.contrib.contenttypes.models import ContentType
from core.models import ActivityLog

if (lead.status == Lead.Status.NEW and lead.last_contacted_at is None
        and lead.created_at < timezone.now() - timedelta(days=7)):
    messages.info(self.request, "This lead has waited over 7 days without a
call.")   # shown in this same render
context["activity"] = ActivityLog.objects.filter(
    content_type=ContentType.objects.get_for_model(Lead),
object_id=str(lead.pk),
).select_related("actor")[:10]
context["bulk_form"] = LeadBulkForm()          # 23: agent dropdown for the
reassign box

# LeadListView.get_context_data — 22 (add)
context["bulk_form"] = LeadBulkForm()
```

```
# leads/urls.py (today's routes)
path("new/", views.LeadCreateView.as_view(), name="new"),
path("bulk/", views.LeadBulkActionView.as_view(), name="bulk"),
path("<int:lead_id>/edit/", views.LeadUpdateView.as_view(), name="edit"),
path("<int:lead_id>/status/edit/", views.LeadStatusView.as_view(),
name="status_edit"),
path("<int:lead_id>/log-call/", views.LogCallView.as_view(),
name="log_call"),
path("<int:lead_id>/convert/", views.ConvertLeadView.as_view(),
name="convert"),
path("<int:lead_id>/archive/", views.LeadArchiveView.as_view(),
name="archive"),
path("<int:lead_id>/restore/", views.LeadRestoreView.as_view(),
name="restore"),
path("<int:lead_id>/delete/confirm/", views.LeadConfirmDeleteView.as_view(),
name="delete_confirm"),
path("<int:lead_id>/workspace/", views.LeadWorkspaceView.as_view(),
name="workspace"),
```

Templates (3, 7, 9, 17, 19, 21–23, 30)

```
<!-- templates/base.html — 7: above {% block content %} -->
{% if messages %}
<ul class="messages">
  {% for message in messages %}
    <li class="message {{ message.tags }}">{{ message }}</li>
  {% endfor %}
</ul>
{% endif %}
```

```
.messages { list-style: none; padding: 0; margin: 0 0 1rem; }
.message { padding: .6rem .9rem; border-radius: 8px; margin-bottom: .5rem; }
.message.success { background: #dcfce7; }
.message.info { background: #dbeafe; }
.message.warning { background: #fef9c3; }
.message.danger { background: #fee2e2; }
```

```
# settings.py — 17
from django.contrib.messages import constants as message_constants
MESSAGE_TAGS = {message_constants.ERROR: "danger"}
# extra tags: messages.success(request, "...", extra_tags="sticky") →
class="message sticky success"
```

```
<!-- leads/lead_form.html — 9: second submit button -->
<button type="submit">Save</button>
<button type="submit" name="_addanother">Save and add another</button>
```

```
<!-- leads/templates/leads/lead_confirm_delete.html — 3, 19 -->
{% extends "base.html" %}
{% block content %}
<h1>Delete {{ object.name }}?</h1>
<p>This also deletes {{ call_count }} call log{{ call_count|pluralize }}.
   {% if is_client %}The client record stays but loses its link to this lead.
{% endif %}</p>
<p>Prefer <strong>Archive</strong> unless this lead was created by mistake.
</p>
<form method="post">{% csrf_token %}
  <input type="hidden" name="next" value="{{ request.GET.next }}">
  <button type="submit" class="danger">Delete permanently</button>
  <a href="{{ object.get_absolute_url }}">Cancel</a>
</form>
{% endblock content %}
```

```
<!-- leads/lead_detail.html — 21, 23, 31 -->
<div class="actions">
  <a href="{% url 'leads:edit' lead.pk %}">Edit</a>
  <a href="{% url 'leads:status_edit' lead.pk %}">Change status</a>
  {% if not lead.client %}<a href="{% url 'leads:convert' lead.pk %}">Convert
to client</a>{% endif %}
  <form method="post" action="{% url 'leads:archive' lead.pk %}"
class="inline">{% csrf_token %}
    <button type="submit">Archive</button>
  </form>
</div>

<form method="post" action="{% url 'leads:bulk' %}" class="inline">{%
csrf_token %}
  <input type="hidden" name="leads" value="{{ lead.pk }}">
  <input type="hidden" name="action" value="assign">
  <input type="hidden" name="next" value="{{ request.path }}">
  {{ bulk_form.agent }} <button type="submit">Reassign</button>
</form>

<h2>Activity</h2>
<ul>{% for a in activity %}<li>{{ a.created_at|date:"d M H:i" }} · {{
a.actor|default:"system" }} {{ a.verb }} {{ a.changes|join:", " }}</li>{%
empty %}<li>—</li>{% endfor %}</ul>
```

`{% if not lead.client %}` is safe: the missing reverse one-to-one raises a `DoesNotExist`, which templates treat as empty.

```
<!-- leads/lead_list.html — 22: bulk bar above the table -->
<form id="bulk-form" method="post" action="{% url 'leads:bulk' %}">{%
csrf_token %}
  <input type="hidden" name="next" value="{{ request.get_full_path }}">
  {{ bulk_form.action }} {{ bulk_form.agent }}
  <button type="submit">Apply to selected</button>
</form>

<!-- _lead_rows.html — first cell of each row; form= ties the checkbox to the
bulk form above -->
<td><input type="checkbox" name="leads" value="{{ lead.pk }}" form="bulk-
form"></td>
```

```
<!-- leads/templates/leads/lead_restore.html — 30 -->
{% extends "base.html" %}
{% block content %}
<h1>Restore {{ lead.name }}?</h1>
<form method="post">{% csrf_token %}<button type="submit">Restore</button>
</form>
{% endblock content %}
```

The Undo link survives the redirect because `format_html` returns a safe string and Django's message storage keeps it marked safe (30).

Clients (13–14, 20, 29)

```
# clients/views.py
from django.contrib import messages
from django.db import transaction
from django.db.models import F, ProtectedError
from django.shortcuts import get_object_or_404, redirect
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, DeleteView, UpdateView

from .forms import DepositForm, TradingAccountFormSet
from .models import Client, TradingAccount


class ClientUpdateView(PageTitleMixin, UpdateView):
# 13
    model = Client
    slug_field = "ref"
```

```
    slug_url_kwarg = "ref"
    fields = ["name", "email", "account_type", "kyc_status", "broker"]
    template_name = "clients/client_form.html"
    page_title = "Edit client"

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context.setdefault("formset",
TradingAccountFormSet(instance=self.object))
        return context

    def post(self, request, *args, **kwargs):
        self.object = self.get_object()
        form = self.get_form()
        formset = TradingAccountFormSet(request.POST, instance=self.object)
        if form.is_valid() and formset.is_valid():
            with transaction.atomic():             # both or neither
                self.object = form.save()
                formset.save()
            messages.success(request, "Client saved.")
            return redirect(self.object)
        return self.render_to_response(self.get_context_data(form=form,
formset=formset))


class DepositCreateView(PageTitleMixin, CreateView):
# 14, 29
    form_class = DepositForm
    template_name = "clients/deposit_form.html"
    page_title = "New deposit"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, **kwargs)
        self.account =
get_object_or_404(TradingAccount.objects.select_related("client"),
number=kwargs["number"])

    def form_valid(self, form):
        form.instance.account = self.account
        with transaction.atomic():
            response = super().form_valid(form)
            TradingAccount.objects.filter(pk=self.account.pk).update(
                balance=F("balance") + form.instance.amount,
            )
        messages.success(self.request, f"Deposit of
{form.instance.amount:,.2f} added to {self.account.number}.")
```

```
        return response

    def get_success_url(self):
        return self.account.client.get_absolute_url()

    def get_context_data(self, **kwargs):
        return {**super().get_context_data(**kwargs), "account":
self.account}


class ClientDeleteView(PageTitleMixin, DeleteView):
# 20
    model = Client
    slug_field = "ref"
    slug_url_kwarg = "ref"
    template_name = "clients/client_confirm_delete.html"
    success_url = reverse_lazy("clients:list")
    page_title = "Delete client"

    def form_valid(self, form):
        try:
            response = super().form_valid(form)    # self.object.delete()
        except ProtectedError:
            messages.error(self.request, "This client has deposits and can't
be deleted. Deactivate them instead.")
            return redirect(self.object)
        messages.success(self.request, "Client deleted.")
        return response

# clients/urls.py
path("<uuid:ref>/edit/", views.ClientUpdateView.as_view(), name="edit"),
path("<uuid:ref>/delete/", views.ClientDeleteView.as_view(), name="delete"),
path("accounts/<str:number>/deposits/new/",
views.DepositCreateView.as_view(), name="deposit_new"),
```

```
<!-- clients/templates/clients/client_form.html — 13 -->
{% extends "base.html" %}
{% block content %}
<h1>Edit {{ object.name }}</h1>
<form method="post">{% csrf_token %}
  {{ form.as_div }}
  <h2>Trading accounts</h2>
  {{ formset.management_form }}
  {{ formset.non_form_errors }}
  {% for f in formset %}<fieldset>{{ f.as_div }}</fieldset>{% endfor %}
  <button type="submit">Save</button>
</form>
{% endblock content %}
```

Tags (15, 34)

```
# leads/views.py
from django.views.generic import ListView
from .forms import TagForm
from .models import Tag


class TagFormMixin(SuccessMessageMixin):
    model = Tag
    form_class = TagForm
    template_name = "leads/tag_form.html"
    success_url = "/leads/tags/?saved={id}"            # 34: formatted with
the saved object's fields
    success_message = "Tag %(name)s saved."


class TagListView(ListView):
    model = Tag                                         # template:
leads/tag_list.html, context: tag_list


class TagCreateView(TagFormMixin, CreateView):
    pass


class TagUpdateView(TagFormMixin, UpdateView):
    pass


class TagDeleteView(SuccessMessageMixin, DeleteView):
```

```
    model = Tag
    template_name = "leads/tag_confirm_delete.html"
    success_url = reverse_lazy("leads:tags")
    success_message = "Tag deleted."

# leads/urls.py
path("tags/", views.TagListView.as_view(), name="tags"),
path("tags/new/", views.TagCreateView.as_view(), name="tag_new"),
path("tags/<int:pk>/edit/", views.TagUpdateView.as_view(), name="tag_edit"),
path("tags/<int:pk>/delete/", views.TagDeleteView.as_view(),
name="tag_delete"),
```

```
<!-- leads/templates/leads/tag_list.html -->
{% extends "base.html" %}
{% block content %}
<h1>Tags</h1>
<a href="{% url 'leads:tag_new' %}">+ New tag</a>
<ul>
  {% for tag in tag_list %}
    <li><span class="badge" style="background:{{ tag.color }};color:#fff">{{
tag.name }}</span>
        <a href="{% url 'leads:tag_edit' tag.pk %}">Edit</a> · <a href="{%
url 'leads:tag_delete' tag.pk %}">Delete</a></li>
  {% endfor %}
</ul>
{% endblock content %}
```

Workspace template (33)

```
<!-- leads/templates/leads/lead_workspace.html -->
{% extends "base.html" %}
{% block content %}
<h1>{{ lead.name }}</h1>
<div class="cards">
  <form method="post" class="card">{% csrf_token %}
    {{ lead_form.as_div }}                  {# inputs are named lead-name,
lead-phone, ... #}
    <button type="submit" name="save_lead">Save lead</button>
  </form>
  <form method="post" class="card">{% csrf_token %}
    {{ call_form.as_div }}                  {# call-outcome, call-
duration_seconds, ... #}
    <button type="submit" name="log_call">Log call</button>
  </form>
</div>
{% endblock content %}
```

Tests (37–40)

```
# leads/tests.py
from datetime import timedelta

from clients.models import Client, Deposit, TradingAccount


class EditingViewTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.lead = Lead.objects.create(name="Edit Me", phone="971509990701")

    def test_create_redirects_with_message(self):                   # 37
        response = self.client.post(
            reverse("leads:new"),
            {"name": "new person", "phone": "0509990702", "source":
"google"},
            follow=True,
        )
        lead = Lead.objects.get(phone="971509990702")
        self.assertRedirects(response, lead.get_absolute_url())
        self.assertTrue(any("New Person created" in str(m) for m in
response.context["messages"]))

    def test_stale_edit_rejected(self):                             # 38
```

```
        stale = (self.lead.updated_at - timedelta(seconds=5)).isoformat()
        response = self.client.post(reverse("leads:edit", args=
[self.lead.pk]), {
            "name": "Edit Me", "phone": self.lead.phone, "source": "manual",
"loaded_at": stale,
        })
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Someone else changed this lead")

    def test_protected_client_delete(self):                          # 39
        rich = Client.objects.create(name="Rich", email="r@x.test",
account_number="7100001")
        account = TradingAccount.objects.create(client=rich,
number="7100001")
        Deposit.objects.create(account=account, amount=500, method="bank")
        response = self.client.post(reverse("clients:delete", args=
[rich.ref]), follow=True)
        self.assertTrue(Client.objects.filter(pk=rich.pk).exists())
        self.assertTrue(any("can't be deleted" in str(m) for m in
response.context["messages"]))

    def test_bulk_mark_contacted(self):                              # 40
        other = Lead.objects.create(name="Bulk Two", phone="971509990703")
        self.client.post(reverse("leads:bulk"), {"leads": [self.lead.pk,
other.pk], "action": "contacted"})
        self.assertEqual(
            Lead.objects.filter(pk__in=[self.lead.pk, other.pk],
status=Lead.Status.CONTACTED).count(), 2,
        )
```

Day 15 — Authentication

Goal: the whole CRM sits behind a login. Accounts are created by admins only — no sign-up page — and new staff get an invite email to set their own password. Agents can log in with username or email; repeated failures lock the account for 15 minutes.

After exercise 5, every page needs a logged-in user, so earlier view tests need a login too (solution 5 gives a `LoggedInTestCase` base).

A. Drills (1–10) — Django's auth views

- [ ] **1.** Include `django.contrib.auth.urls` at `/accounts/`; `registration/login.html`; set `LOGIN_URL`, `LOGIN_REDIRECT_URL`, `LOGOUT_REDIRECT_URL`.
- [ ] **2.** Sign-out as a POST form (Django 5+ refuses GET logout).
- [ ] **3.** `base.html`: signed-in user's name + role and Sign out; hide the sidebar when signed out.
- [ ] **4.** Protect one function view with `@login_required` and one CBV with `LoginRequiredMixin`.
- [ ] **5.** Django 5.1 `LoginRequiredMiddleware`: everything requires login; mark health, robots.txt and the API-key endpoints `login_not_required`. Fix the old tests.
- [ ] **6.** Check `?next=` brings you back to the page you asked for.
- [ ] **7.** In the shell: `authenticate()` with right and wrong passwords; `check_password()`.
- [ ] **8.** Password change: templates for the form and done pages; link from the user menu.
- [ ] **9.** Password reset: console email backend, the 4 page templates, and a custom email body/subject.
- [ ] **10.** `AUTH_PASSWORD_VALIDATORS`: minimum length 10.

B. Core (11–20) — admin-created accounts

- [ ] **11.** `invite_staff_user(username, email, role, ...)`: creates the user with an unusable password and emails a set-password link.
- [ ] **12.** Admin action "Send set-password invite" on users.
- [ ] **13.** Separate invite email templates (subject + body) from the normal reset email.
- [ ] **14.** `EmailOrUsernameBackend`: log in with either.
- [ ] **15.** Confirm a deactivated user (`is_active=False`) can't log in.
- [ ] **16.** `LoginEvent` model + `user_logged_in` / `user_login_failed` signal receivers (username, success, IP).

- [ ] **17.** Lockout: 5 failures in 15 minutes blocks that username (`CRMAuthenticationForm.clean`).
- [ ] **18.** `CRMLoginView(LoginView)`: custom form, `redirect_authenticated_user=True`.
- [ ] **19.** Sessions: 8-hour cookie by default; "Keep me signed in" stretches it to 14 days, unticked expires at browser close.
- [ ] **20.** Profile page `/team/me/`: edit name, email, extension; last 10 sign-ins; change-password link.

C. Nexora CRM (21–30)

- [ ] **21.** Use `request.user` as the acting agent: calls you log are yours.
- [ ] **22.** "My leads" at `/leads/mine/` — the lead list filtered to `request.user`.
- [ ] **23.** "My queue" at `/team/me/queue/` reusing the Day 12 agent queue.
- [ ] **24.** Dashboard greeting plus personal numbers: my open leads, my calls today.
- [ ] **25.** Role-based landing page after login: agents → my queue, managers/admins → reports.
- [ ] **26.** JSON endpoints answer `401` JSON instead of redirecting to the login page.
- [ ] **27.** Remove `csrf_exempt` from the browser-facing POST endpoints (status, delete, quick-add, create).
- [ ] **28.** Prove changing your password signs out your other sessions but keeps the current one.
- [ ] **29.** Admin user form (Django 5.1+): create users with password login disabled, then send the invite.
- [ ] **30.** Sidebar by role: agents don't see Reports or Brokers; managers see everything.

D. Challenge (31–40)

- [ ] **31.** `must_change_password` flag + middleware that sends flagged users to the password-change page until they comply.
- [ ] **32.** Custom password validator: no "nexora", no username.
- [ ] **33.** Argon2 as the default hasher; watch an old hash upgrade on next login.
- [ ] **34.** Shell: `make_password`, `set_unusable_password`, `has_usable_password`, `validate_password`.
- [ ] **35.** Build a set-password link by hand with `default_token_generator` + `urlsafe_base64_encode`; check when the token stops working.
- [ ] **36.** Admin action "Unlock" that clears recent failed sign-ins.
- [ ] **37.** Test: anonymous visit to `/leads/` redirects to login with `next`.

- [ ] **38.** Test: login with email works.
- [ ] **39.** Test: invite sends one email with a reset link; the user has no usable password.
- [ ] **40.** Test: 5 wrong passwords lock the account even for the right password.

Solutions

Settings (1, 5, 9–10, 14, 19, 26, 31–33)

```
# django_project/settings.py
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "core.middleware.CRMLoginRequiredMiddleware",        # 5, 26 — must come
after AuthenticationMiddleware
    "core.middleware.ForcePasswordChangeMiddleware",     # 31
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]

LOGIN_URL = "login"                       # 1 — URL names work here
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"

AUTHENTICATION_BACKENDS = ["accounts.backends.EmailOrUsernameBackend"]     #
14

AUTH_PASSWORD_VALIDATORS = [                                               #
10, 32
    {"NAME":
"django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME":
"django.contrib.auth.password_validation.MinimumLengthValidator", "OPTIONS":
{"min_length": 10}},
    {"NAME":
"django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME":
"django.contrib.auth.password_validation.NumericPasswordValidator"},
    {"NAME": "accounts.validators.NoCompanyNameValidator"},
]

PASSWORD_HASHERS = [                                                       #
```

```
33
    "django.contrib.auth.hashers.Argon2PasswordHasher",     # first = used
for new hashes
    "django.contrib.auth.hashers.PBKDF2PasswordHasher",     # still verifies
(and upgrades) old ones
    "django.contrib.auth.hashers.PBKDF2SHA1PasswordHasher",
    "django.contrib.auth.hashers.ScryptPasswordHasher",
]

SESSION_COOKIE_AGE = 60 * 60 * 8                                        #
19: one working day

EMAIL_BACKEND = "django.core.mail.backends.console.EmailBackend"         # 9:
emails print in the terminal
DEFAULT_FROM_EMAIL = "Nexora CRM <no-reply@nexora.test>"
```

```
python -m pip install "django[argon2]"      # 33
```

Middleware (5, 26, 31)

```
# core/middleware.py
from django.contrib.auth.middleware import LoginRequiredMiddleware
from django.http import JsonResponse
from django.shortcuts import redirect


class CRMLoginRequiredMiddleware(LoginRequiredMiddleware):              # 5,
26
    def handle_no_permission(self, request, view_func):
        if "/api/" in request.path or "application/json" in
request.headers.get("Accept", ""):
            return JsonResponse({"error": "authentication required"},
status=401)
        return super().handle_no_permission(request, view_func)    # redirect
to LOGIN_URL?next=...


class ForcePasswordChangeMiddleware:                                    # 31
    ALLOWED = {"password_change", "password_change_done", "logout"}

    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        return self.get_response(request)
```

```
    def process_view(self, request, view_func, view_args, view_kwargs):
        user = request.user
        if (user.is_authenticated and getattr(user, "must_change_password",
False)
                and request.resolver_match.url_name not in self.ALLOWED):
            return redirect("password_change")
        return None
```

5 — public endpoints and old tests.

```
from django.contrib.auth.decorators import login_not_required    # Django
5.1+

# core/urls.py
path("health/", login_not_required(views.HealthView.as_view()),
name="health"),
path("robots.txt",
login_not_required(TemplateView.as_view(template_name="robots.txt",
content_type="text/plain"))),

# leads/api_urls.py — machine clients authenticate with X-API-Key instead
path("leads/", login_not_required(views.ApiLeadListView.as_view()),
name="list"),
path("leads/<int:lead_id>/",
login_not_required(views.ApiLeadDetailView.as_view()), name="detail"),
```

```
# core/testing.py
from django.contrib.auth import get_user_model
from django.test import TestCase


class LoggedInTestCase(TestCase):
    role = "manager"

    @classmethod
    def setUpTestData(cls):
        super().setUpTestData()
        cls.staff = get_user_model().objects.create_user(f"test_{cls.role}",
password="x", role=cls.role)

    def setUp(self):
        super().setUp()
        self.client.force_login(self.staff)    # skips the password check
entirely
```

Every earlier test class that calls `self.client` (including the old `SimpleTestCase` ones — sessions now need the database) inherits from `LoggedInTestCase` instead. Classes with their own `setUpTestData` must call `super().setUpTestData()` first. The CSRF test still gets 403: CSRF runs before the login check.

Accounts models, backend, validator, signals (11, 14, 16, 31–32)

```
# accounts/models.py (add)
from django.conf import settings


class CustomUser(AbstractUser):
    ...
    must_change_password = models.BooleanField(default=False)          # 31


class LoginEvent(models.Model):                                         # 16
    user = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True,
                             on_delete=models.CASCADE,
related_name="login_events")
    username = models.CharField(max_length=150)       # what was typed —
failures may not match a user
    success = models.BooleanField()
    ip = models.GenericIPAddressField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created_at"]
        indexes = [models.Index(fields=["username", "created_at"],
name="login_user_created_idx")]
```

```
# accounts/backends.py                                                  # 14
from django.contrib.auth import get_user_model
from django.contrib.auth.backends import ModelBackend
from django.db.models import Q


class EmailOrUsernameBackend(ModelBackend):
    def authenticate(self, request, username=None, password=None, **kwargs):
        if username is None or password is None:
            return None
        User = get_user_model()
        try:
            user = User.objects.get(Q(username__iexact=username) |
Q(email__iexact=username))
        except (User.DoesNotExist, User.MultipleObjectsReturned):
            User().set_password(password)      # spend the same time as a
real check (timing attacks)
            return None
        if user.check_password(password) and
self.user_can_authenticate(user):   # inactive → False (15)
            return user
        return None
```

```
# accounts/validators.py                                                # 32
from django.core.exceptions import ValidationError


class NoCompanyNameValidator:
    def validate(self, password, user=None):
        lowered = password.lower()
        if "nexora" in lowered:
            raise ValidationError("Your password can't contain the company
name.", code="password_has_company")
        if user is not None and user.get_username() and
user.get_username().lower() in lowered:
            raise ValidationError("Your password can't contain your
username.", code="password_has_username")

    def get_help_text(self):
        return "Your password can't contain “nexora” or your username."
```

Validators run in forms and `validate_password()`, not in `set_password()` — the seeded `nexora123` passwords still exist until users change them.

```
# accounts/signals.py                                                    # 16
from django.contrib.auth.signals import user_logged_in, user_login_failed
from django.dispatch import receiver

from .models import LoginEvent


def _ip(request):
    return request.META.get("REMOTE_ADDR") if request else None


@receiver(user_logged_in)
def record_login(sender, request, user, **kwargs):
    LoginEvent.objects.create(user=user, username=user.get_username(),
success=True, ip=_ip(request))


@receiver(user_login_failed)
def record_failure(sender, credentials, request=None, **kwargs):
    LoginEvent.objects.create(username=credentials.get("username", "")[:150],
success=False, ip=_ip(request))


# accounts/apps.py — connect the receivers
class AccountsConfig(AppConfig):
    default_auto_field = "django.db.models.BigAutoField"
    name = "accounts"

    def ready(self):
        from . import signals  # noqa: F401
```

```
python manage.py makemigrations accounts && python manage.py migrate
```

Forms (17, 19–20, 29)

```
# accounts/forms.py
from datetime import timedelta

from django import forms
from django.contrib.auth.forms import AdminUserCreationForm,
AuthenticationForm, UserChangeForm
from django.utils import timezone

from .models import CustomUser, LoginEvent
```

```
LOCK_ATTEMPTS = 5
LOCK_WINDOW = timedelta(minutes=15)


class CustomUserCreationForm(AdminUserCreationForm):     # 29: Django 5.1+
adds "password-based auth: disabled"
    class Meta(AdminUserCreationForm.Meta):
        model = CustomUser
        fields = ("username", "email", "role", "team_lead")


class CRMAuthenticationForm(AuthenticationForm):                        # 17,
19
    remember_me = forms.BooleanField(required=False, label="Keep me signed in
for 14 days")

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.fields["username"].label = "Username or email"

    def clean(self):
        username = self.cleaned_data.get("username", "")
        recent_failures = LoginEvent.objects.filter(
            username__iexact=username, success=False,
created_at__gte=timezone.now() - LOCK_WINDOW,
        ).count()
        if recent_failures >= LOCK_ATTEMPTS:
            raise forms.ValidationError("Too many failed attempts. Try again
in 15 minutes.", code="locked")
        return super().clean()       # authenticate(); a failure fires
user_login_failed → LoginEvent


class ProfileForm(forms.ModelForm):                                      # 20
    class Meta:
        model = CustomUser
        fields = ["first_name", "last_name", "email", "phone_ext"]

    def clean_email(self):
        email = self.cleaned_data["email"].strip().lower()
        if email and
CustomUser.objects.filter(email__iexact=email).exclude(pk=self.instance.pk).e
xists():
            raise forms.ValidationError("That email belongs to another
user.")   # email login needs uniqueness
        return email
```

Invite service (11, 13)

```
# accounts/services.py
from django.contrib.auth import get_user_model
from django.contrib.auth.forms import PasswordResetForm
from django.db import transaction


class InviteForm(PasswordResetForm):
    def get_users(self, email):
        # the default skips users without a usable password — exactly the
people we invite
        return get_user_model()._default_manager.filter(email__iexact=email,
is_active=True)


def send_invite(user, request=None):
    form = InviteForm({"email": user.email})
    form.is_valid()
    form.save(
        request=request,
        domain_override=None if request else "localhost:8000",     # no
request → say where links point
        use_https=request.is_secure() if request else False,
        subject_template_name="registration/invite_subject.txt",
        email_template_name="registration/invite_email.txt",
    )


def invite_staff_user(username, email, role, team_lead=None, first_name="",
request=None):
    with transaction.atomic():
        user = get_user_model().objects.create_user(
            username=username, email=email, role=role, team_lead=team_lead,
first_name=first_name,
        )
        user.set_unusable_password()       # nobody knows a password until
they set one
        user.save(update_fields=["password"])
    send_invite(user, request)
    return user
```

```
# templates/registration/invite_subject.txt
Your Nexora CRM account
```

```
# templates/registration/invite_email.txt
Hi {{ user.get_short_name|default:user.get_username }},

An account has been created for you on Nexora CRM.
Username: {{ user.get_username }}

Set your password here (valid for 3 days):
{{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid
token=token %}
```

Views (18–25, 31)

```
# accounts/views.py (add)
from django.contrib import messages
from django.contrib.auth.mixins import LoginRequiredMixin
from django.contrib.auth.views import LoginView, PasswordChangeView
from django.contrib.messages.views import SuccessMessageMixin
from django.urls import reverse, reverse_lazy
from django.views.generic.edit import UpdateView

from .forms import CRMAuthenticationForm, ProfileForm


class CRMLoginView(LoginView):                                      # 18,
19, 25
    form_class = CRMAuthenticationForm
    redirect_authenticated_user = True        # signed-in users skip the
login page
    template_name = "registration/login.html"

    def form_valid(self, form):
        response = super().form_valid(form)   # calls login(); session key
rotates here
        if form.cleaned_data.get("remember_me"):
            self.request.session.set_expiry(60 * 60 * 24 * 14)
        else:
            self.request.session.set_expiry(0)          # cookie dies when
the browser closes
        return response

    def get_default_redirect_url(self):       # used only when there's no
safe ?next=
        role = getattr(self.request.user, "role", "")
        if role == "agent":
            return reverse("accounts:my_queue")
```

```
        if role in {"manager", "admin"}:
            return reverse("reports:index")
        return super().get_default_redirect_url()


class CRMPasswordChangeView(PasswordChangeView):                        # 31
    def form_valid(self, form):
        response = super().form_valid(form)   # also update_session_auth_hash
→ this session survives

type(self.request.user).objects.filter(pk=self.request.user.pk).update(must_c
hange_password=False)
        messages.success(self.request, "Password changed.")
        return response


class ProfileView(LoginRequiredMixin, PageTitleMixin, SuccessMessageMixin,
UpdateView):   # 20
    form_class = ProfileForm
    template_name = "accounts/profile.html"
    success_url = reverse_lazy("accounts:profile")
    success_message = "Profile saved."
    page_title = "My profile"

    def get_object(self, queryset=None):
        return self.request.user               # no pk in the URL — you can
only edit yourself

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context["logins"] = self.request.user.login_events.all()[:10]
        return context


class MyQueueView(AgentQueueView):                                      # 23
    page_title = "My queue"

    def setup(self, request, *args, **kwargs):
        super().setup(request, *args, pk=request.user.pk, **kwargs)   #
managers get 404 (agents only)

# accounts/urls.py
path("me/", views.ProfileView.as_view(), name="profile"),
path("me/queue/", views.MyQueueView.as_view(), name="my_queue"),
```

```
# django_project/urls.py — 1, 18, 31 (custom views before the include so they
win)
from accounts.views import CRMLoginView, CRMPasswordChangeView

urlpatterns = [
    path("admin/", admin.site.urls),
    path("accounts/login/", CRMLoginView.as_view(), name="login"),
    path("accounts/password_change/", CRMPasswordChangeView.as_view(),
name="password_change"),
    path("accounts/", include("django.contrib.auth.urls")),   # logout,
password_change_done, password_reset*...
    path("team/", include("accounts.urls")),
    ...
]
```

```
# leads/views.py — 21, 22, 4
from django.contrib.auth.mixins import LoginRequiredMixin


class LogCallView(SuccessMessageMixin, FormView):
    ...
    def form_valid(self, form):
        log_call(self.lead, self.request.user, **form.cleaned_data)    # 21:
the caller is whoever is signed in
        return super().form_valid(form)


class MyLeadsView(LeadListView):                                       # 22
    page_title = "My leads"

    def get_queryset(self):
        return super().get_queryset().filter(assigned_to=self.request.user)

# leads/urls.py
path("mine/", views.MyLeadsView.as_view(), name="mine"),

# 4 — explicit protection (works with or without the middleware)
class LeadCreateView(LoginRequiredMixin, JsonFormMixin, AuditMixin, ...):   #
LoginRequiredMixin first
    ...

from django.contrib.auth.decorators import login_required
```

```
@login_required
def quick_add_many(request):
    ...
```

```
# core/views.py — DashboardView.get_context_data (24)
from leads.models import CallLog
from leads.queries import OPEN_STATUSES

user = self.request.user
context.update(
    my_open=Lead.objects.filter(assigned_to=user, is_active=True,
status__in=OPEN_STATUSES).count(),
    my_calls_today=CallLog.objects.filter(agent=user,
started_at__date=timezone.localdate()).count(),
)
```

```
<!-- core/dashboard.html — 24 -->
<h1>Welcome back, {{ user.first_name|default:user.username }}</h1>
<p>{{ my_open }} open lead{{ my_open|pluralize }} · {{ my_calls_today }}
call{{ my_calls_today|pluralize }} today</p>
```

27. Delete `@method_decorator(csrf_exempt, name="dispatch")` from `LeadStatusUpdateView`, `LeadDeleteView` (the JSON one), `QuickAddView` and `create_lead`. Browser forms already send `{% csrf_token %}`; JavaScript callers send the `X-CSRFToken` header (Day 27). Endpoints for machines keep the API key instead.

Templates (1–3, 8–9, 20, 30)

```
<!-- templates/registration/login.html -->
{% extends "base.html" %}
{% block title %}Sign in · {{ block.super }}{% endblock title %}
{% block content %}
<h1>Sign in</h1>
<form method="post">{% csrf_token %}
  {{ form.as_div }}                                  {# username, password,
remember_me + errors #}
  <input type="hidden" name="next" value="{{ next }}">
  <button type="submit">Sign in</button>
</form>
<p><a href="{% url 'password_reset' %}">Forgot your password?</a></p>
{% endblock content %}
```

```
<!-- templates/partials/_navbar.html — 2, 3 -->
{% load static %}
<header class="topbar">
  <img src="{% static 'img/logo.png' %}" alt="Nexora" height="28">
  <span>{{ page_title|default:"Nexora CRM" }}</span>
  {% if user.is_authenticated %}
    <span class="spacer"></span>
    <a href="{% url 'accounts:profile' %}">{{
user.get_full_name|default:user.username }}</a>
    <span class="muted">{{ user.get_role_display }}</span>
    <form method="post" action="{% url 'logout' %}" class="inline">{%
csrf_token %}
      <button type="submit">Sign out</button>   {# GET logout was removed in
Django 5.0 #}
    </form>
  {% endif %}
</header>
```

```
<!-- templates/base.html — sidebar, 3 + 30 -->
<aside class="sidebar">
  <a href="{% url 'home' %}" class="brand">Nexora CRM</a>
  {% if user.is_authenticated %}
    {% with ns=request.resolver_match.namespace
name=request.resolver_match.url_name %}
      <a href="{% url 'dashboard' %}" class="{% if name == 'dashboard'
%}active{% endif %}">Dashboard</a>
      <a href="{% url 'leads:mine' %}">My leads</a>
      <a href="{% url 'leads:list' %}" class="{% if ns == 'leads' %}active{%
endif %}">All leads</a>
      <a href="{% url 'clients:list' %}" class="{% if ns == 'clients'
%}active{% endif %}">Clients</a>
      {% if user.role != "agent" %}
        <a href="{% url 'brokers:list' %}" class="{% if ns == 'brokers'
%}active{% endif %}">Brokers</a>
        <a href="{% url 'reports:index' %}" class="{% if ns == 'reports'
%}active{% endif %}">Reports</a>
      {% endif %}
    {% endwith %}
  {% endif %}
</aside>
```

Hiding a link is not security — Day 16 blocks the views themselves (30).

```
<!-- templates/registration/password_change_form.html — 8 -->
{% extends "base.html" %}
{% block content %}
<h1>Change password</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Change password</button></form>
{% endblock content %}

<!-- templates/registration/password_change_done.html -->
{% extends "base.html" %}
{% block content %}<h1>Password changed</h1><p><a href="{% url 'dashboard'
%}">Back to the dashboard</a></p>{% endblock content %}
```

```
<!-- 9: the four reset pages, all in templates/registration/ -->

<!-- password_reset_form.html -->
{% extends "base.html" %}
{% block content %}
<h1>Reset your password</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Email me a link</button></form>
{% endblock content %}

<!-- password_reset_done.html -->
{% extends "base.html" %}
{% block content %}<h1>Check your inbox</h1><p>If that email belongs to an
account, a link is on its way.</p>{% endblock content %}

<!-- password_reset_confirm.html — also the invite landing page -->
{% extends "base.html" %}
{% block content %}
{% if validlink %}
  <h1>Set a new password</h1>
  <form method="post">{% csrf_token %}{{ form.as_div }}<button
type="submit">Save password</button></form>
{% else %}
  <h1>Link expired</h1><p>Ask your manager to send a new invite.</p>
{% endif %}
{% endblock content %}

<!-- password_reset_complete.html -->
{% extends "base.html" %}
{% block content %}<h1>Password set</h1><p><a href="{% url 'login' %}">Sign
in</a></p>{% endblock content %}
```

```
# templates/registration/password_reset_subject.txt
Reset your Nexora CRM password

# templates/registration/password_reset_email.html   (rendered as the plain-text body)
Hi {{ user.get_short_name|default:user.get_username }},

Someone asked to reset the password for {{ user.get_username }} on Nexora CRM.
If that was you, open this link within 3 days:
{{ protocol }}://{{ domain }}{% url 'password_reset_confirm' uidb64=uid token=token %}

If it wasn't, ignore this email.
```

```
<!-- accounts/templates/accounts/profile.html — 20 -->
{% extends "base.html" %}
{% block content %}
<h1>My profile</h1>
<form method="post">{% csrf_token %}{{ form.as_div }}<button type="submit">Save</button></form>
<p><a href="{% url 'password_change' %}">Change password</a></p>
<h2>Recent sign-ins</h2>
<ul>{% for e in logins %}<li>{{ e.created_at|date:"d M Y H:i" }} · {{ e.ip|default:"?" }}</li>{% empty %}<li>—</li>{% endfor %}</ul>
{% endblock content %}
```

Admin actions (12, 36)

```
# accounts/admin.py — CustomUserAdmin
from .models import LoginEvent
from .services import send_invite

    actions = ["send_invites", "unlock"]

    @admin.action(description="Send set-password invite")
    def send_invites(self, request, queryset):
        users = list(queryset.exclude(email=""))
        for user in users:
            send_invite(user, request)
        self.message_user(request, f"Invites sent to {len(users)} user(s).")

    @admin.action(description="Unlock (clear failed sign-ins)")
    def unlock(self, request, queryset):
        names = list(queryset.values_list("username", flat=True)) + list(
            queryset.exclude(email="").values_list("email", flat=True))     #
people may type either
        n, _ = LoginEvent.objects.filter(username__in=names,
success=False).delete()
        self.message_user(request, f"Cleared {n} failed attempt(s).")
```

Shell (7, 15, 33–35)

```
# python manage.py shell
from django.contrib.auth import authenticate
from django.contrib.auth.hashers import check_password, make_password
from django.contrib.auth.password_validation import validate_password
from django.contrib.auth.tokens import default_token_generator
from django.urls import reverse
from django.utils.encoding import force_bytes
from django.utils.http import urlsafe_base64_encode
from accounts.models import CustomUser

authenticate(username="agent1", password="nexora123")      # 7 → <CustomUser:
Agent1>
authenticate(username="agent1", password="wrong")          # None (and a
failed LoginEvent row)
u = CustomUser.objects.get(username="agent1")
u.check_password("nexora123")                              # True

u8 = CustomUser.objects.get(username="agent8")              # 15
u8.is_active = False
u8.save(update_fields=["is_active"])
```

```
authenticate(username="agent8", password="nexora123")      # None

u.password[:20]                     # 33: 'pbkdf2_sha256$...' before,
'argon2$argon2id$...' after next login

h = make_password("S3cure-pass-2026")                        # 34
check_password("S3cure-pass-2026", h)                       # True
try:
    validate_password("nexora123", user=u)
except Exception as e:
    print(e.messages)               # too short, contains company name, ...

uid = urlsafe_base64_encode(force_bytes(u.pk))              # 35
token = default_token_generator.make_token(u)
reverse("password_reset_confirm", kwargs={"uidb64": uid, "token": token})
default_token_generator.check_token(u, token)               # True
u.set_password("Another-pass-2026"); u.save()
default_token_generator.check_token(u, token)               # False — tokens
die when the password or last_login changes
```

Tests (28, 37–40)

```
# accounts/tests.py
from django.contrib.auth import get_user_model
from django.core import mail
from django.test import Client, TestCase
from django.urls import reverse

from .services import invite_staff_user

PASSWORD = "Str0ng-pass-2026"


class AuthTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.agent = get_user_model().objects.create_user(
            "auth_agent", email="agent@nexora.test", password=PASSWORD,
role="agent",
        )

    def test_anonymous_redirected_to_login(self):                      # 37
        response = self.client.get(reverse("leads:list"))
        self.assertRedirects(response, f"{reverse('login')}?next=/leads/",
fetch_redirect_response=False)
```

```
    def test_login_with_email(self):                                    # 38
        response = self.client.post(reverse("login"), {"username":
"agent@nexora.test", "password": PASSWORD})
        self.assertEqual(response.status_code, 302)
        self.assertEqual(int(self.client.session["_auth_user_id"]),
self.agent.pk)

    def test_invite_email(self):                                        # 39
        invite_staff_user("new_agent", "new@nexora.test", role="agent")
        self.assertEqual(len(mail.outbox), 1)                    # tests use
the in-memory backend
        self.assertIn("/accounts/reset/", mail.outbox[0].body)

self.assertFalse(get_user_model().objects.get(username="new_agent").has_usabl
e_password())

    def test_lockout_after_five_failures(self):                         # 40
        for _ in range(5):
            self.client.post(reverse("login"), {"username": "auth_agent",
"password": "wrong"})
        response = self.client.post(reverse("login"), {"username":
"auth_agent", "password": PASSWORD})
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, "Too many failed attempts")

    def test_password_change_signs_out_other_sessions(self):            # 28
        other = Client()
        self.assertTrue(self.client.login(username="auth_agent",
password=PASSWORD))
        self.assertTrue(other.login(username="auth_agent",
password=PASSWORD))
        self.client.post(reverse("password_change"), {
            "old_password": PASSWORD,
            "new_password1": "Brand-new-pass-77",
            "new_password2": "Brand-new-pass-77",
        })
        self.assertEqual(self.client.get(reverse("dashboard")).status_code,
200)   # this session kept
        self.assertEqual(other.get(reverse("dashboard")).status_code, 302)
# the other one is out
```

```
python manage.py test
git add -A && git commit -m "day 15"
```